



Tìm kiếm cục bộ

Local search

Cơ sở Trí tuệ nhân tạo

Nội dung

- Khái niệm về tìm kiếm cục bộ
- Giải thuật tìm kiếm cục bộ
 - Hill climbing – leo đồi
 - Simulated Annealing – mô phỏng tôi thép
 - Beam search
 - Tabu search
- Giải thuật di truyền

Học liệu

- Đinh Mạnh Tường, *Trí tuệ nhân tạo – Cách tiếp cận hiện đại*, NXB Khoa học kỹ thuật, 2024
 - Chương 5. Tìm kiếm địa phương
- Russell & Norvig, *AI: A Modern Approach*, 4th ed., Pearson 2020
 - Chapter 4. Search in Complex Environments

Bài toán tìm kiếm

- Tìm kiếm mù: duyệt toàn bộ không gian tìm kiếm
- Tìm kiếm “kinh nghiệm”: cần có hàm kinh nghiệm tốt để định hướng
- Cần lưu biên tìm kiếm, đường đi đến đích
- Trong nhiều trường hợp, không đủ nguồn lực hoặc không có phương pháp có tính hệ thống để tìm lời giải tốt nhất
- Bài toán tối ưu tổ hợp: Lời giải là một tổ hợp các biến rời rạc, chỉ quan tâm lời giải tối ưu, không cần quan tâm đường dẫn tới lời giải
 - Ví dụ: N-queen, knapsack problem, TSP,...

Traveling Salesman Person

- Find the shortest Tour traversing all cities once.
- A Solution: Exhaustive Search
(Generate and Test) !!

The number of all tours
is about $(n-1)!/2$

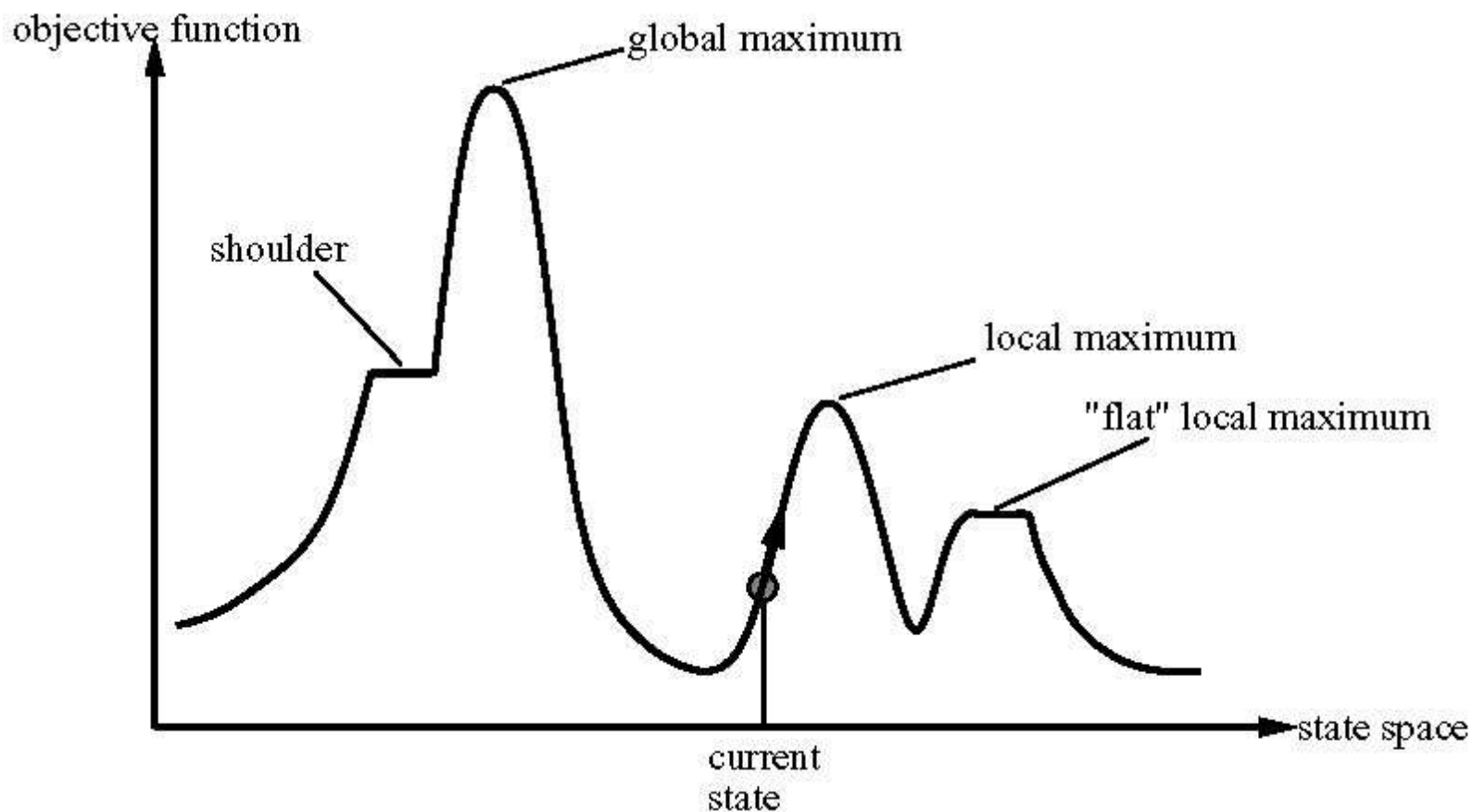
If $n = 36$ the number is about:

566573983193072464833325668761600000000

Not Viable Approach !!



Không gian lời giải



Local search - Tìm kiếm cục bộ

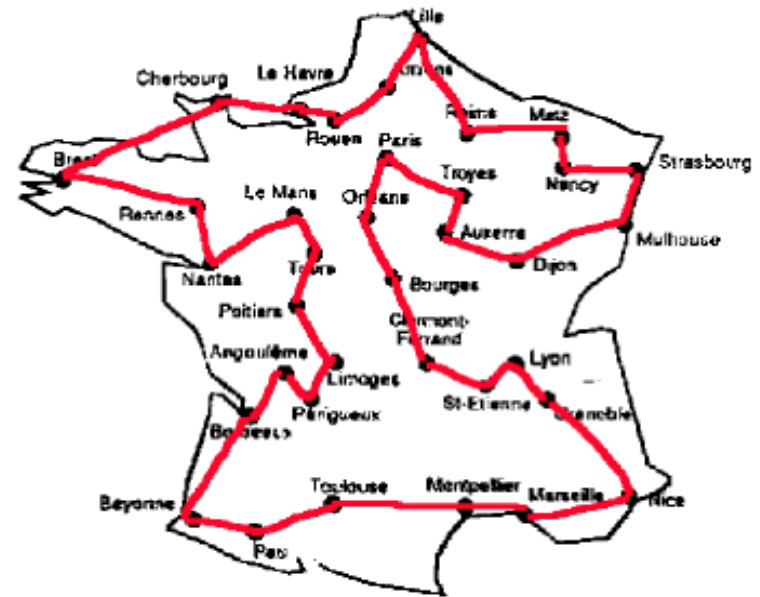
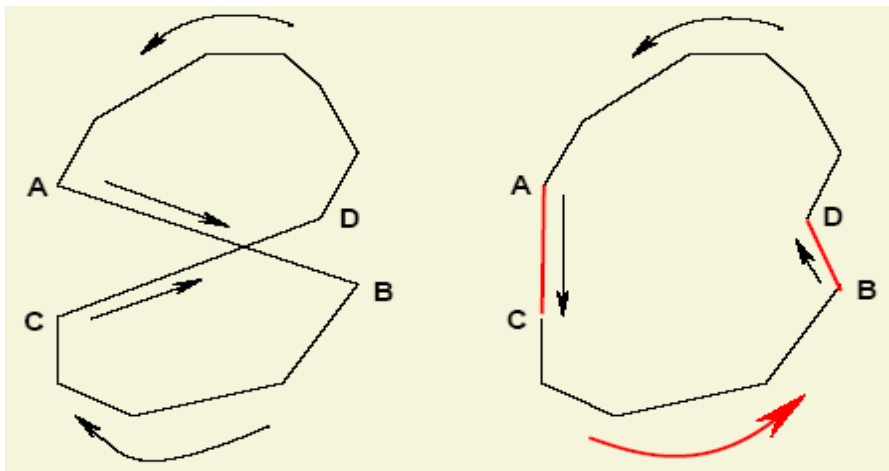
Ý tưởng: xuất phát từ một lời giải, tìm cách biến đổi để sinh ra lời giải tốt hơn



Tiến dần đến lời giải tối ưu

Traveling Salesman Person

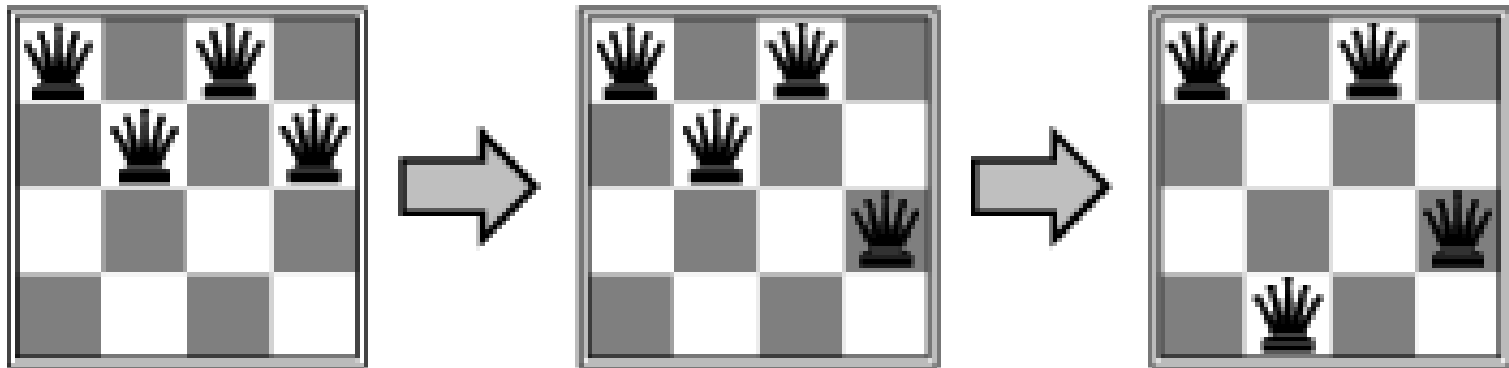
- A Solution: Start from an initial solution and improve using local transformations.



$Z_{opt} = 4441,3\text{Km}$

N-Queen problem

Put N Queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



Initial state ... Improve it ... using local transformations (perturbations)

Hill Climbing (Greedy Local Search)

- Simple, general idea:

- Start wherever
- Repeat: move to the best “neighboring” state (successor state)
- If no neighbors better than current, quit

Complete? No!

Optimal? No!



function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

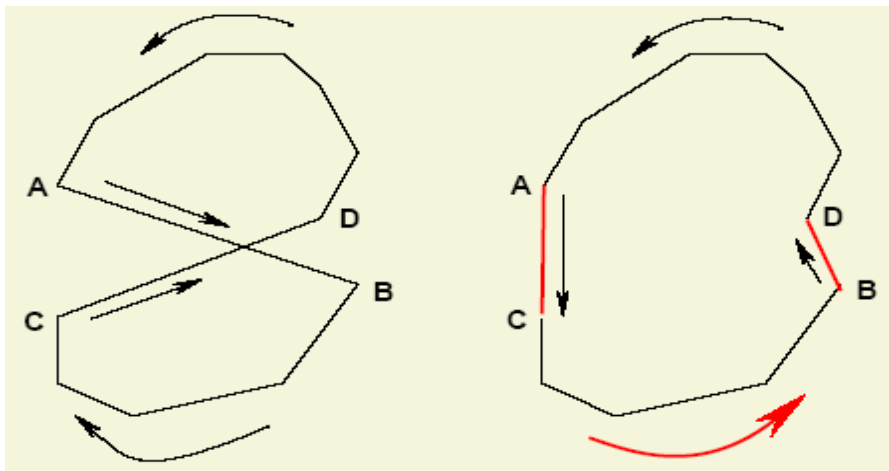
neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

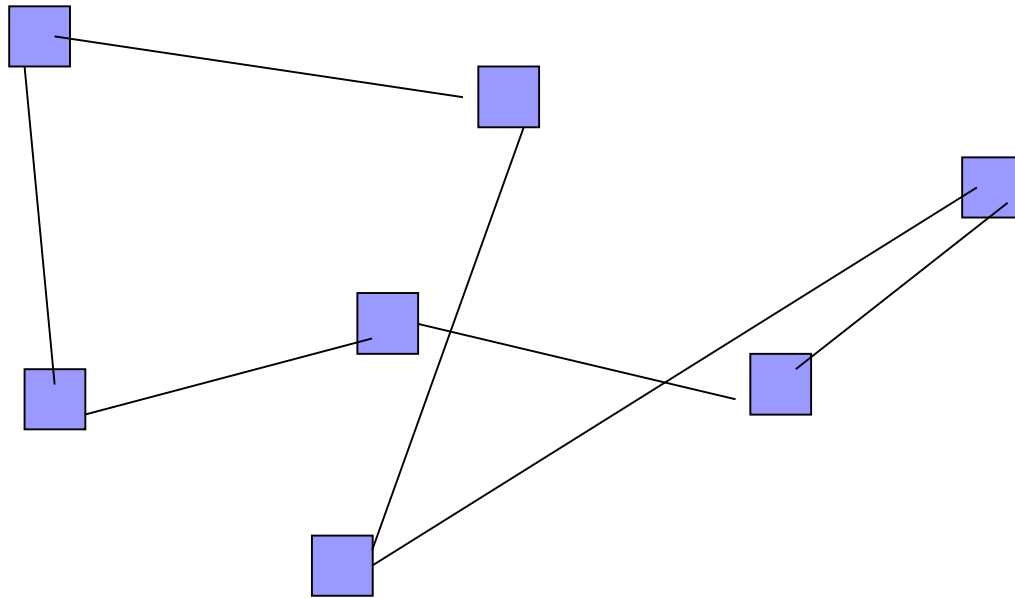
TSP: Neighborhood

- Từ một lời giải, sinh ra lời giải lân cận



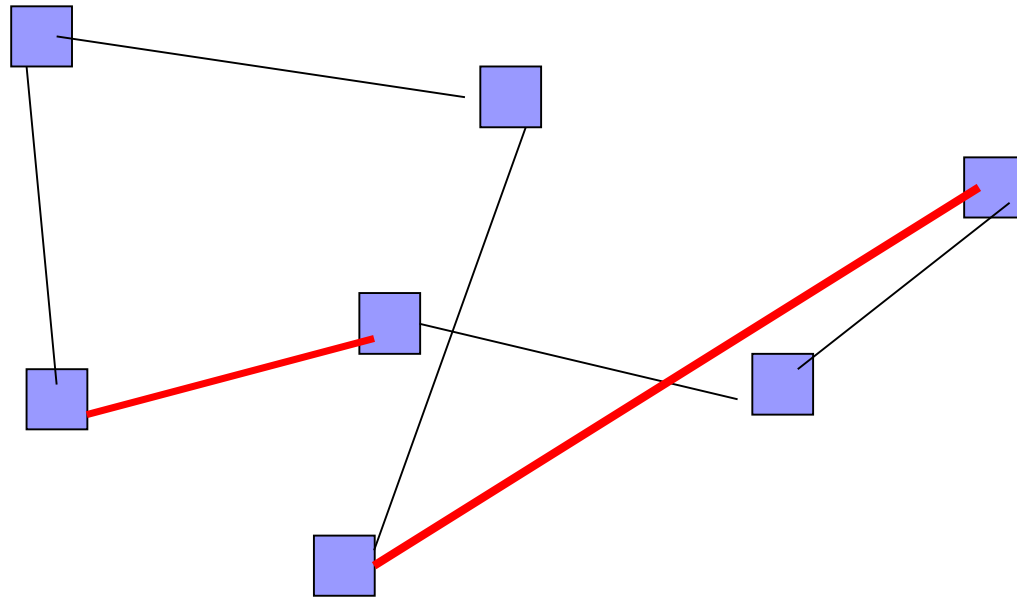
$Z_{opt} = 4441,3\text{Km}$

2-opt mutation for TSP



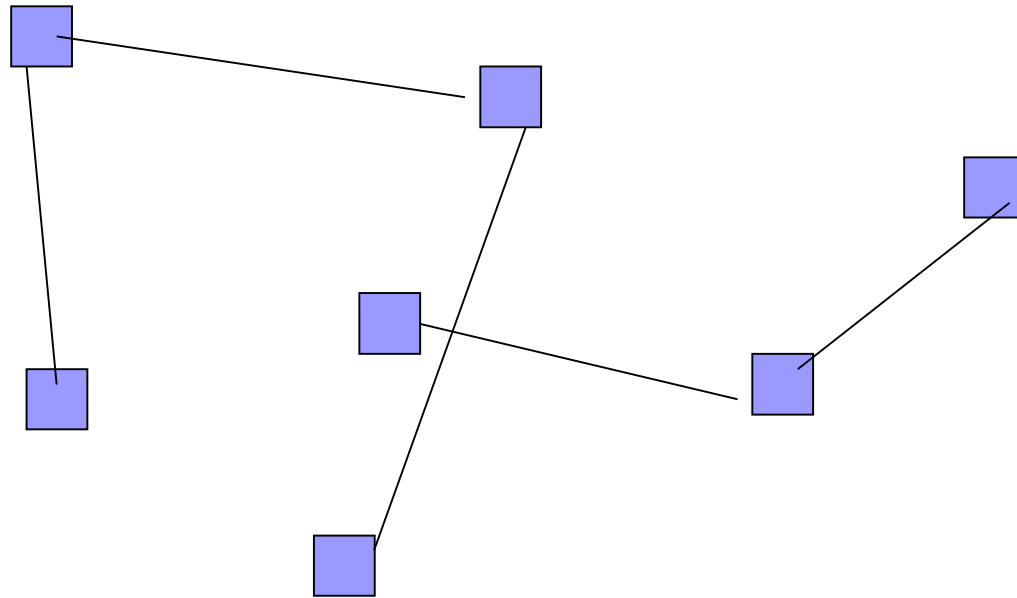
Choose two edges at random

2-opt mutation for TSP



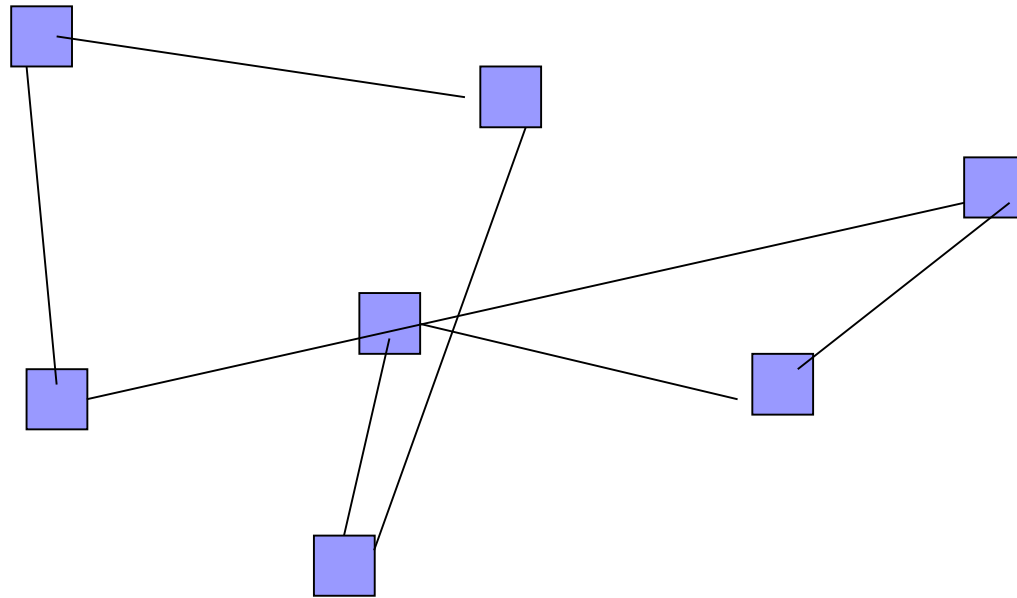
Choose two edges at random

2-opt mutation for TSP



Remove them

2-opt mutation for TSP



Reconnect in a different way (there is only one valid new way)

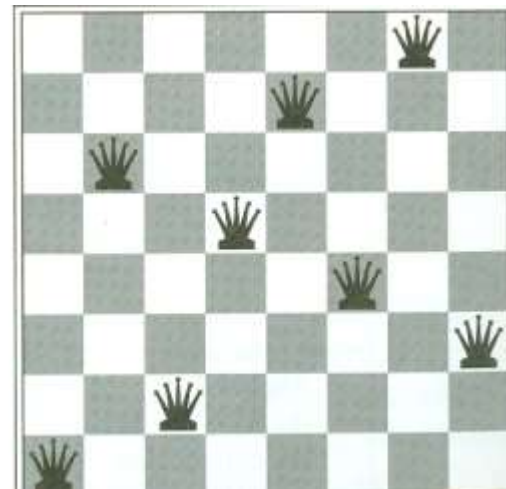
Hill Climbing: Neighborhood

Consider the 8-queen problem:

- The neighborhood of a state is all states generated by moving a single queen to another square in the same column ($8 \times 7 = 56$ next states)
- The objective function $h(s)$ = number of queens that attack each other in state s .

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

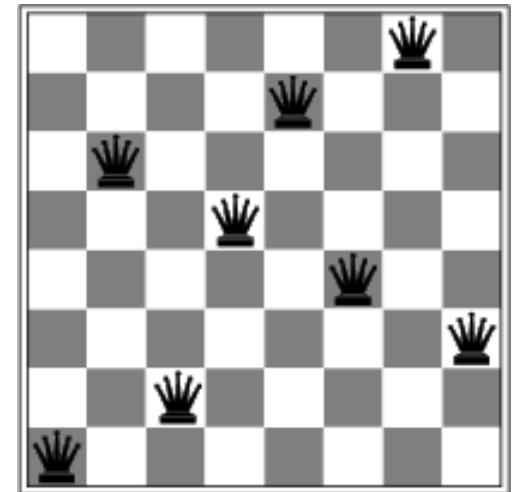
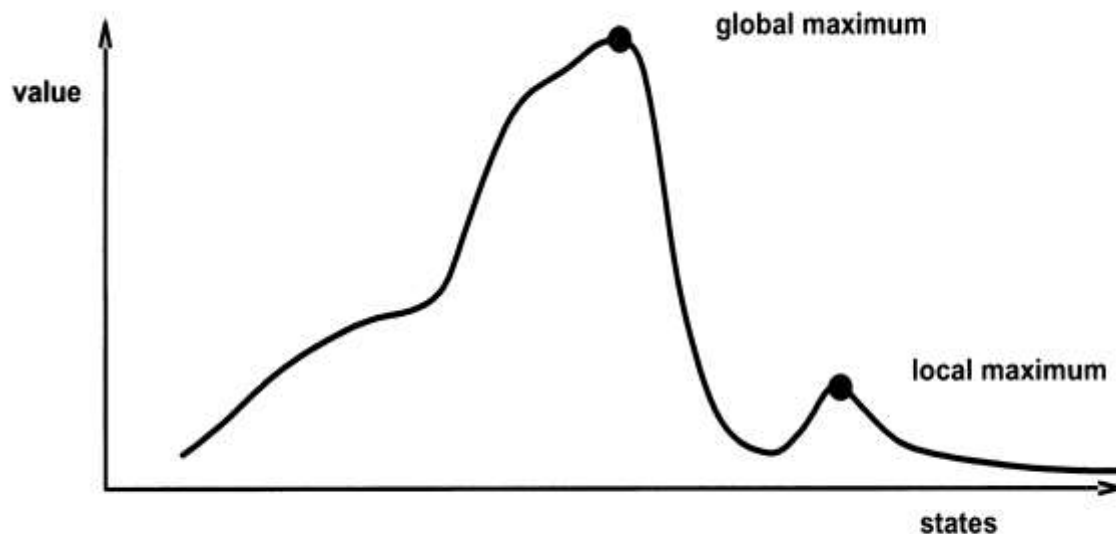
$h(s) = 17$ best next is 12



$h(s)=1$ [local minima]

Hill Climbing Drawbacks

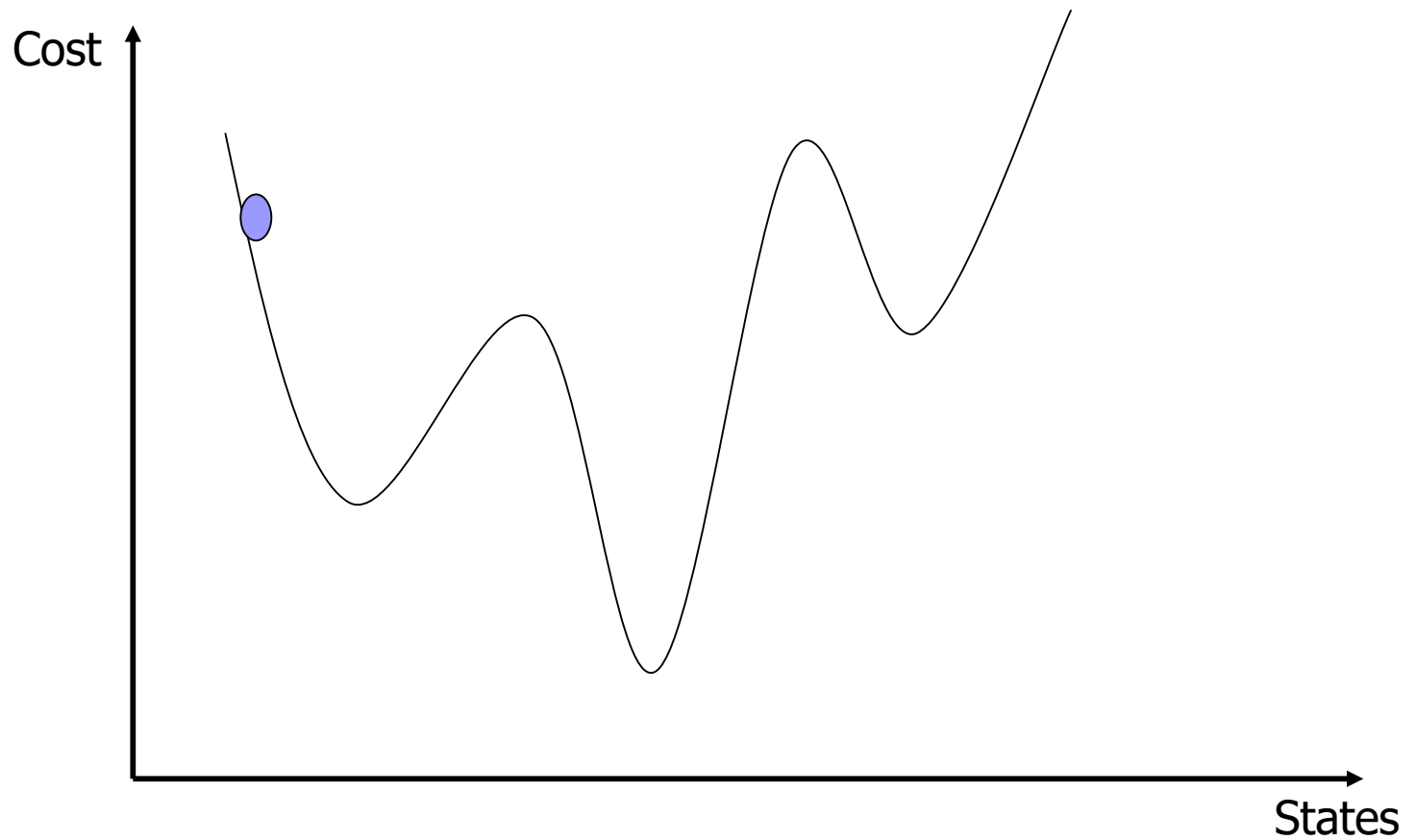
- Cực trị địa phương: tìm kiếm cục bộ hội tụ đến cực trị địa phương và không tìm được lời giải tối ưu



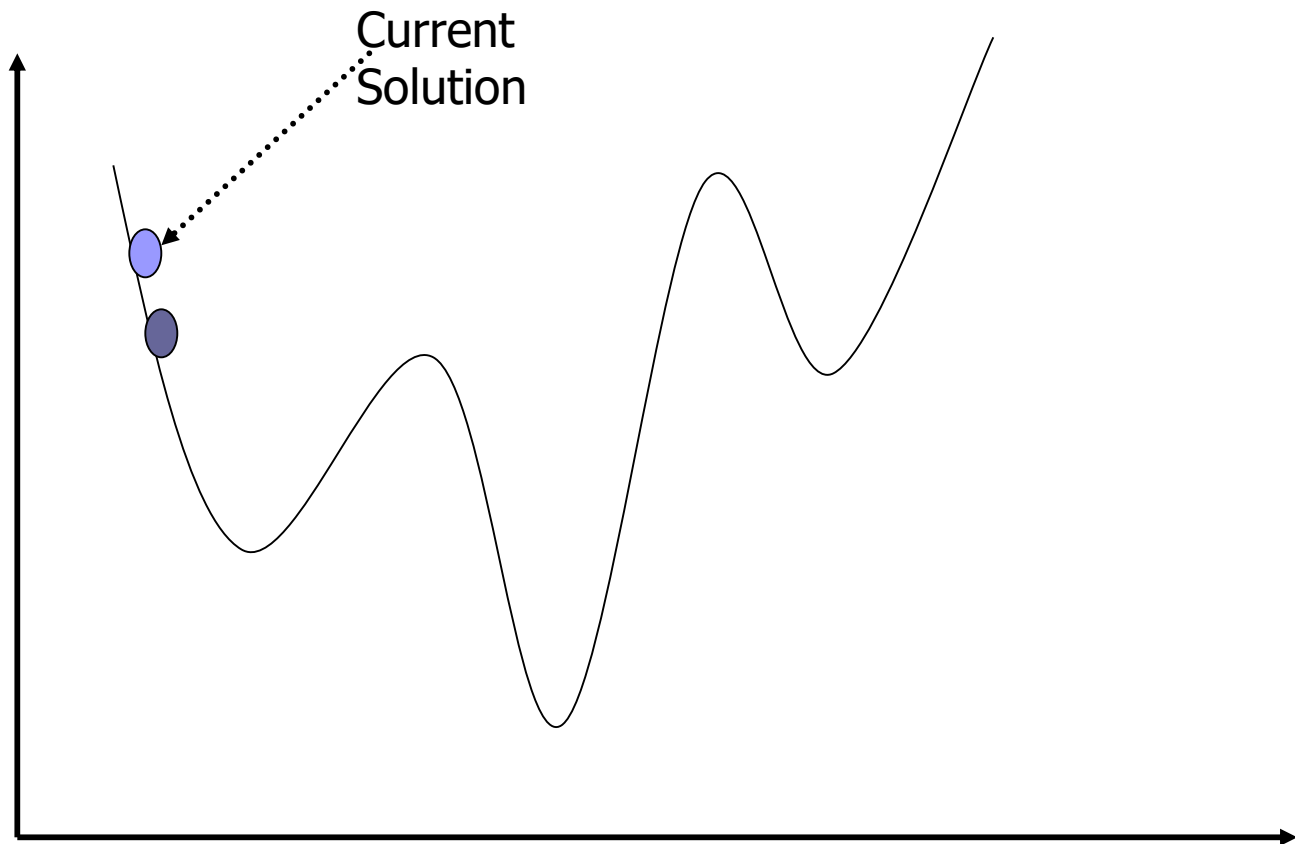
Local minimum

- Giải pháp
 - + Thử lại với xuất phát ngẫu nhiên
 - + Hiệu quả khi có một vài cực trị địa phương

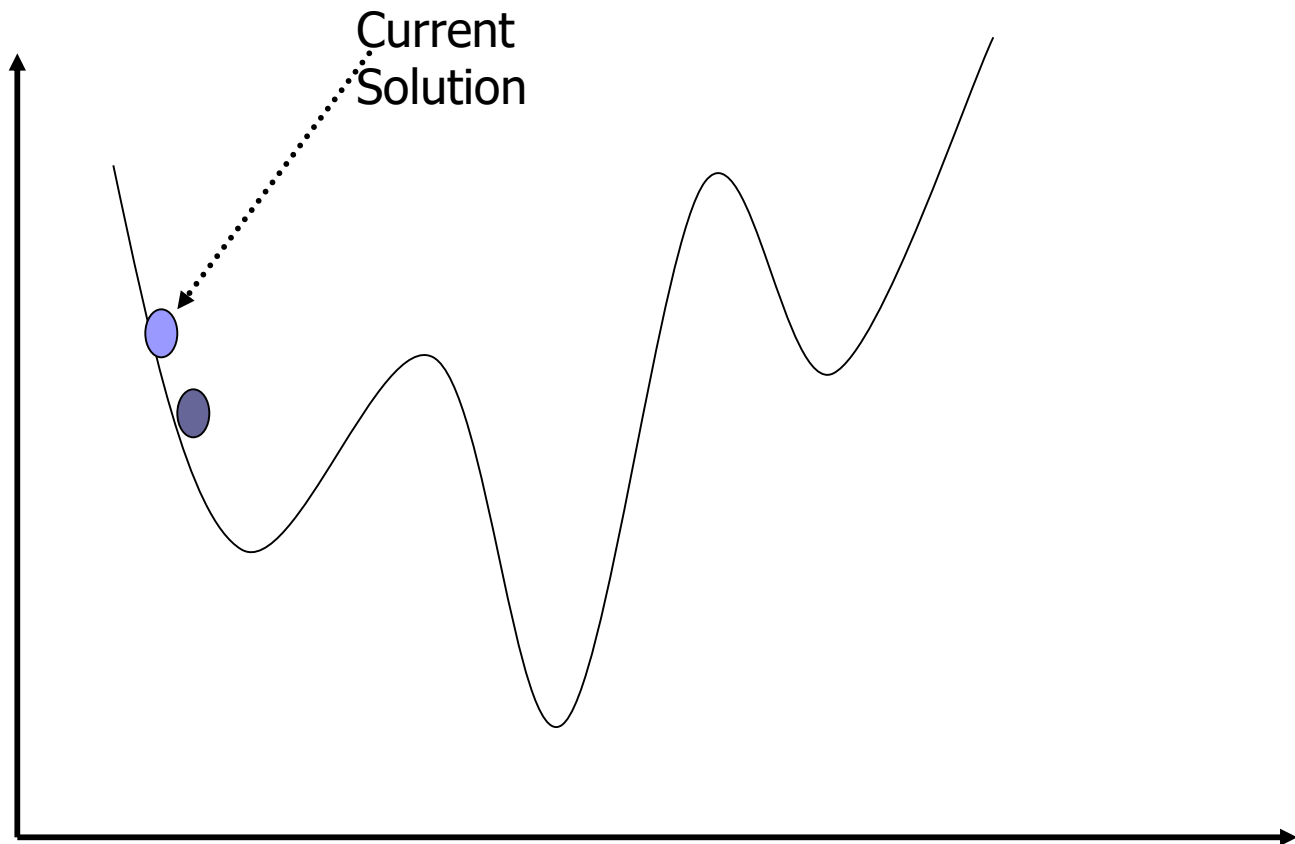
Hill Climbing



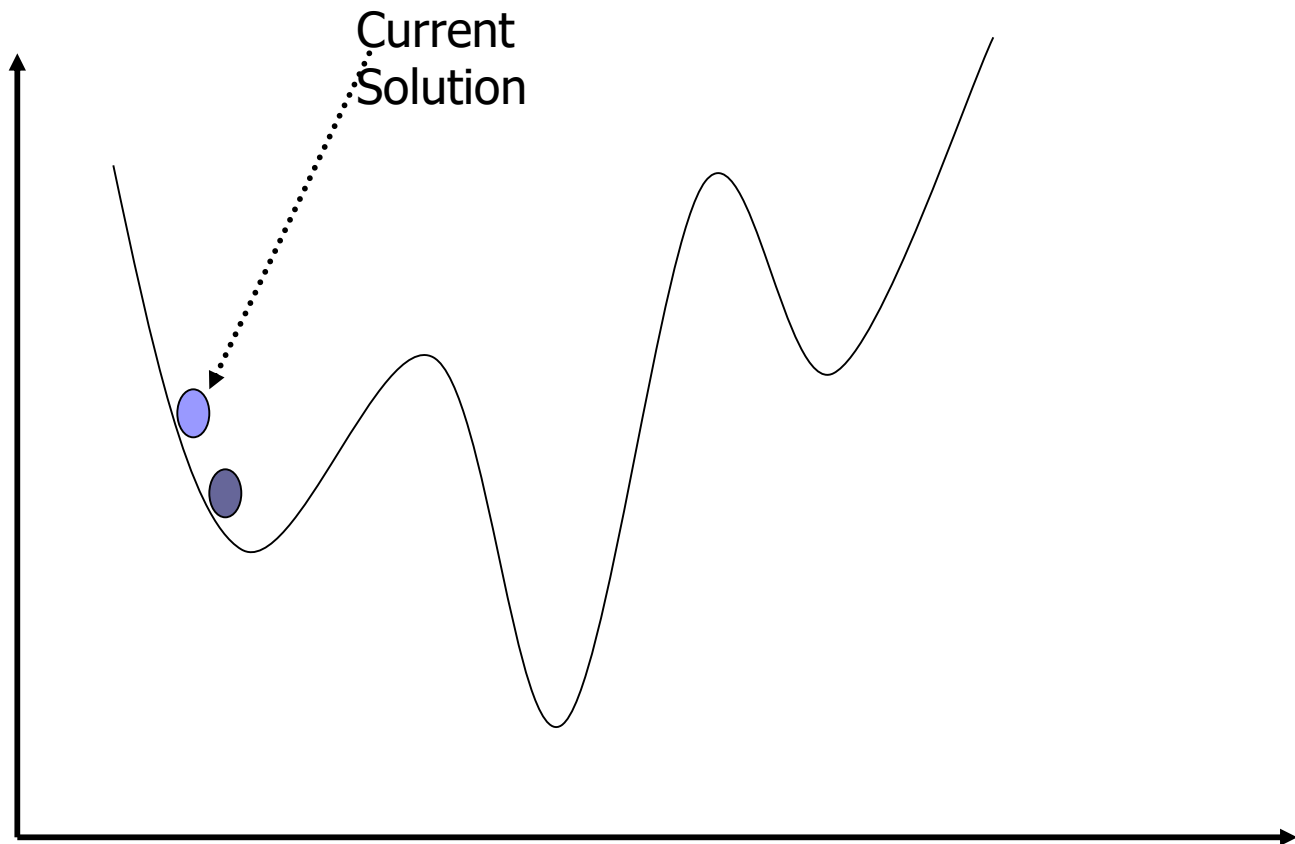
Hill Climbing



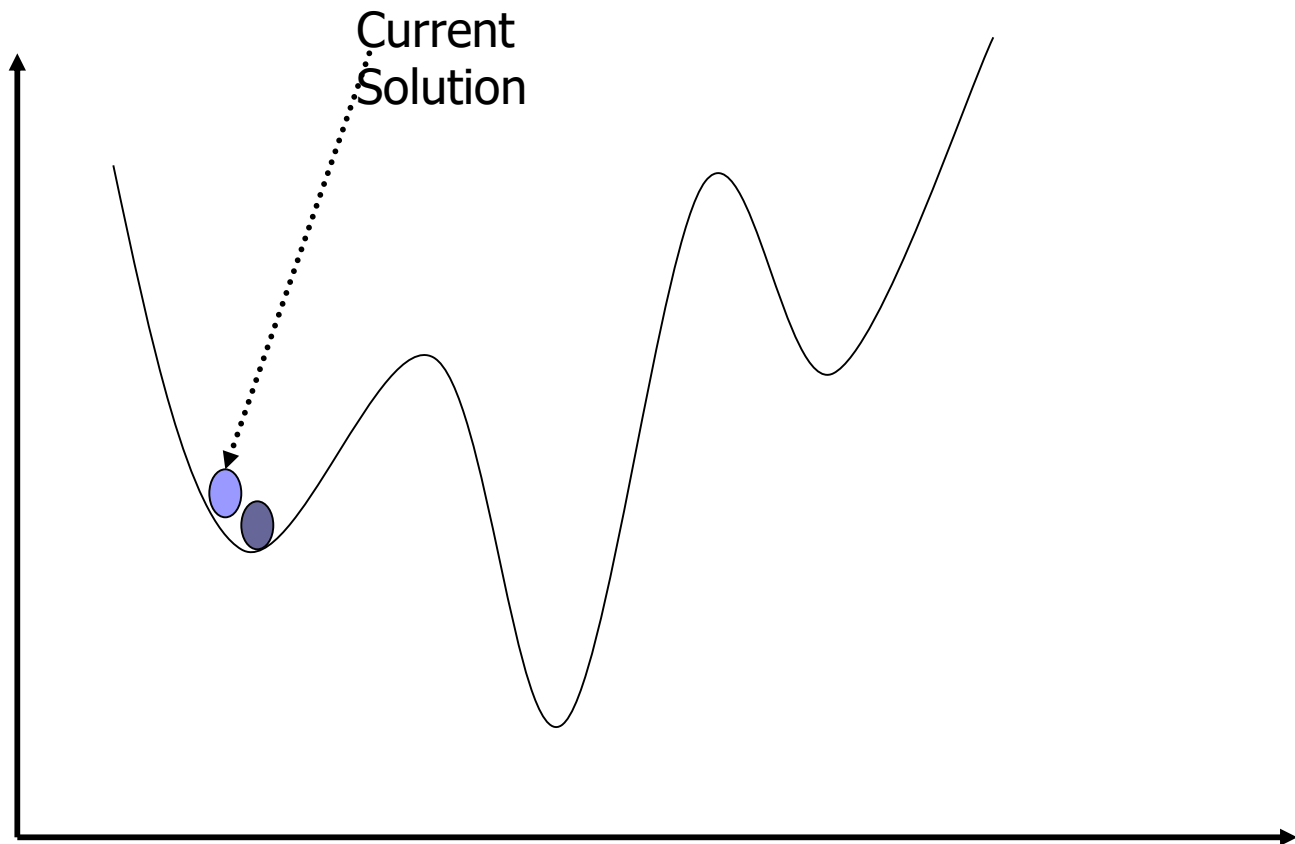
Hill Climbing



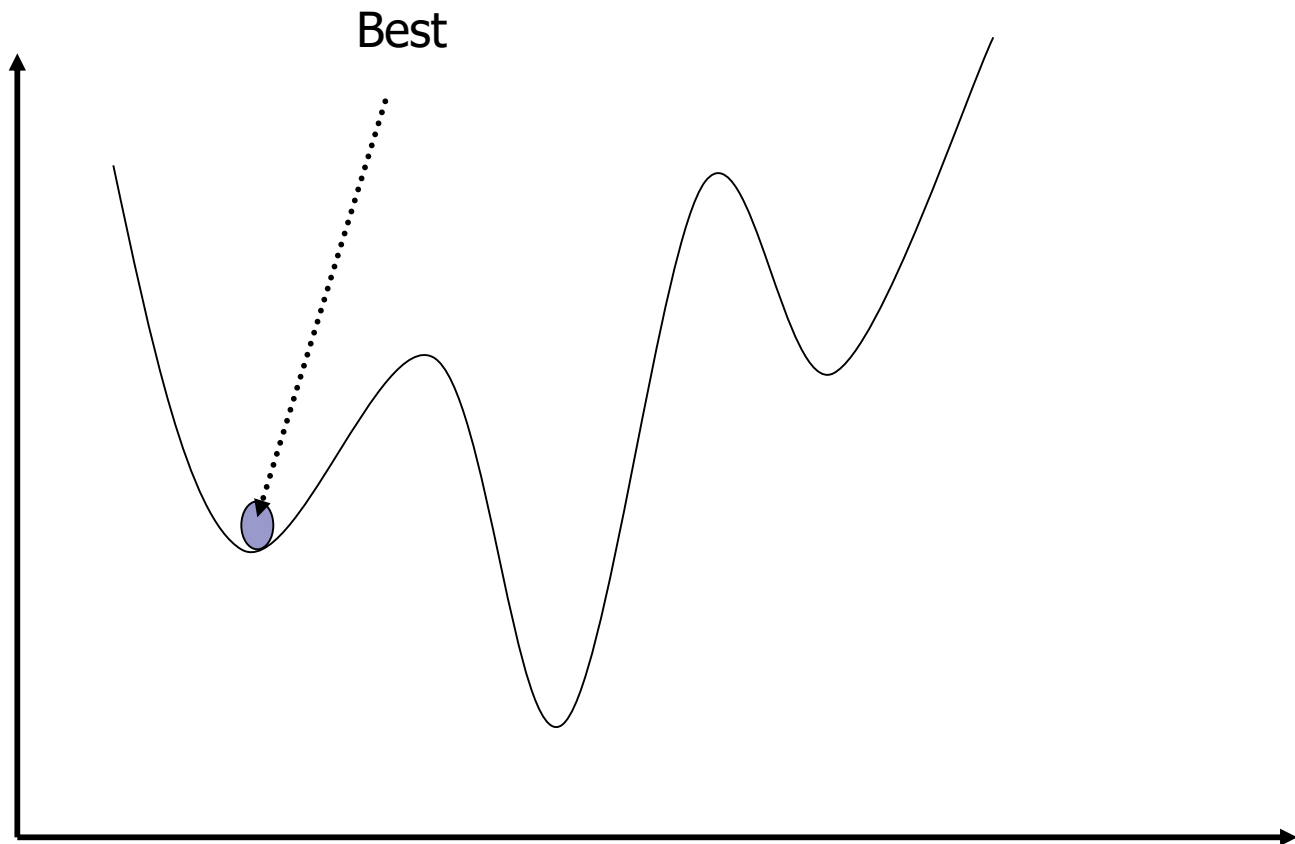
Hill Climbing



Hill Climbing



Hill Climbing



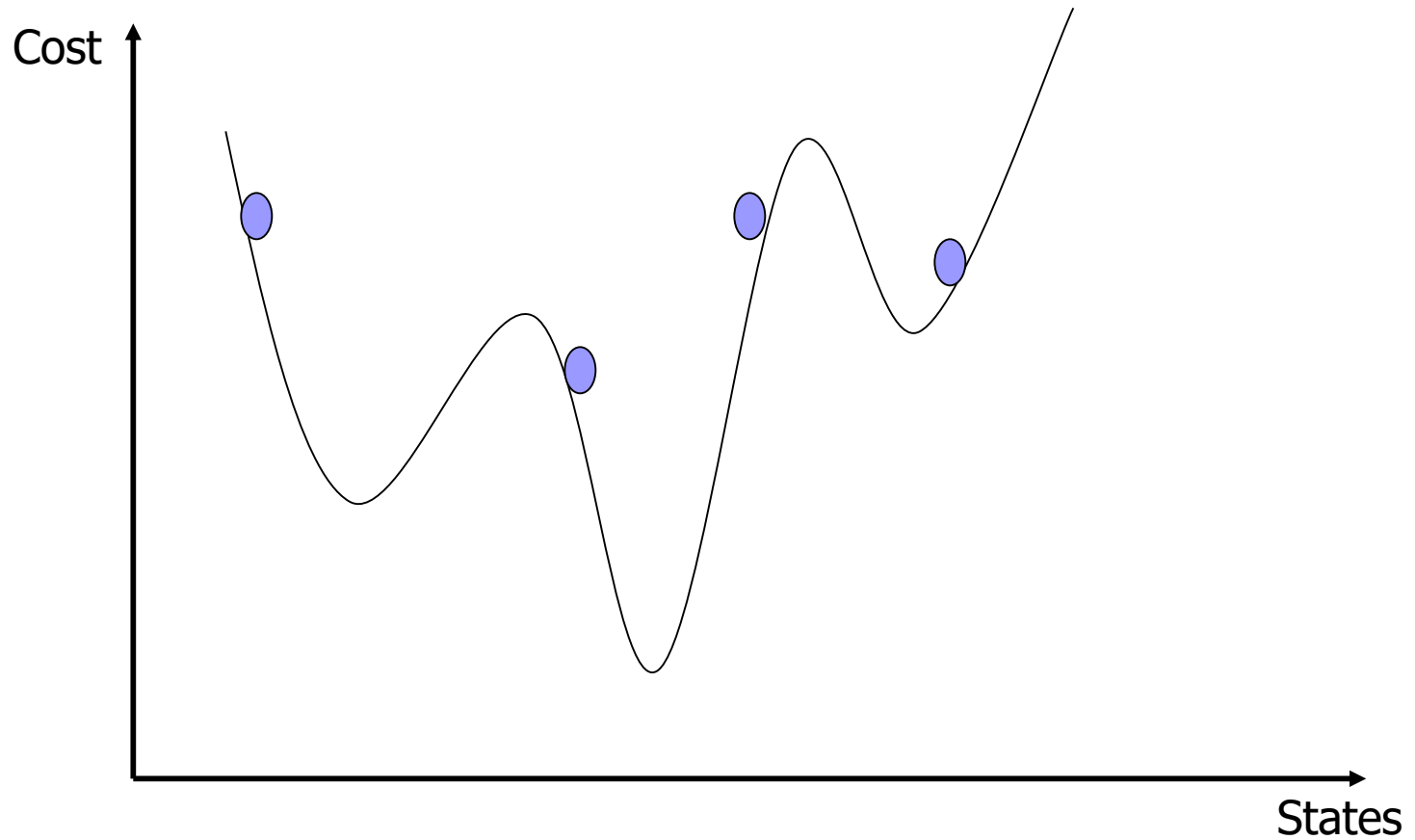
Cải tiến

- Lặp lại Hill climbing từ các điểm xuất phát khác
 - Random walk
- Xử lý song song một quần thể lời giải
 - Local Beam Search
- Thoát cực trị địa phương bằng cách thử cả các lời giải xấu hơn lời giải hiện tại
 - Simulated Annealing (Mô phỏng tôi thép)

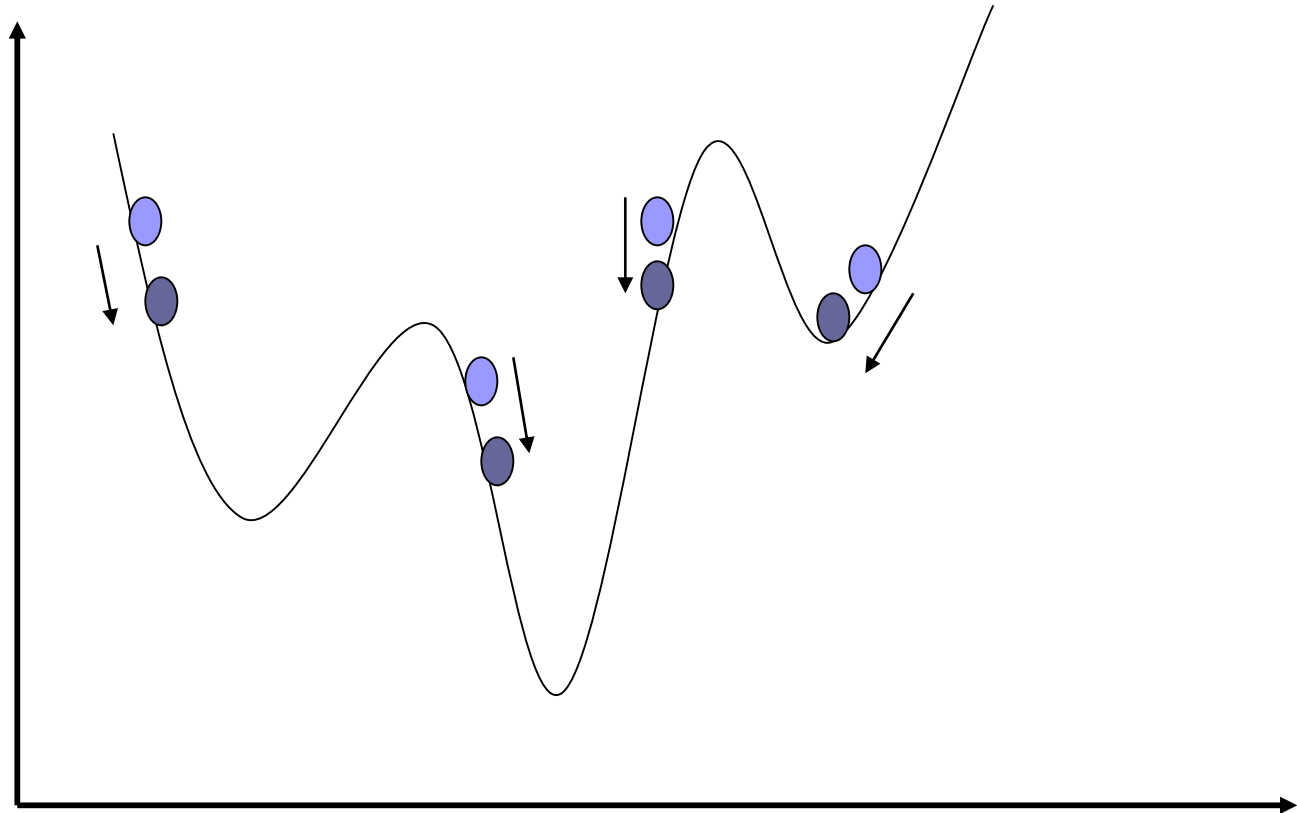
Local Beam Search

- Idea: keep k states instead of just 1
 - Begins with k randomly generated states
 - At each step all the successors of all k states are generated.
 - If one is a goal, we stop, otherwise select k best successors from complete list and repeat

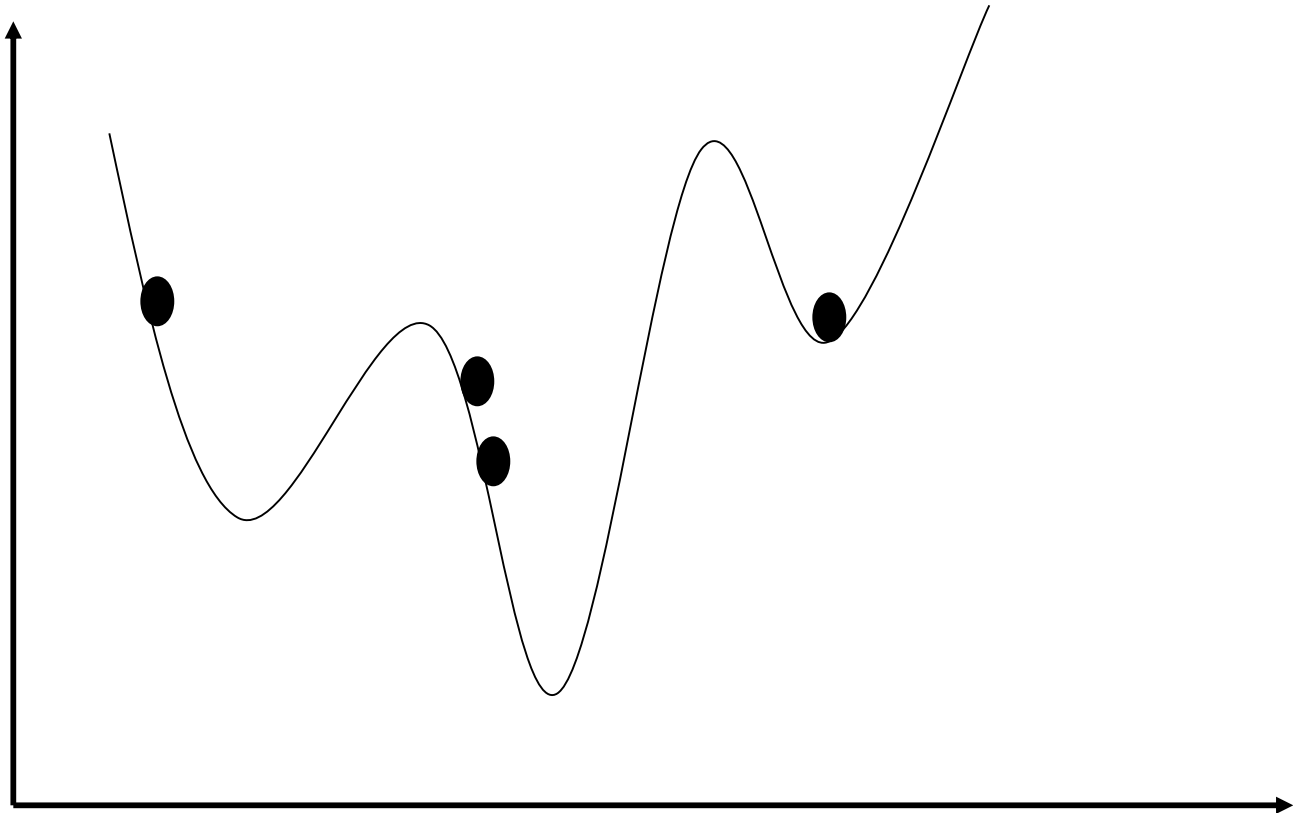
Local Beam Search



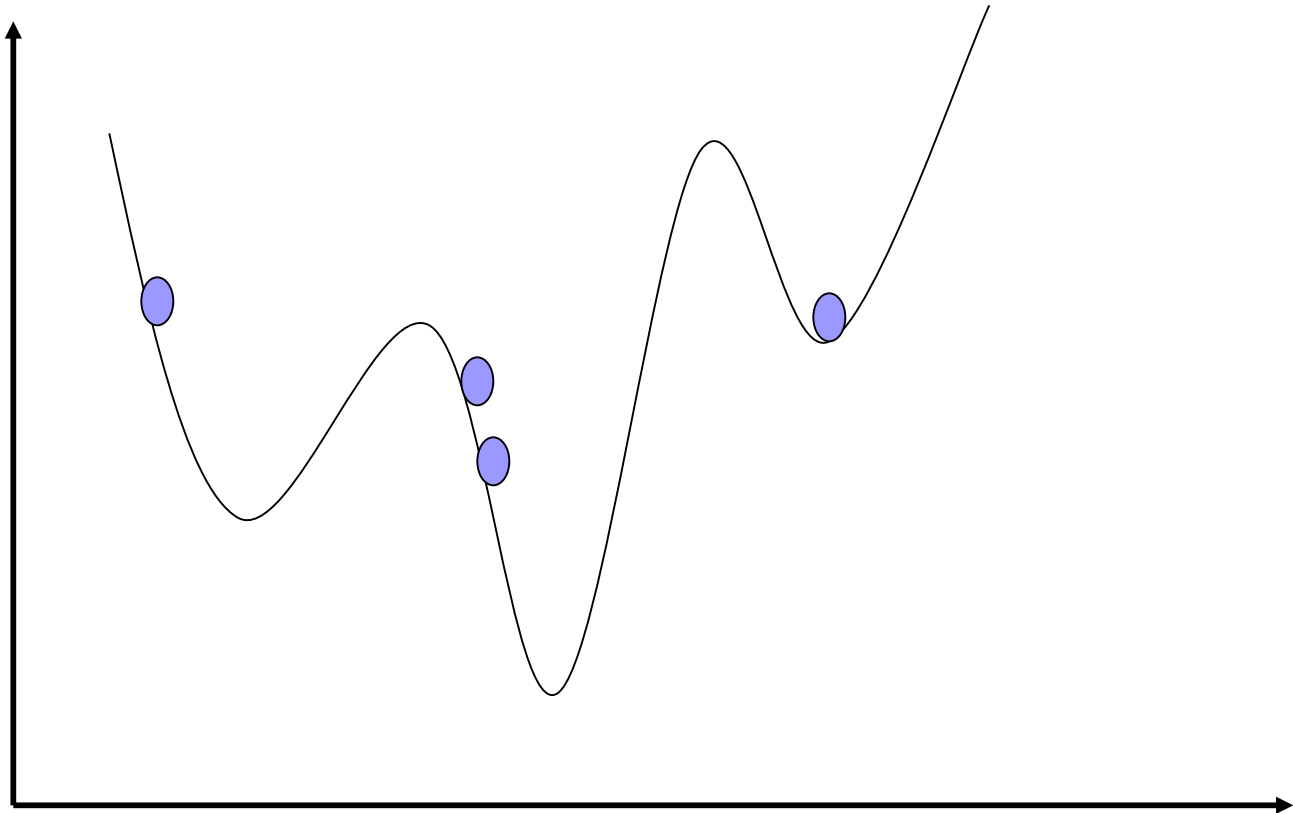
Local Beam Search



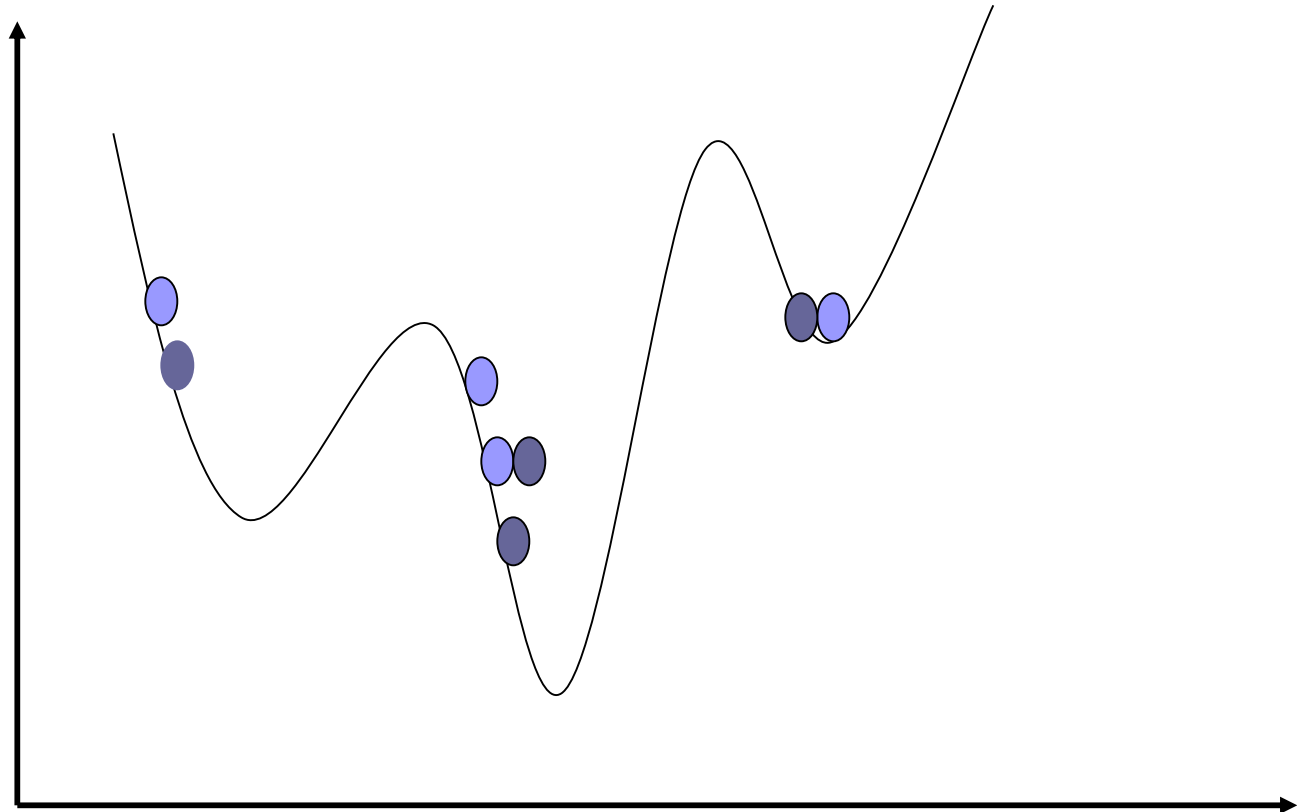
Local Beam Search



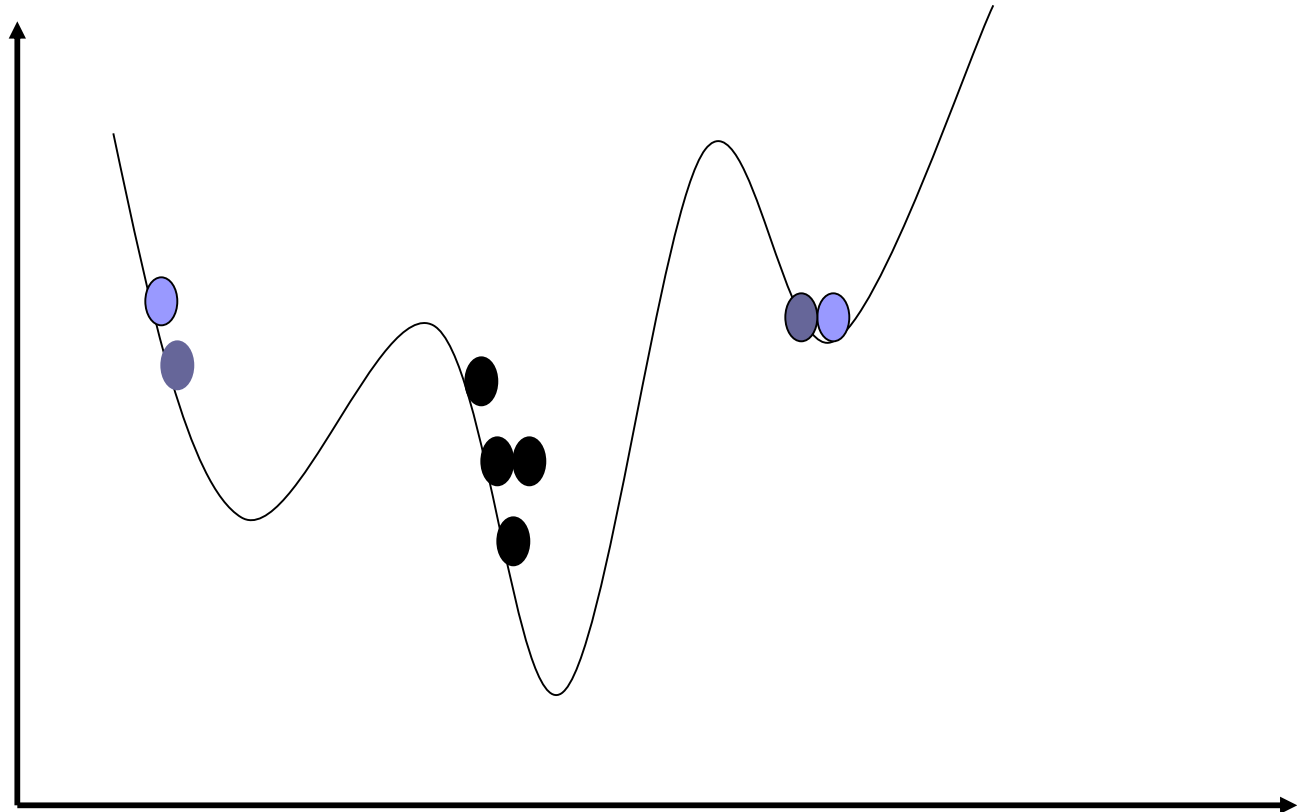
Local Beam Search



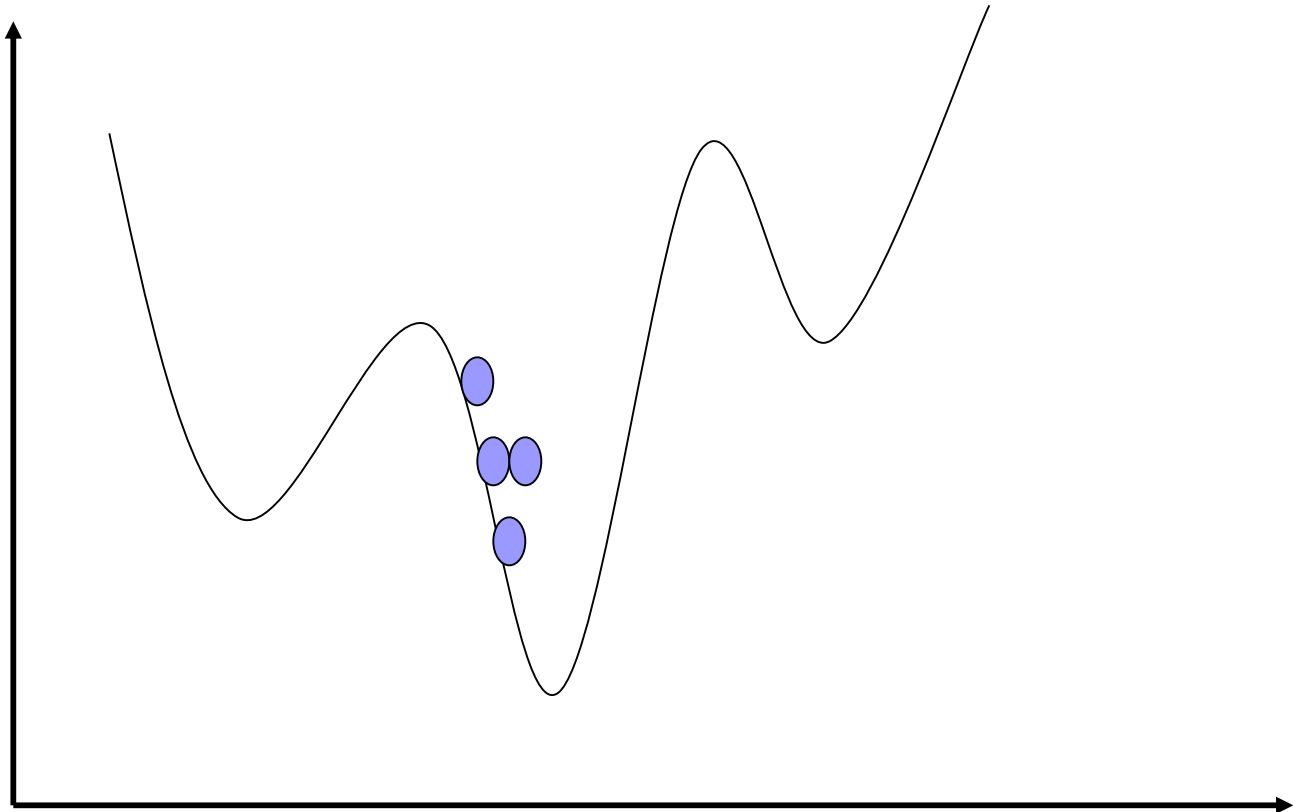
Local Beam Search



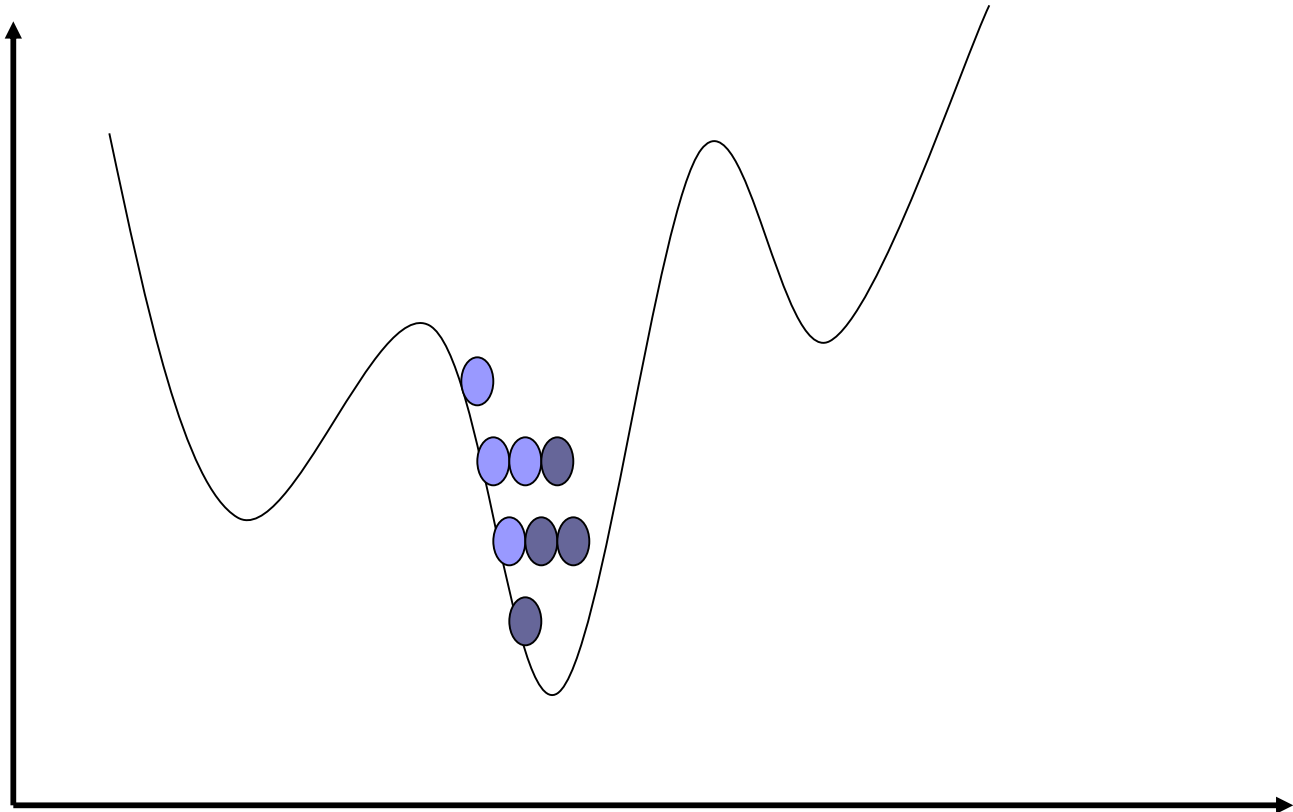
Local Beam Search



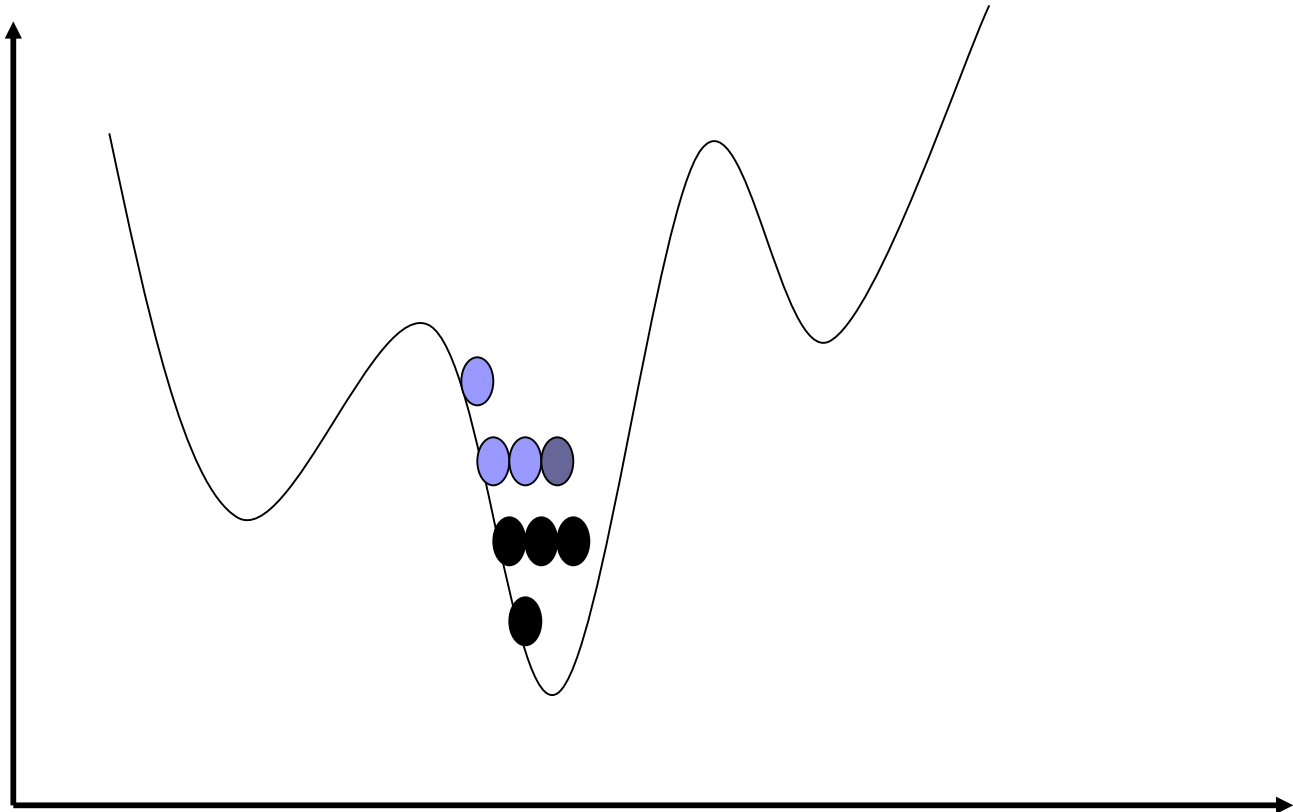
Local Beam Search



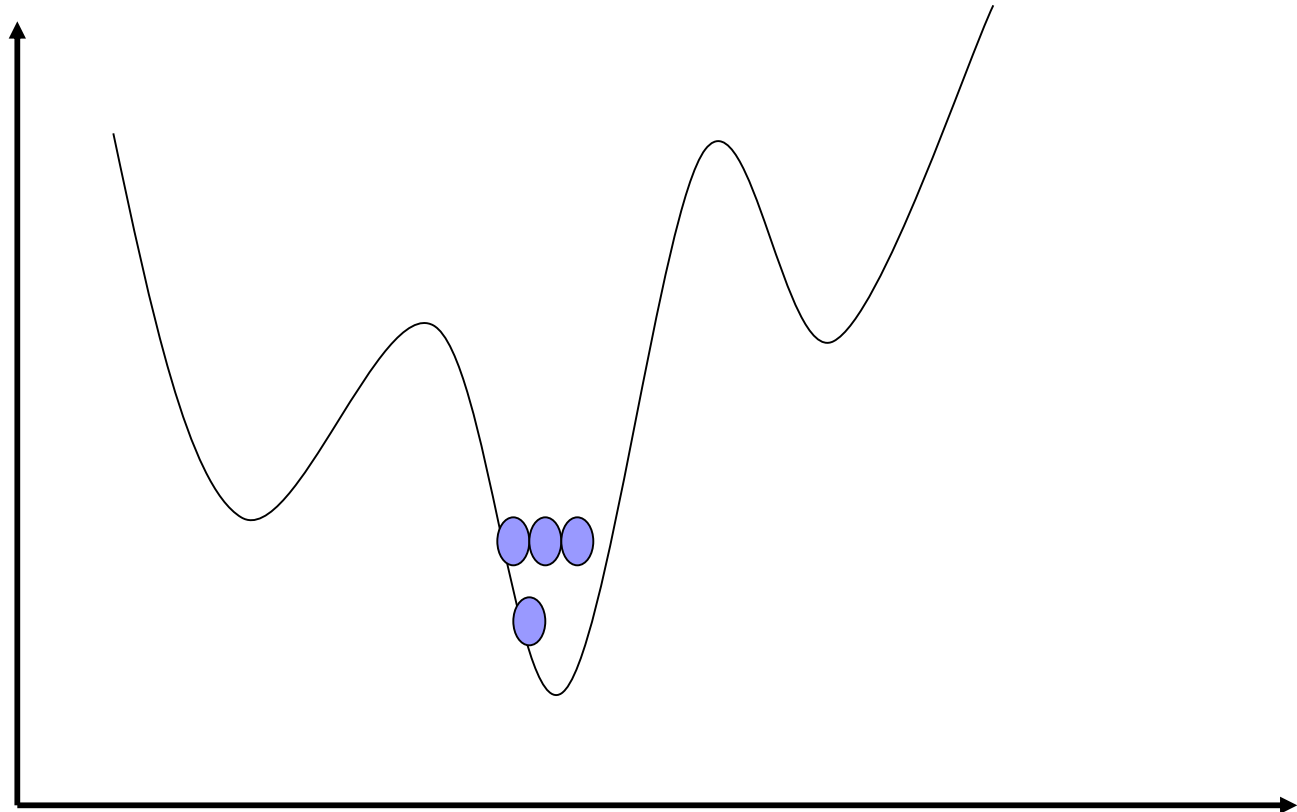
Local Beam Search



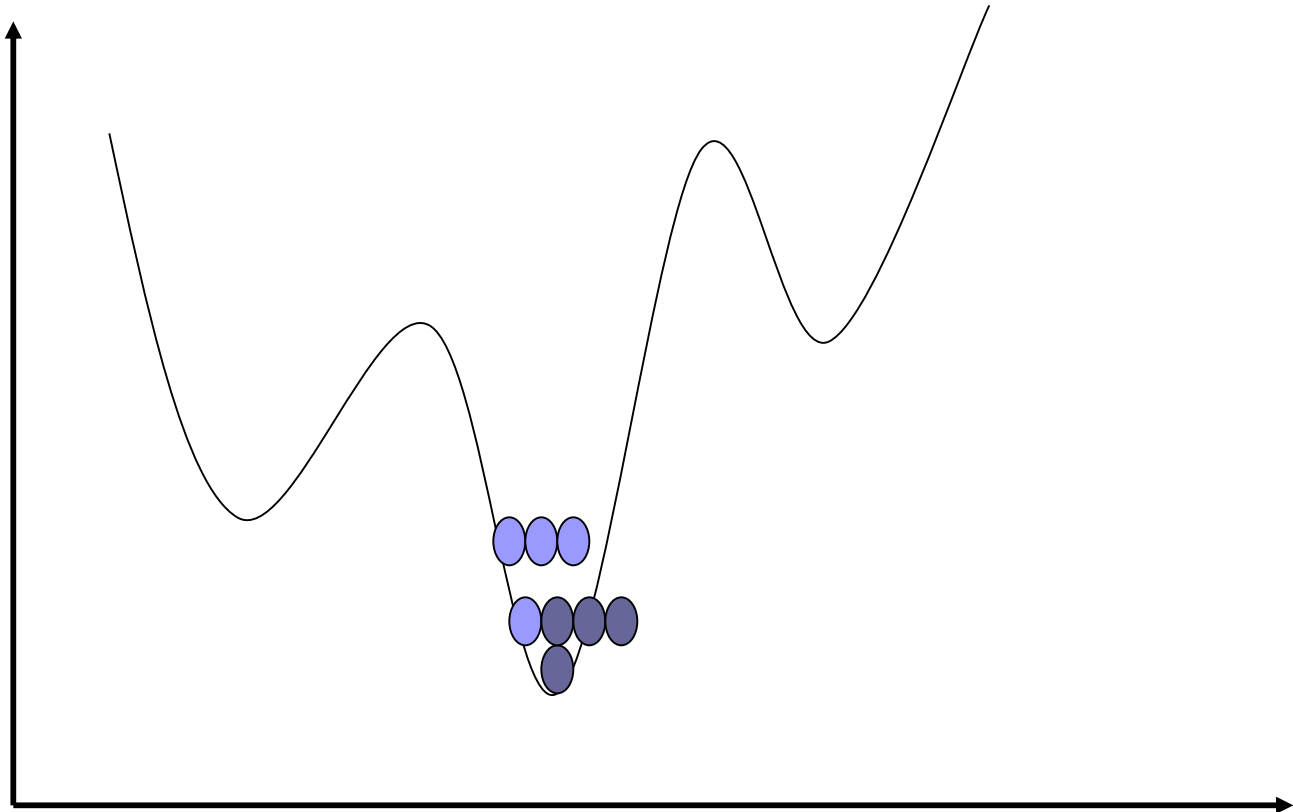
Local Beam Search



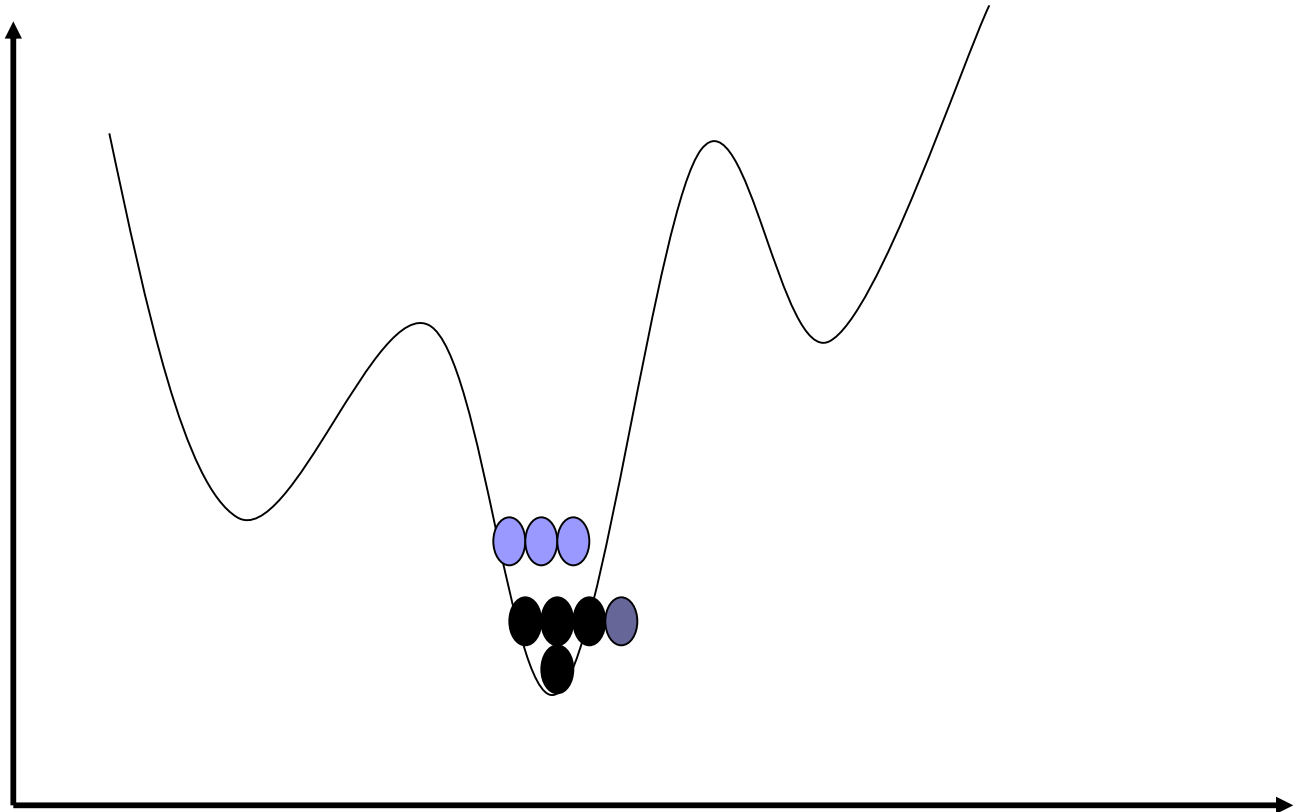
Local Beam Search



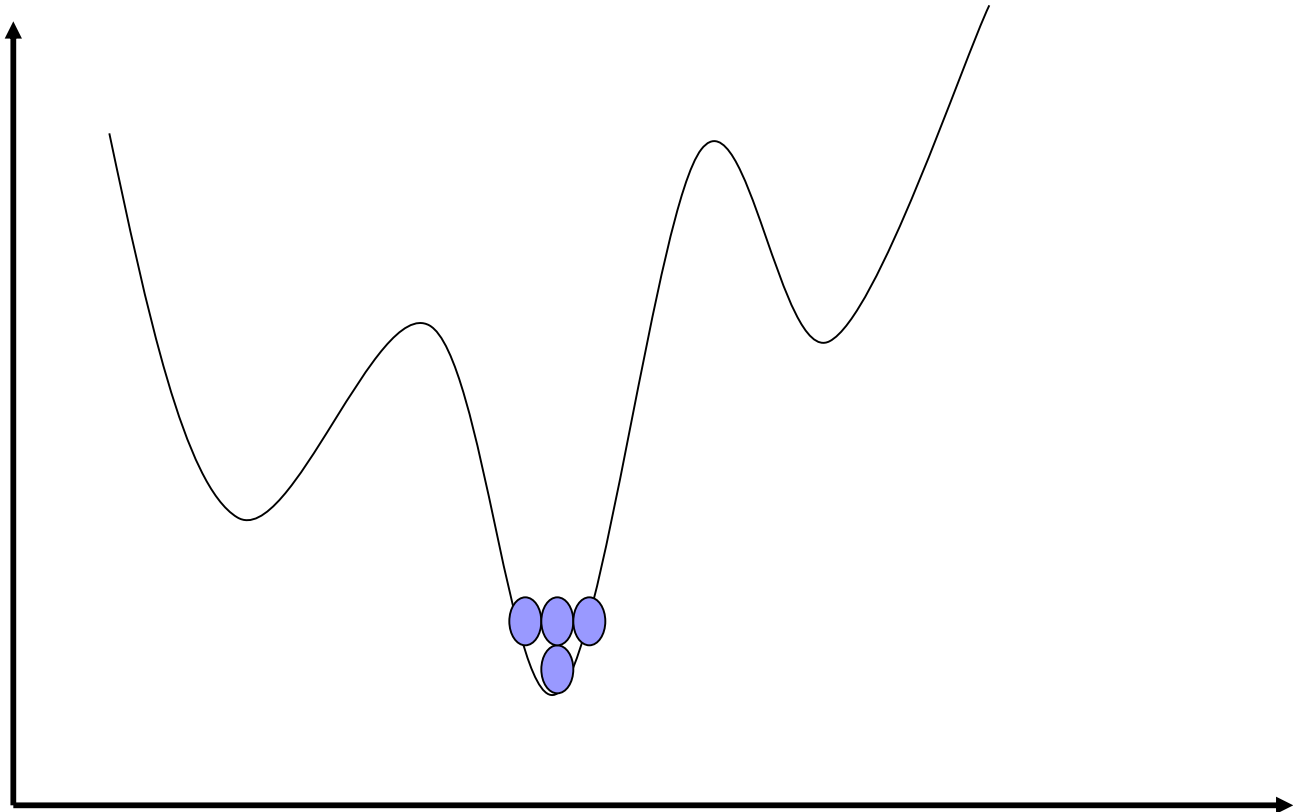
Local Beam Search



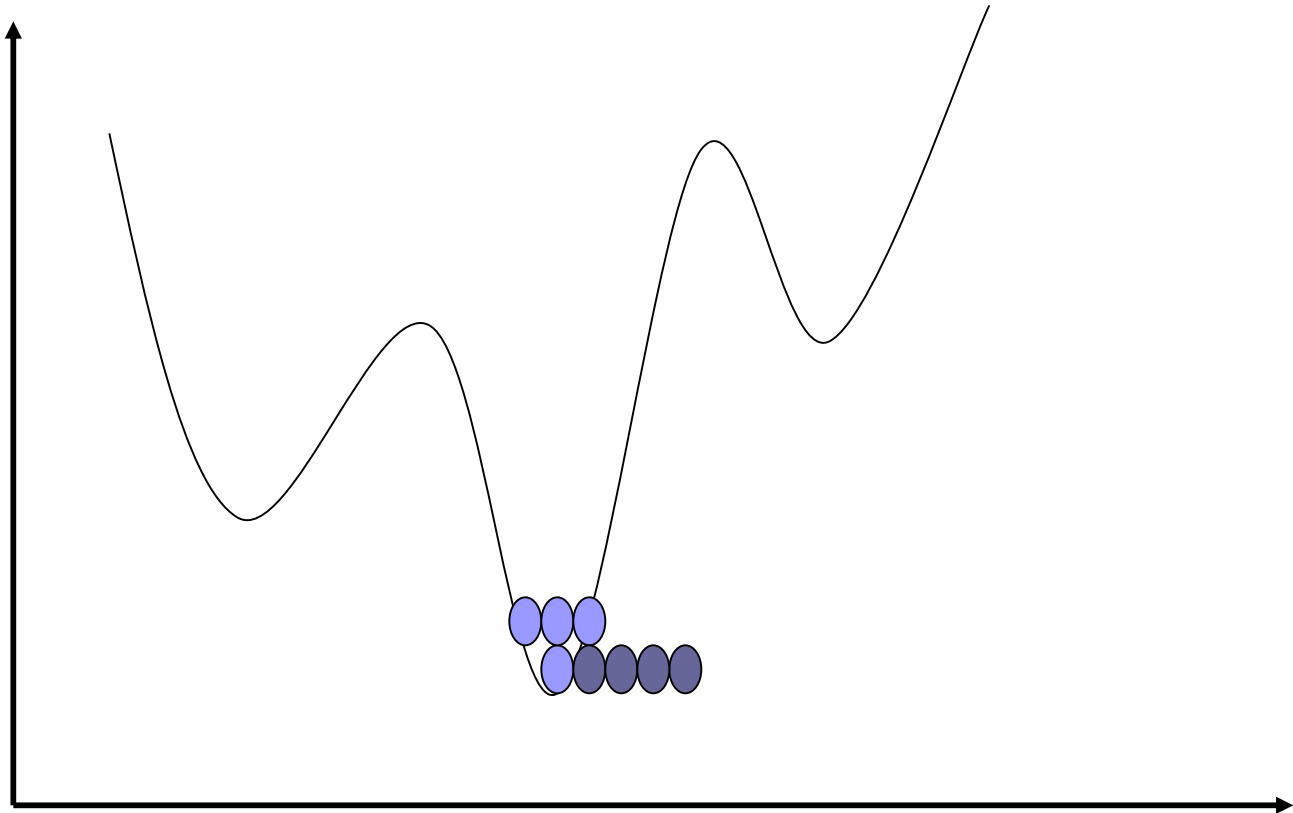
Local Beam Search



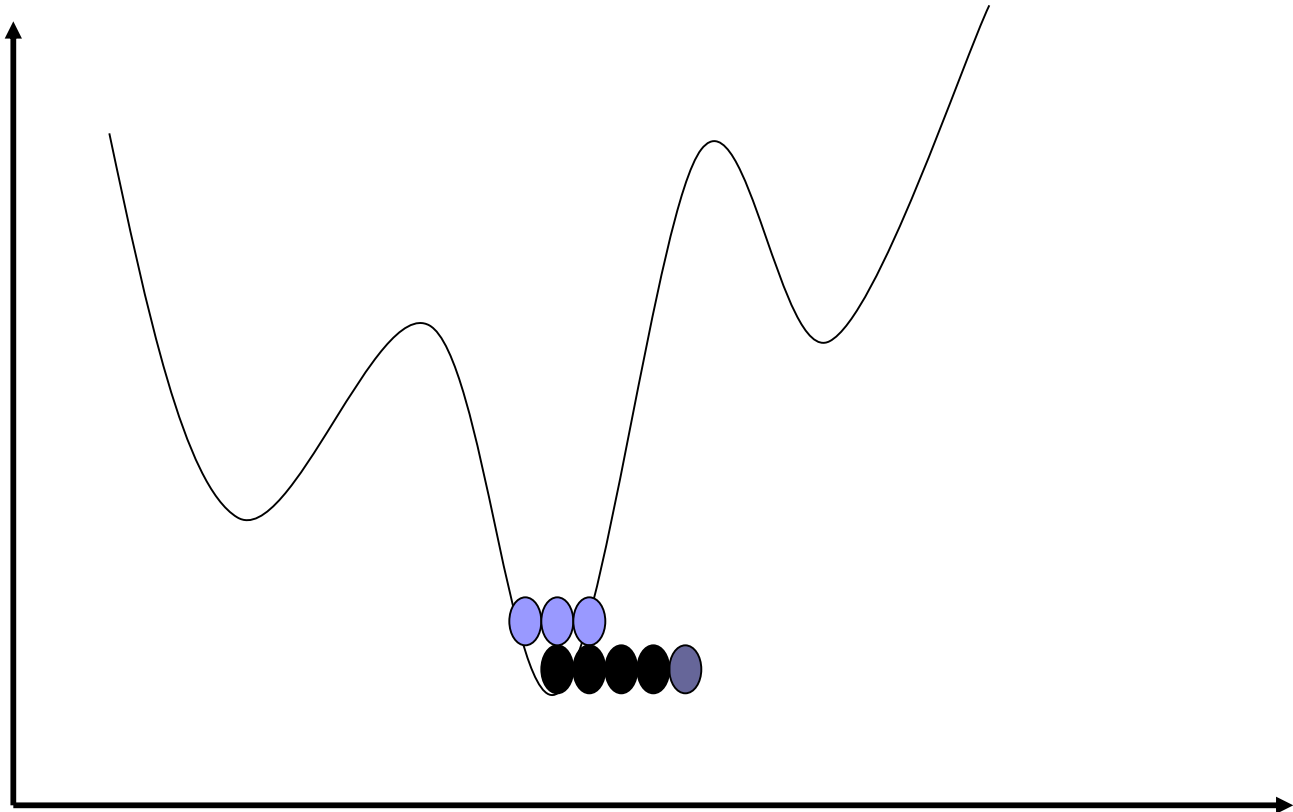
Local Beam Search



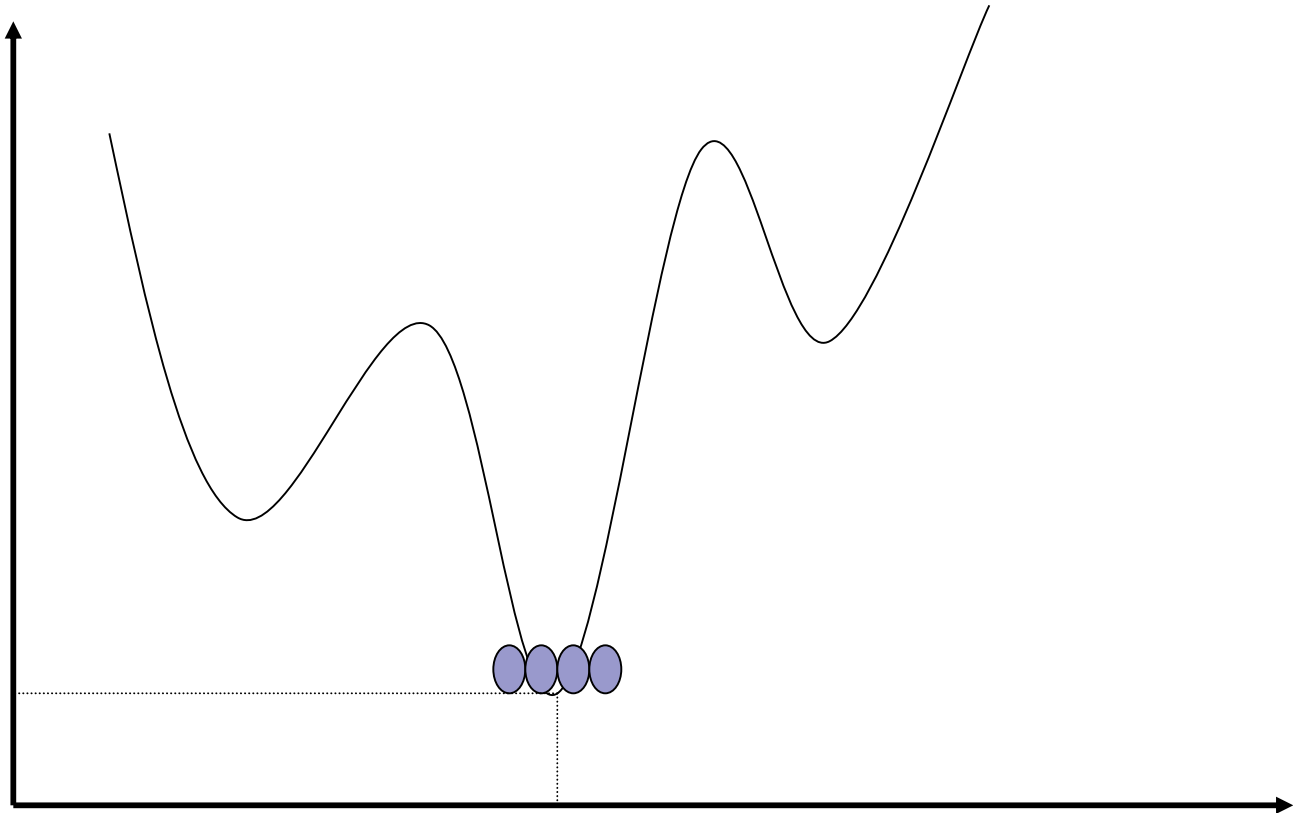
Local Beam Search



Local Beam Search



Local Beam Search



Simulated Annealing

Mô phỏng tôi thép

- Kết hợp giữa “leo đồi” và “ngẫu nhiên”
- Ý tưởng từ nghiên cứu vật lý
- Quá trình tôi thép/luyện kim
 - Nung nóng kim loại rồi để nguội từ từ
 - Kim loại sẽ đạt đến trạng thái kết tinh bền vững
- Tìm kiếm cục bộ “mô phỏng tôi thép”
 - Chấp nhận bước chuyển tới trạng thái xấu hơn, giảm dần xác suất thực hiện việc này
 - Thoát được cực trị địa phương để đạt tới tối ưu toàn cục



Simulated Annealing

- Nhiệt độ T
 - Dùng để xác định xác suất chuyển sang trạng thái kém hơn
 - T lớn thì xác suất lớn, đồng nghĩa với bước chuyển nhiều
- Lịch trình làm lạnh (Cooling Schedule)
 - Tỷ lệ giảm nhiệt độ T
 - Nếu giảm T đủ chậm thì sẽ tìm được tối ưu toàn cục
- Thời điểm bắt đầu, sẽ tìm kiếm với bước nhảy lớn sau đó giảm dần biên độ

Simulated Annealing

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

$T \leftarrow \text{schedule}(t)$

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow \text{next.VALUE} - \text{current.VALUE}$

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

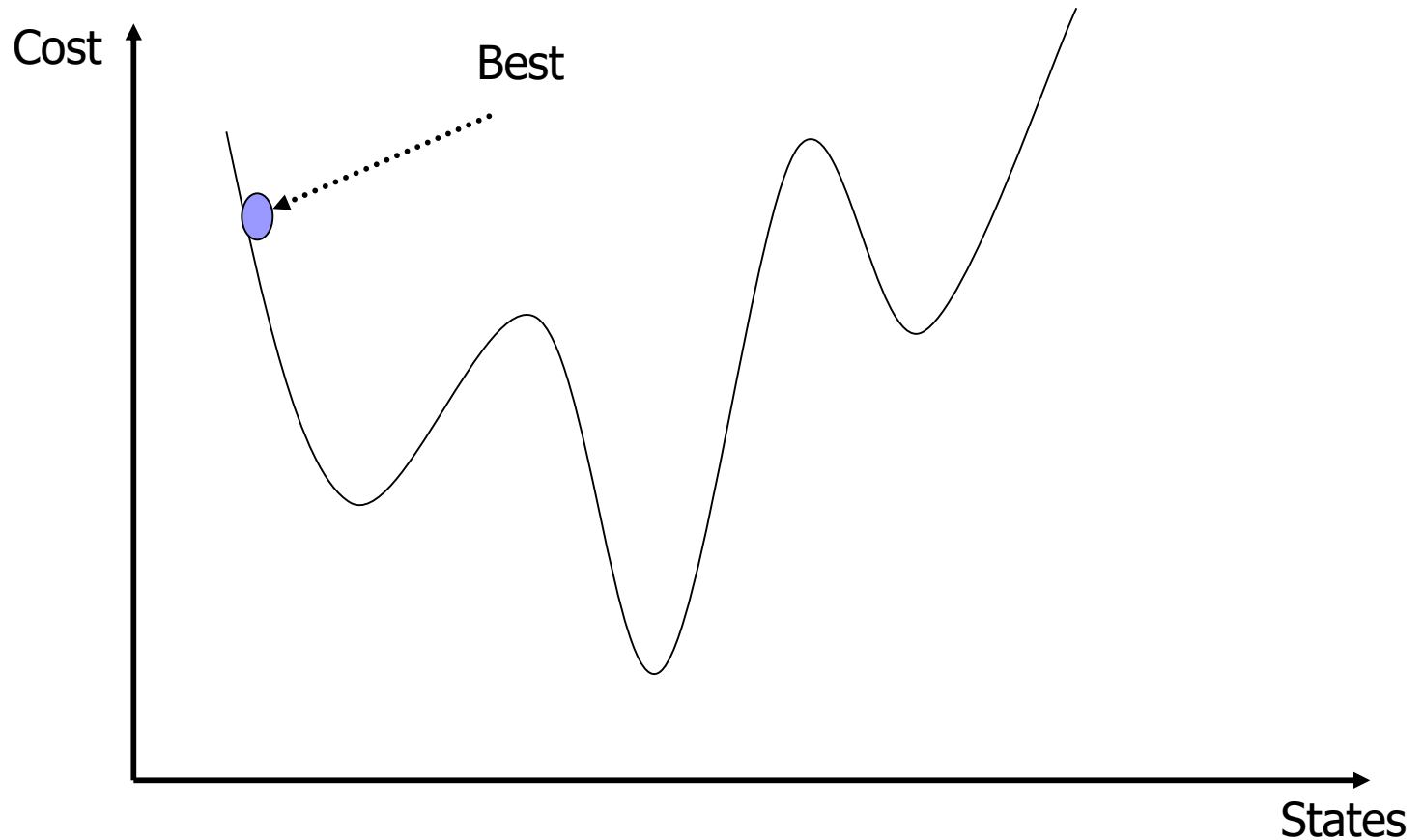
else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

Control the change of
temperature T ($\downarrow$ over time)

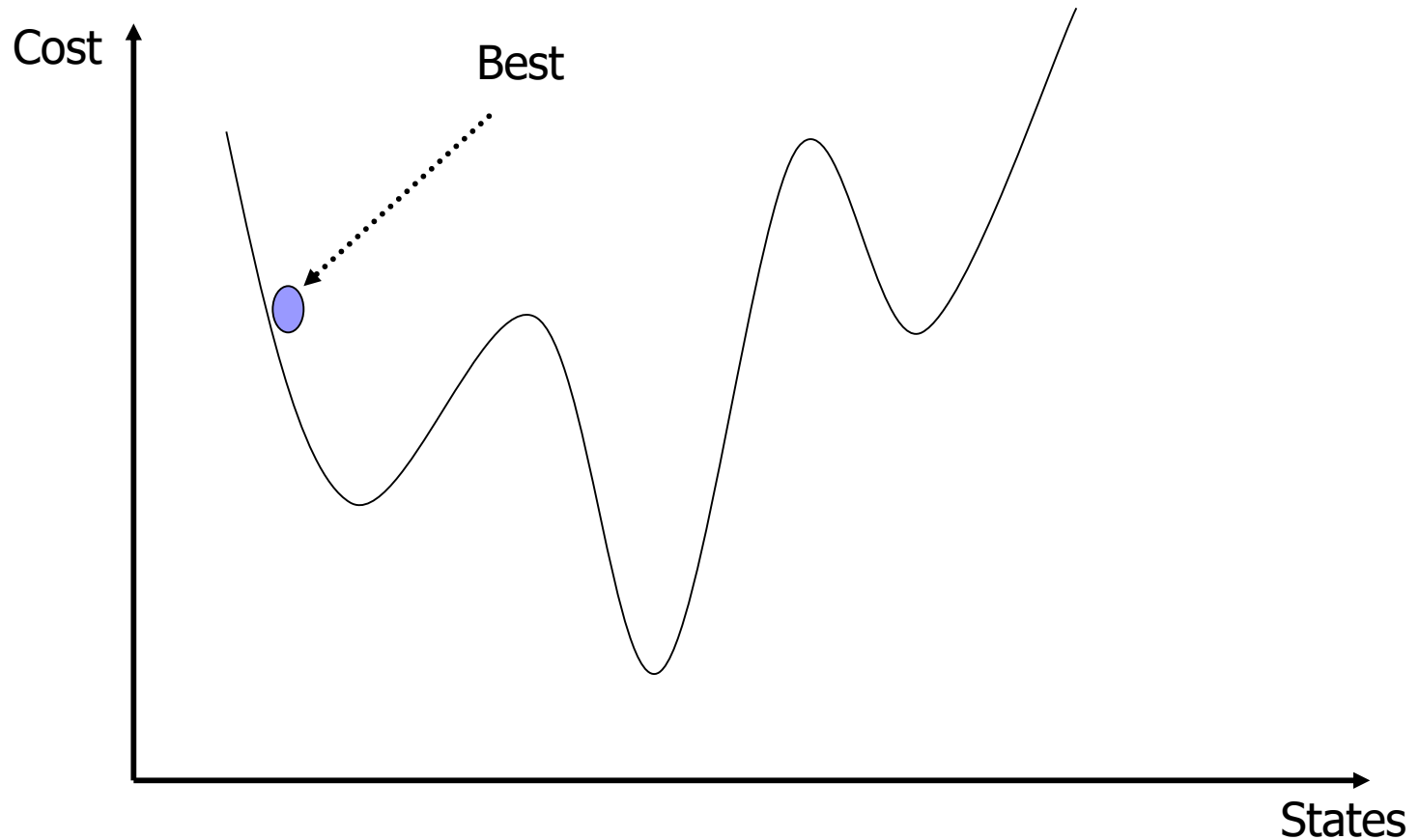
Almost the same as hill climbing
except for a *random* successor

Unlike hill climbing, move
downhill with some prob.

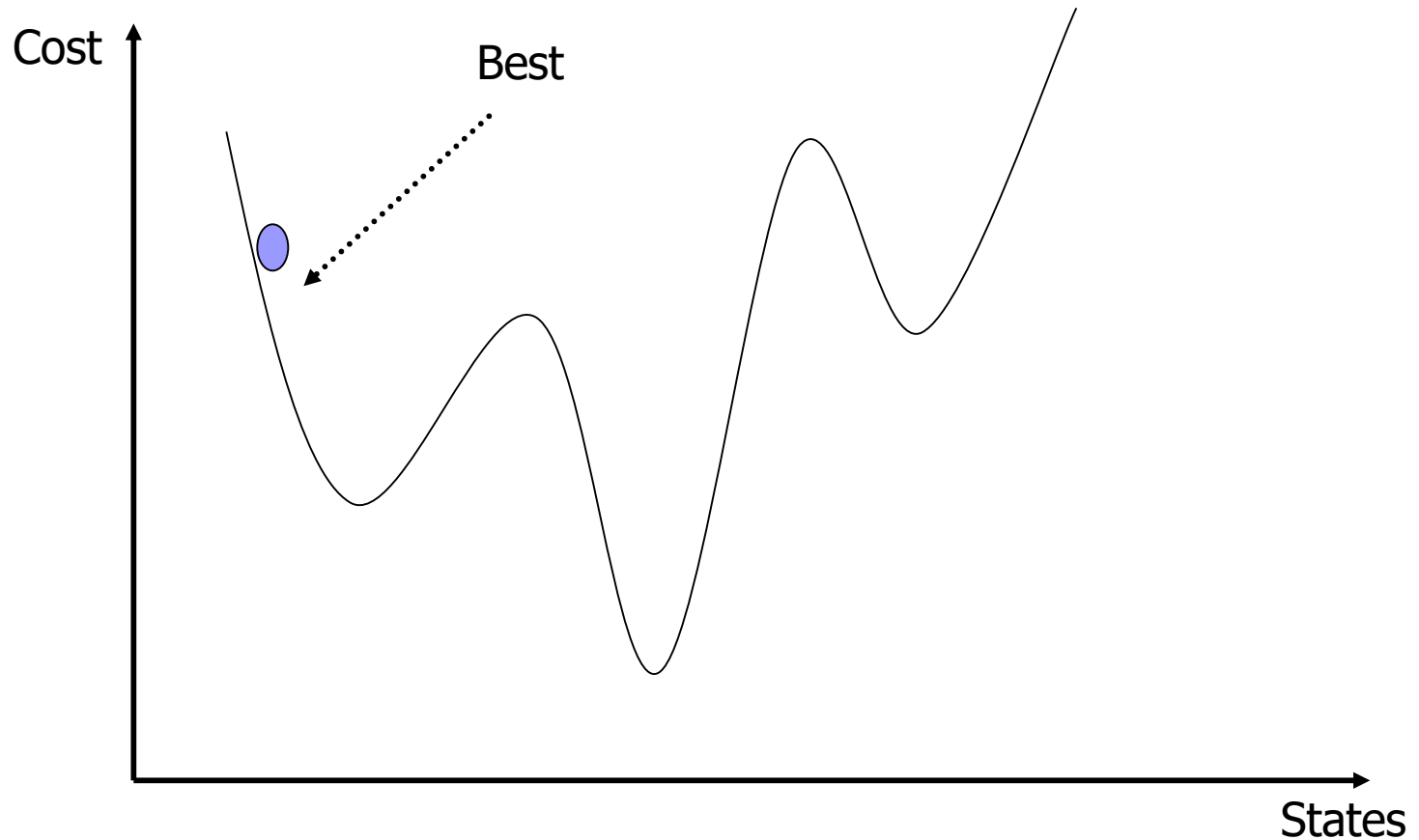
Simulated Annealing



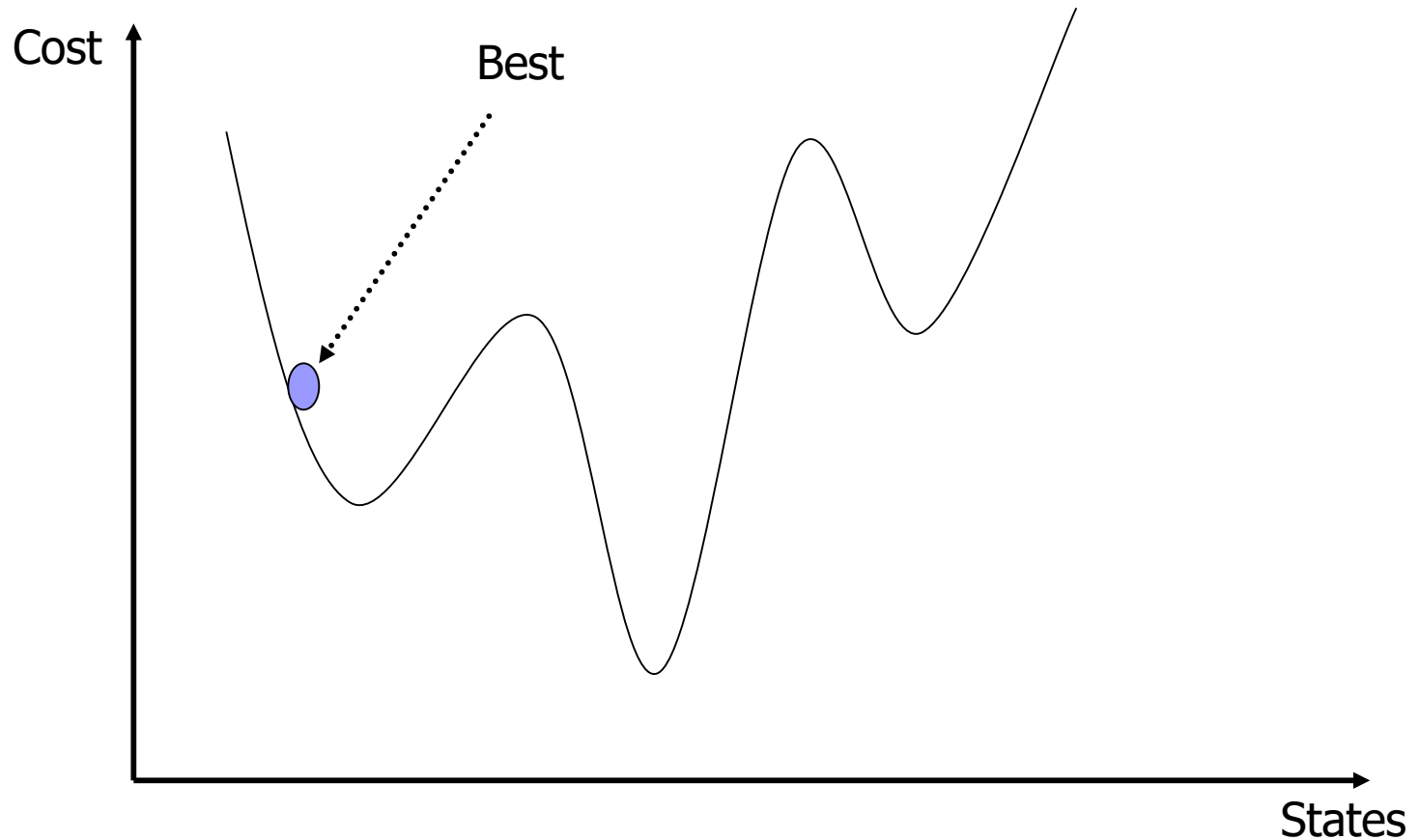
Simulated Annealing



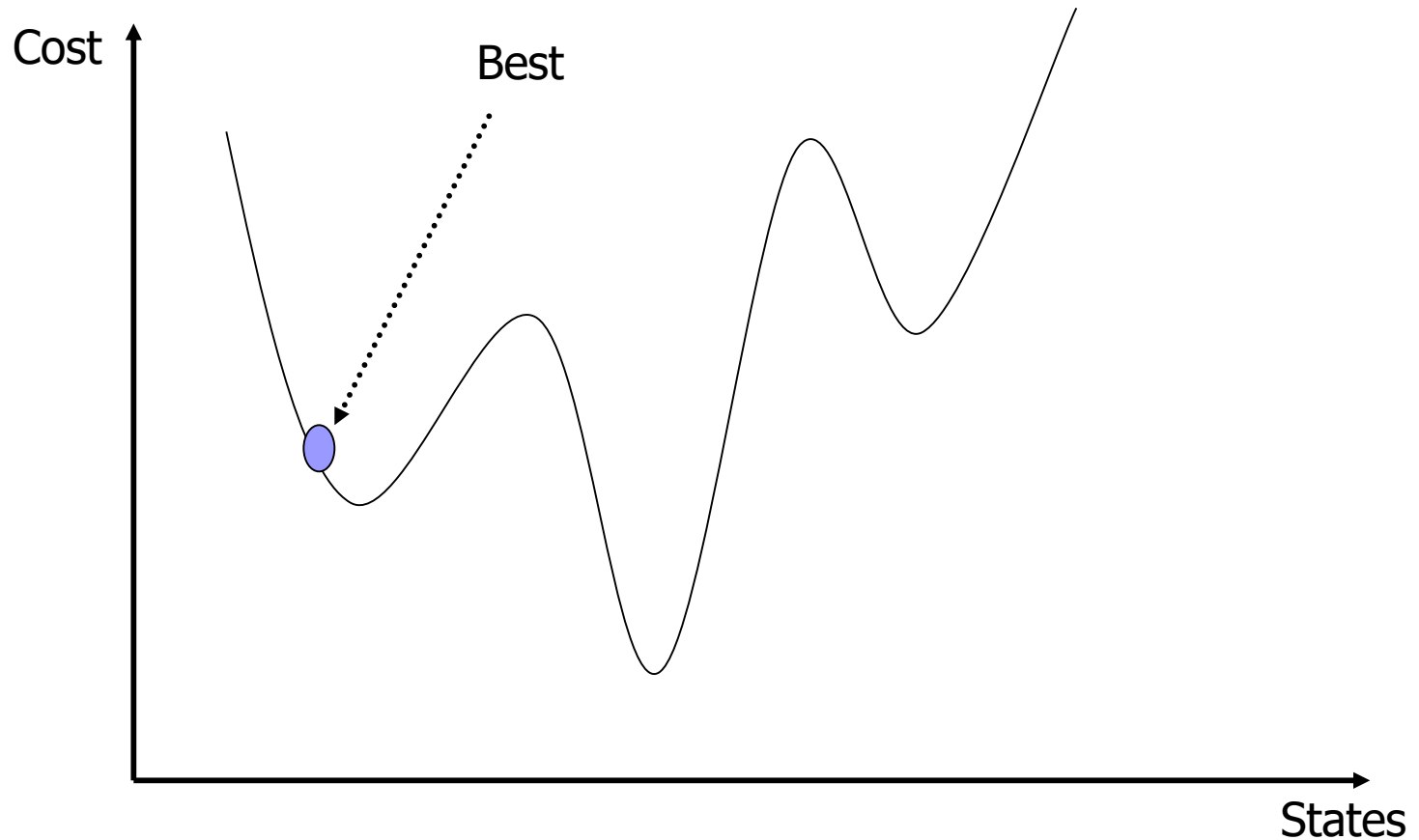
Simulated Annealing



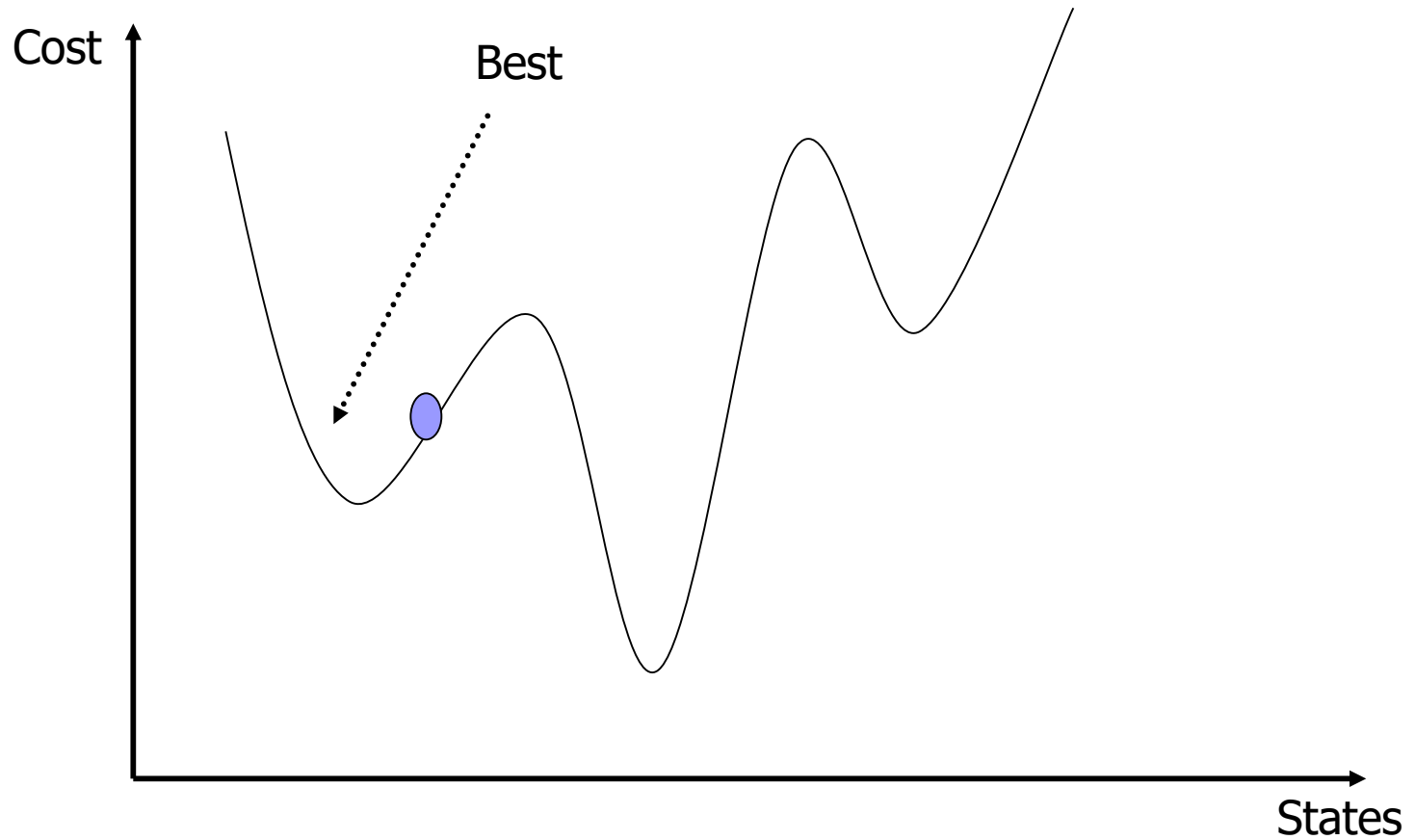
Simulated Annealing



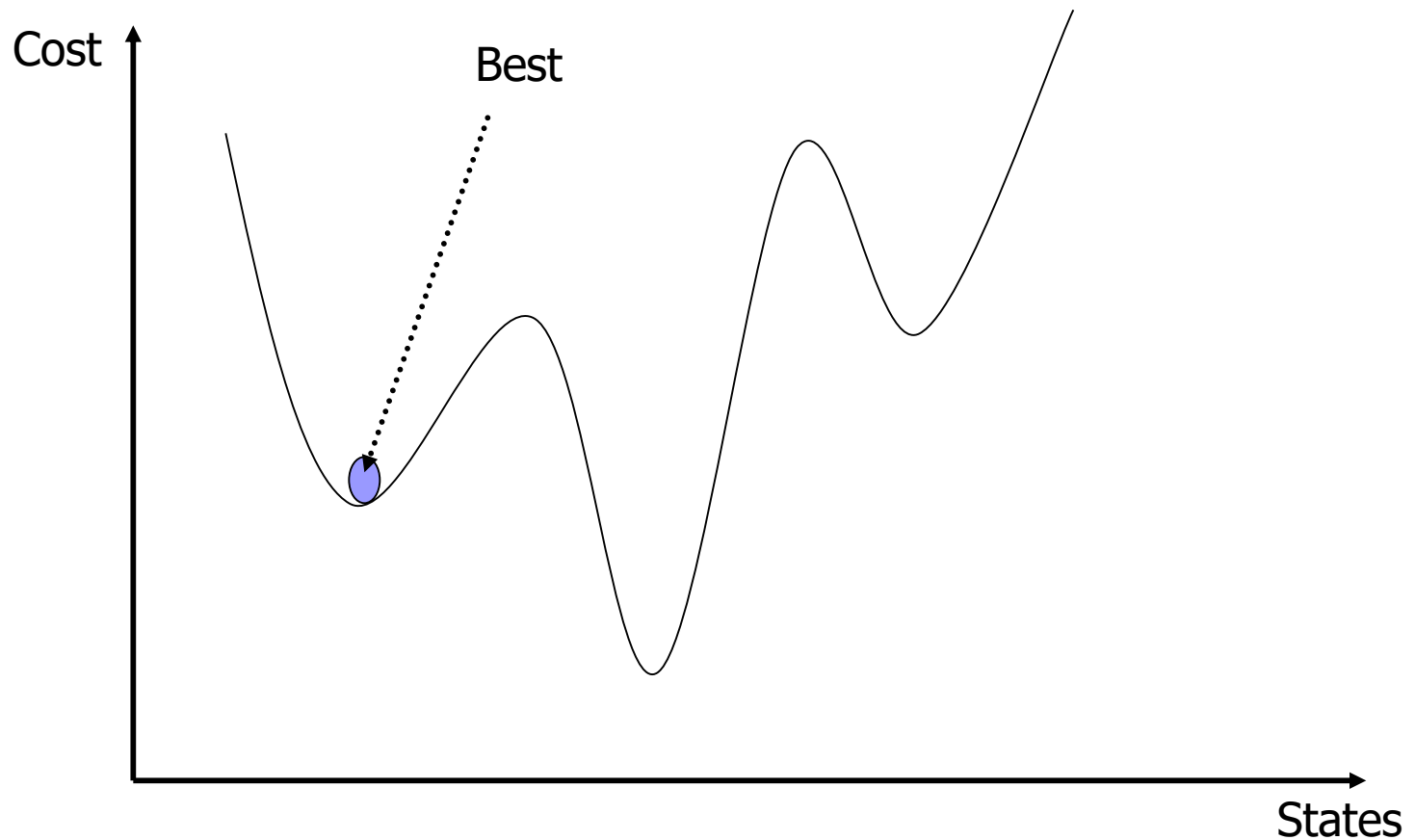
Simulated Annealing



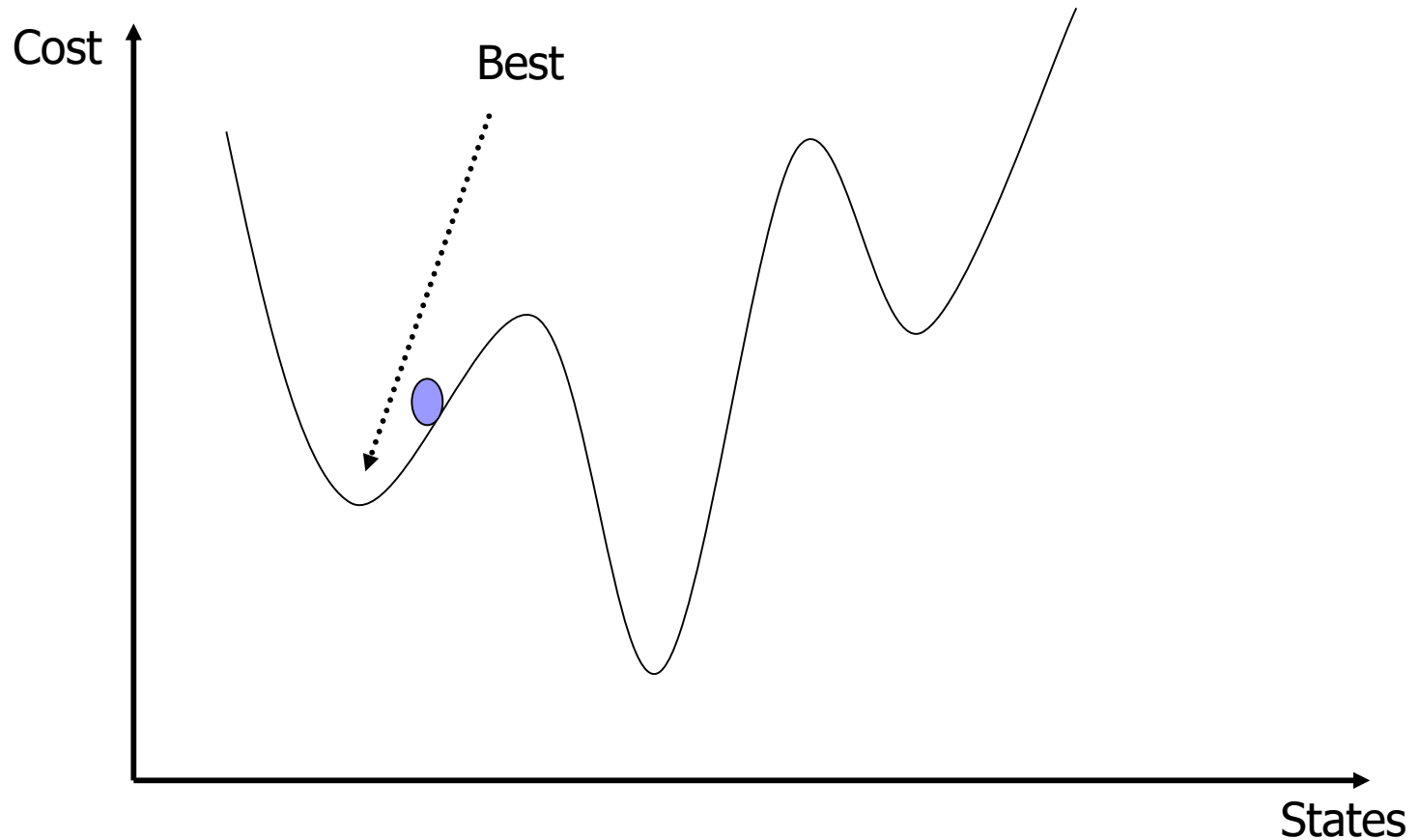
Simulated Annealing



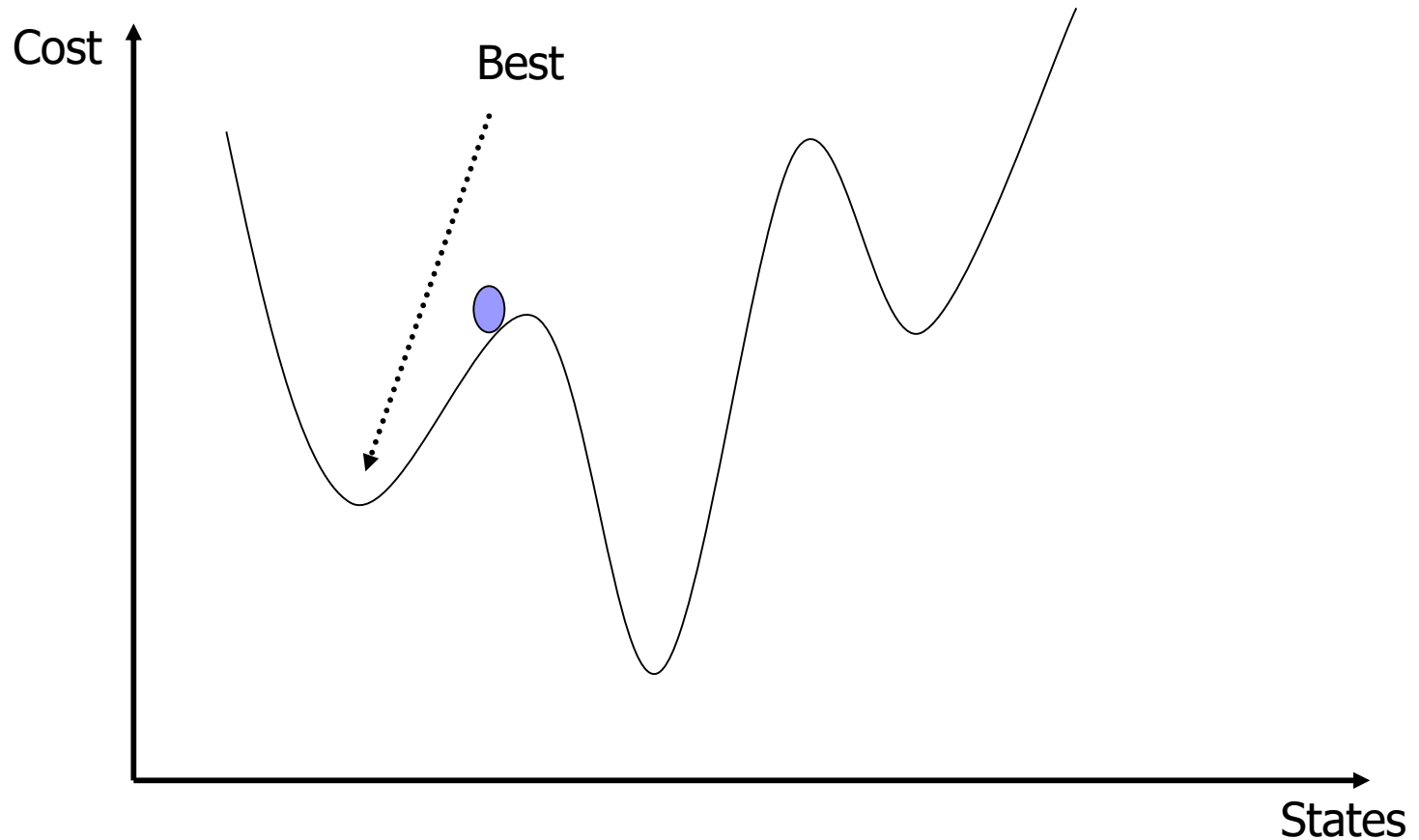
Simulated Annealing



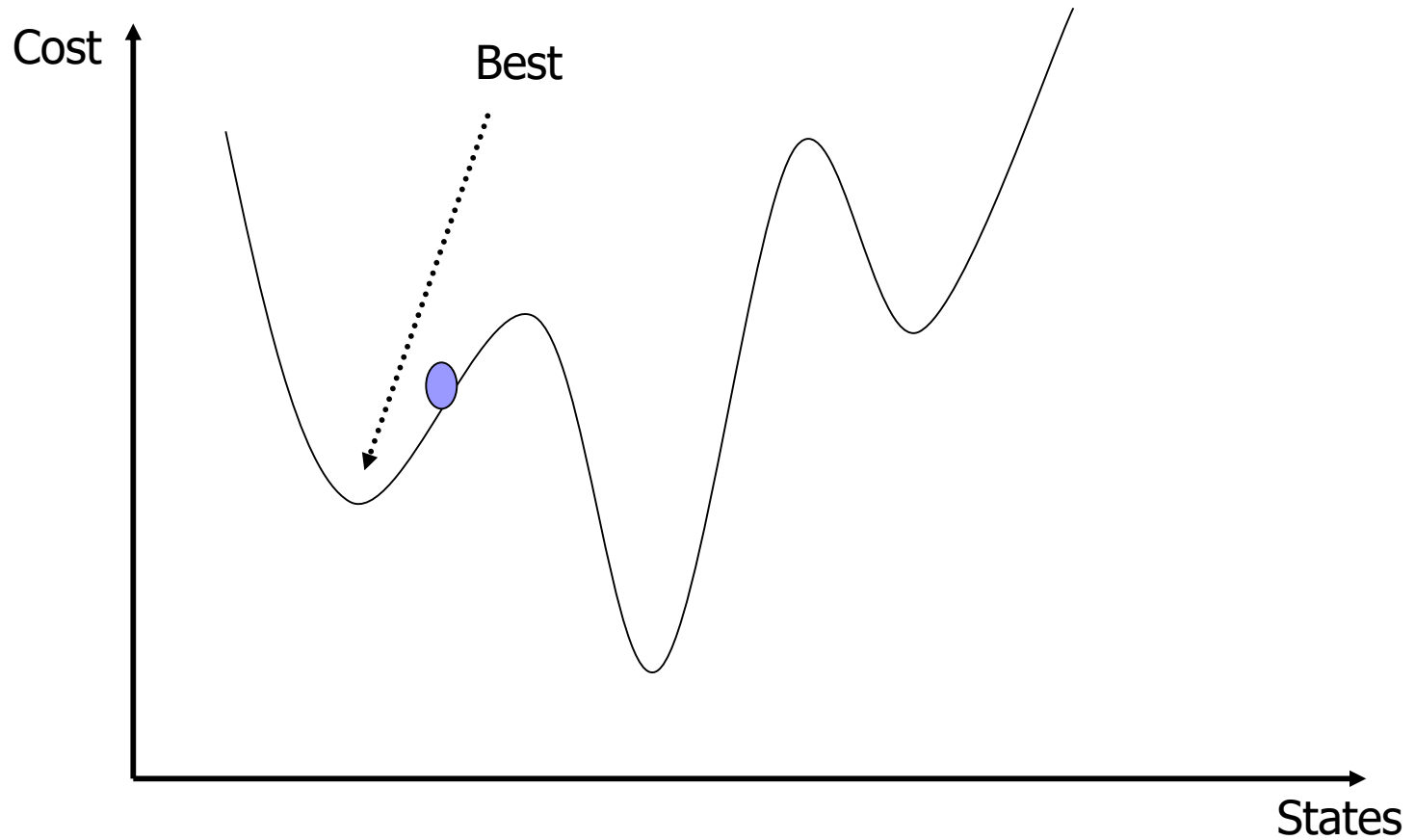
Simulated Annealing



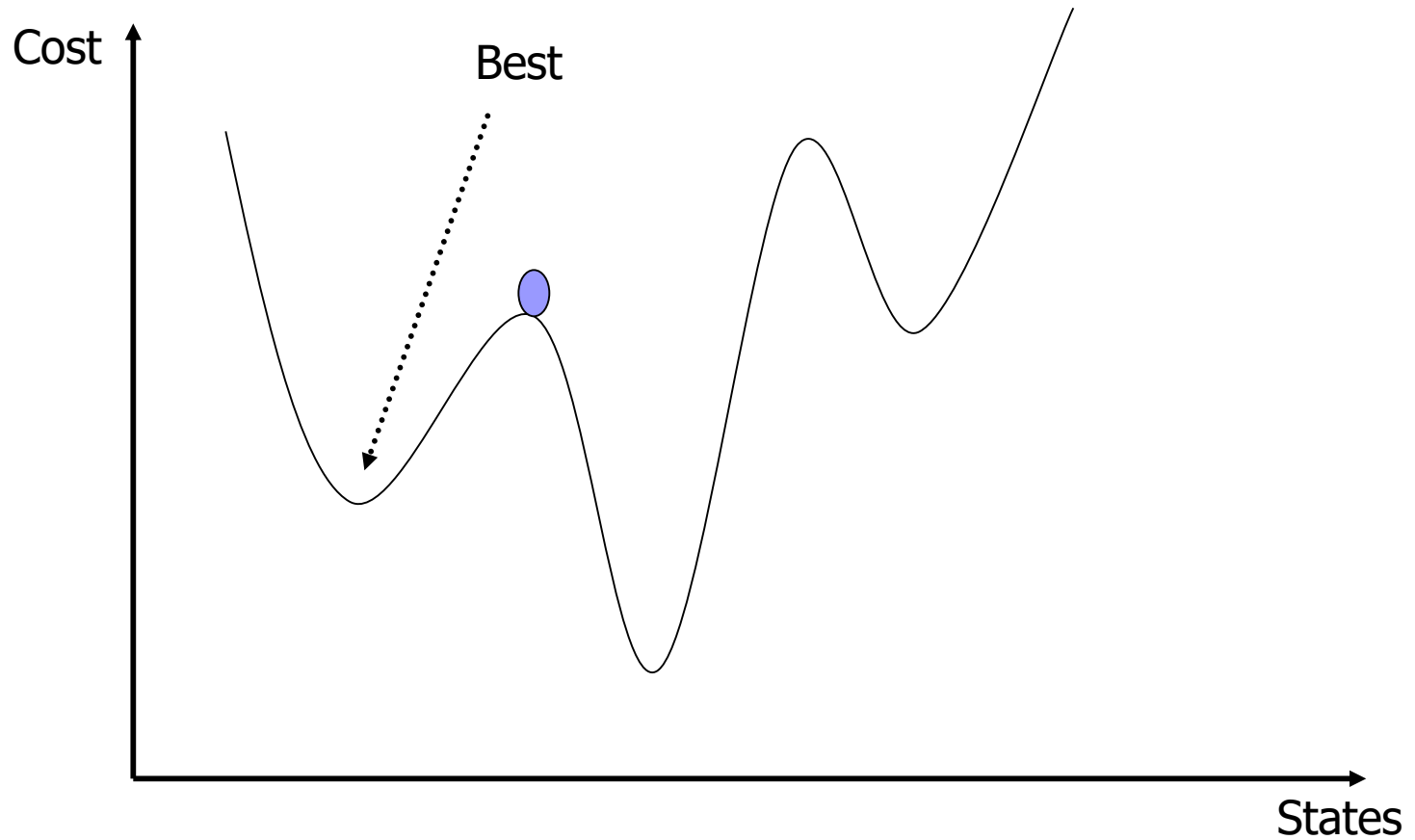
Simulated Annealing



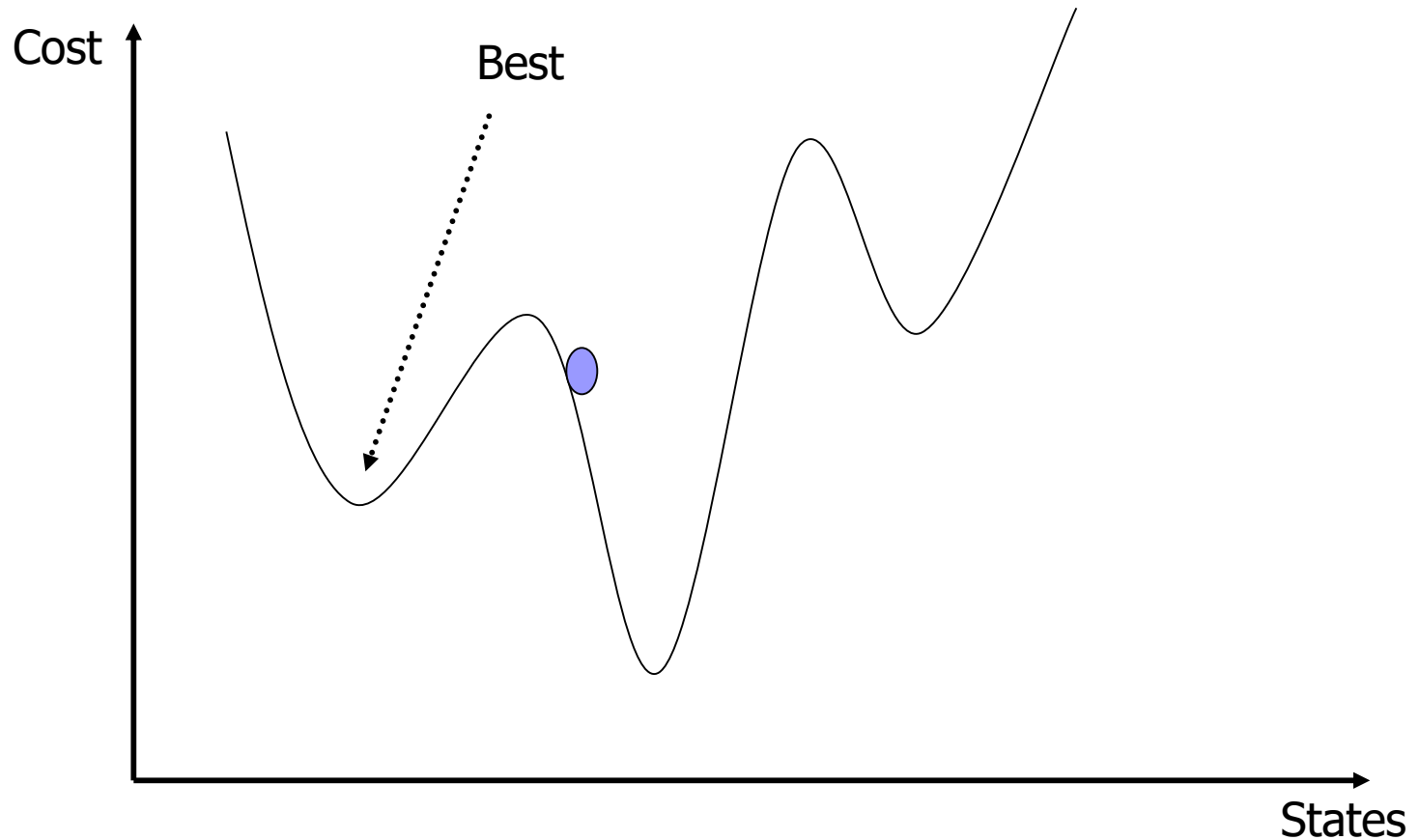
Simulated Annealing



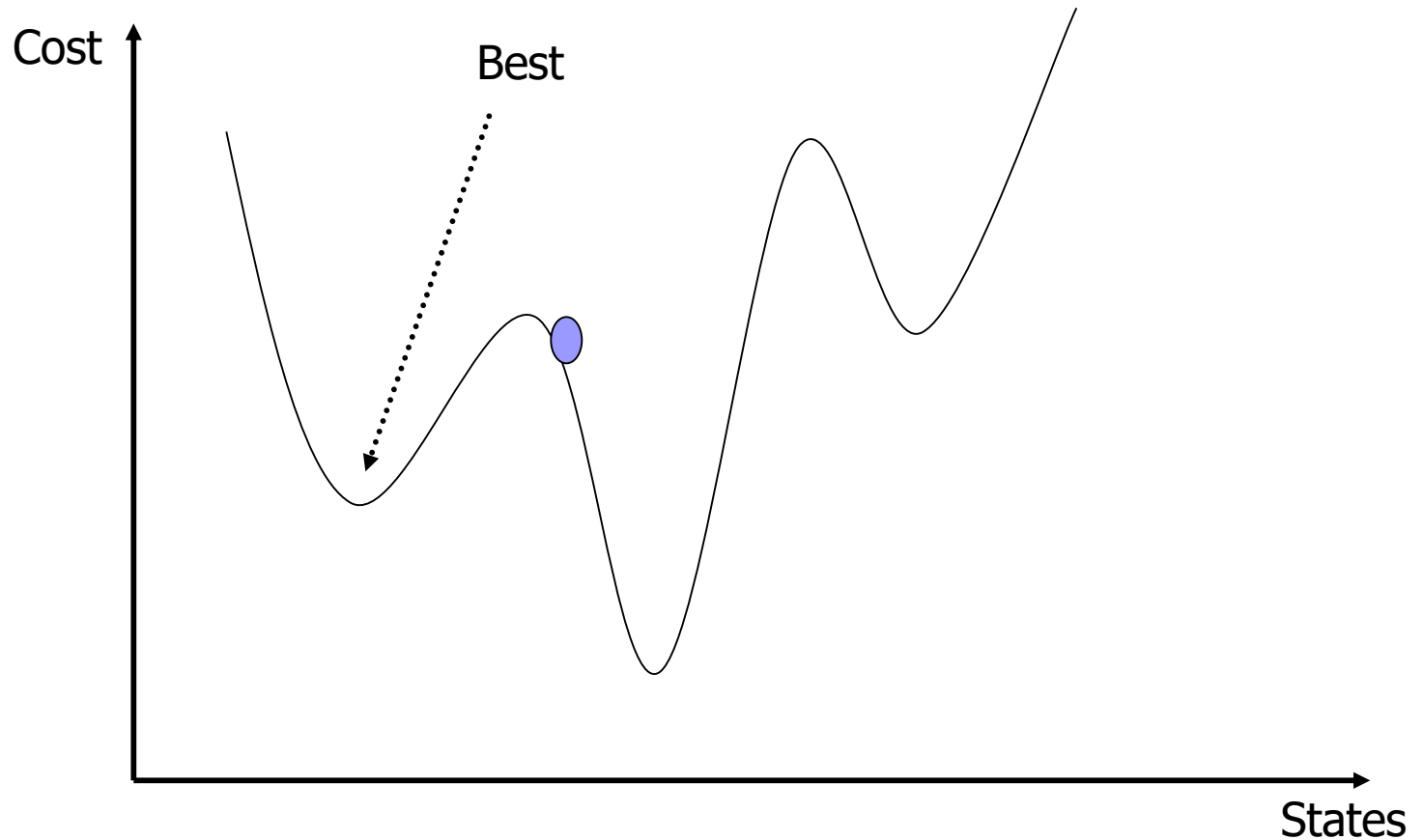
Simulated Annealing



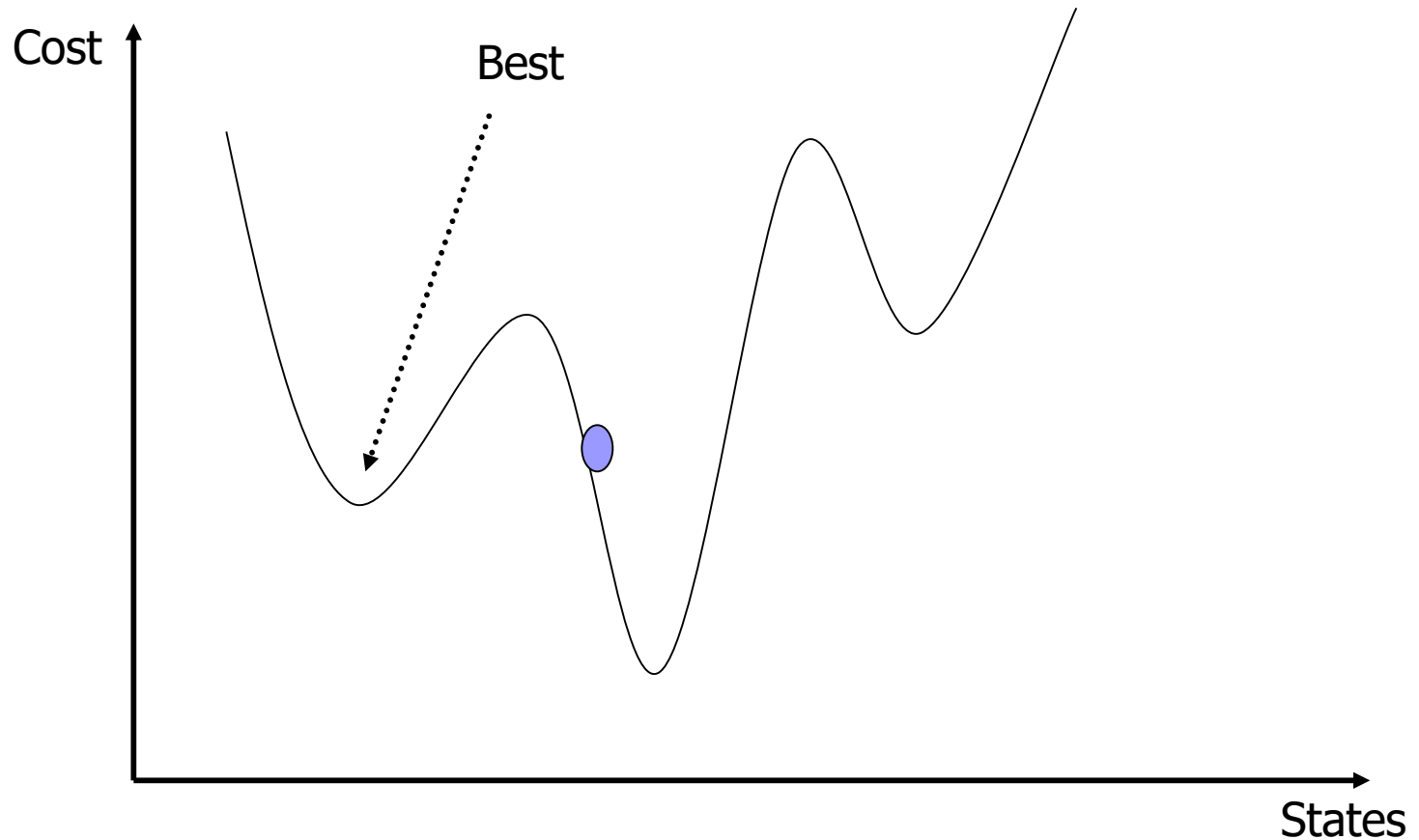
Simulated Annealing



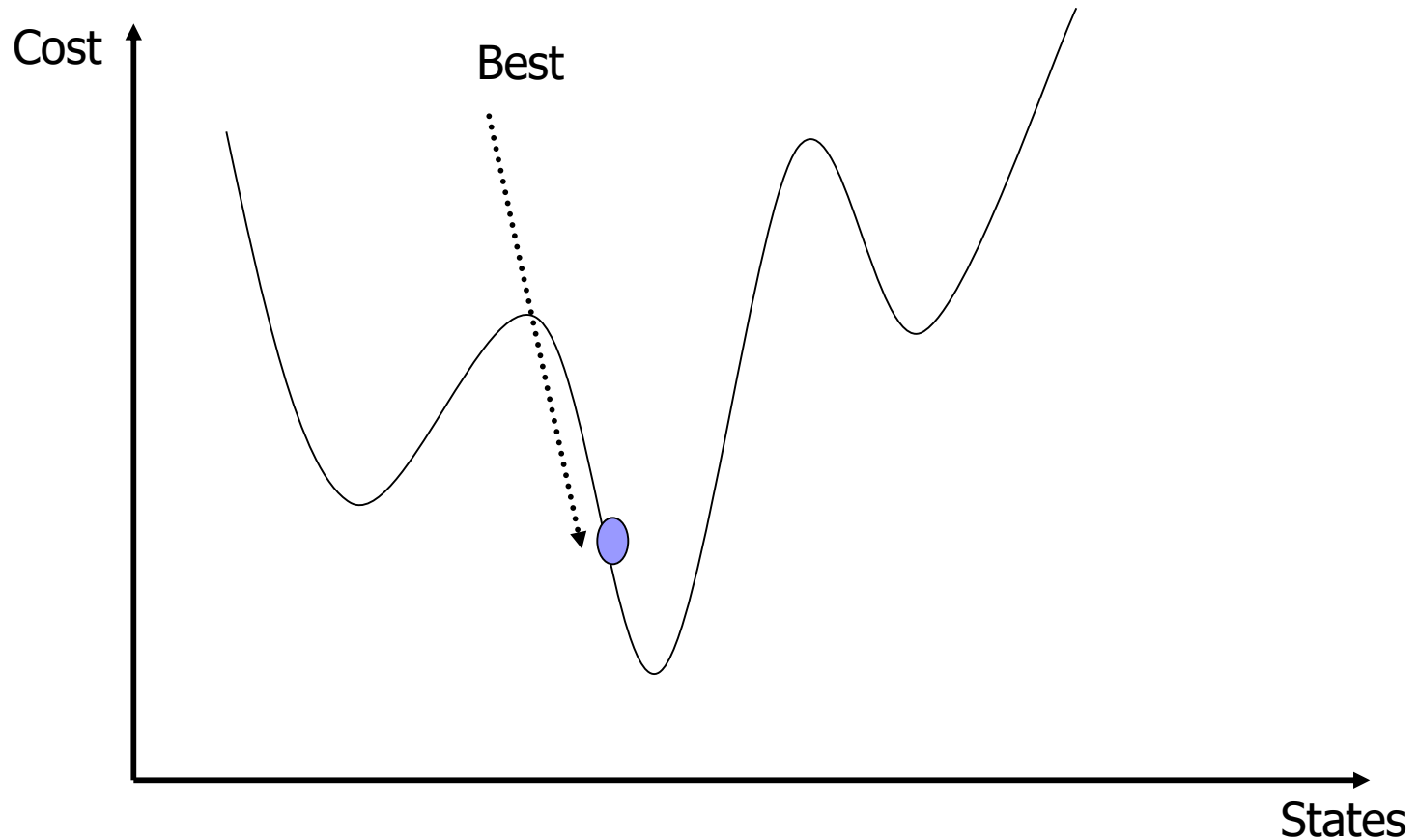
Simulated Annealing



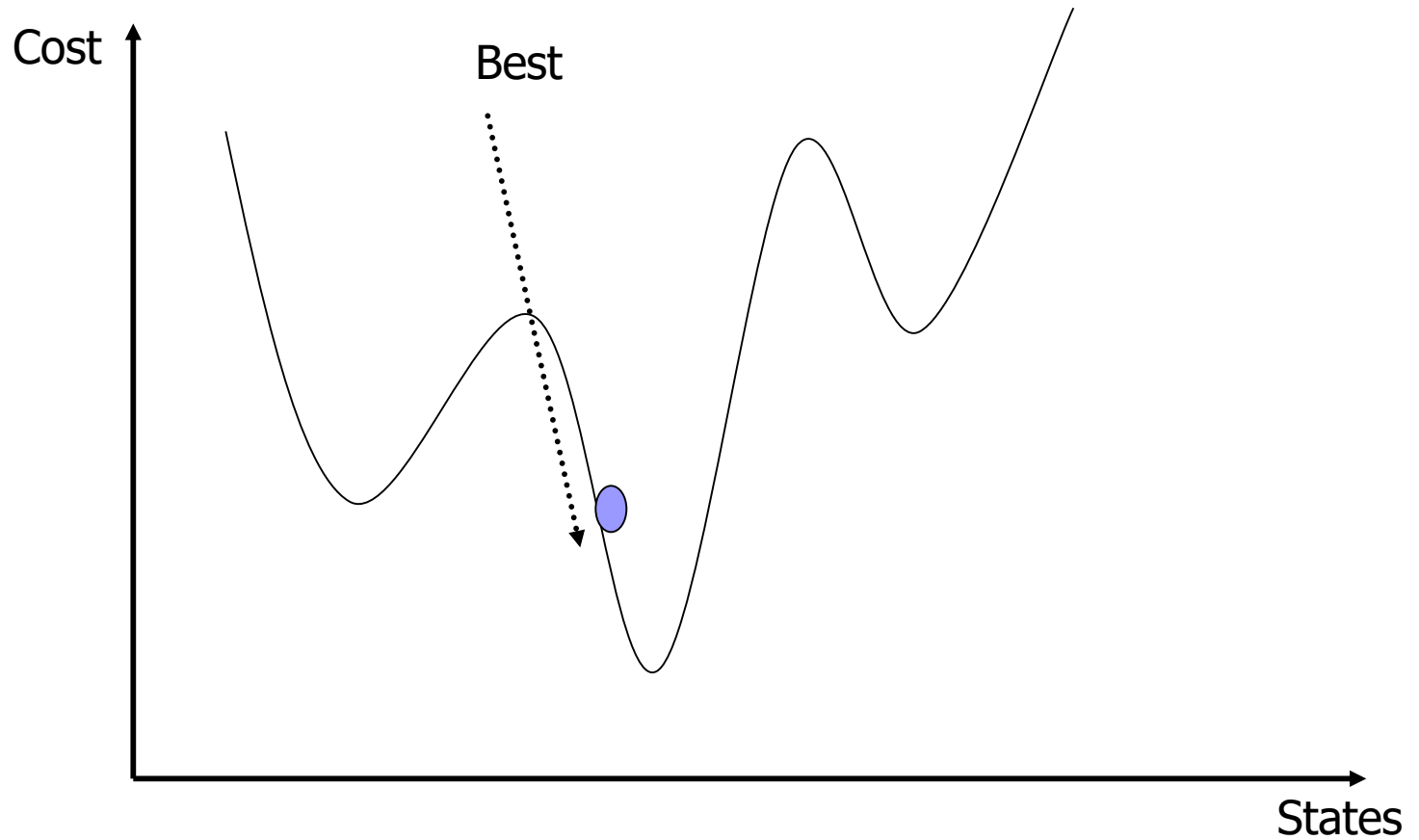
Simulated Annealing



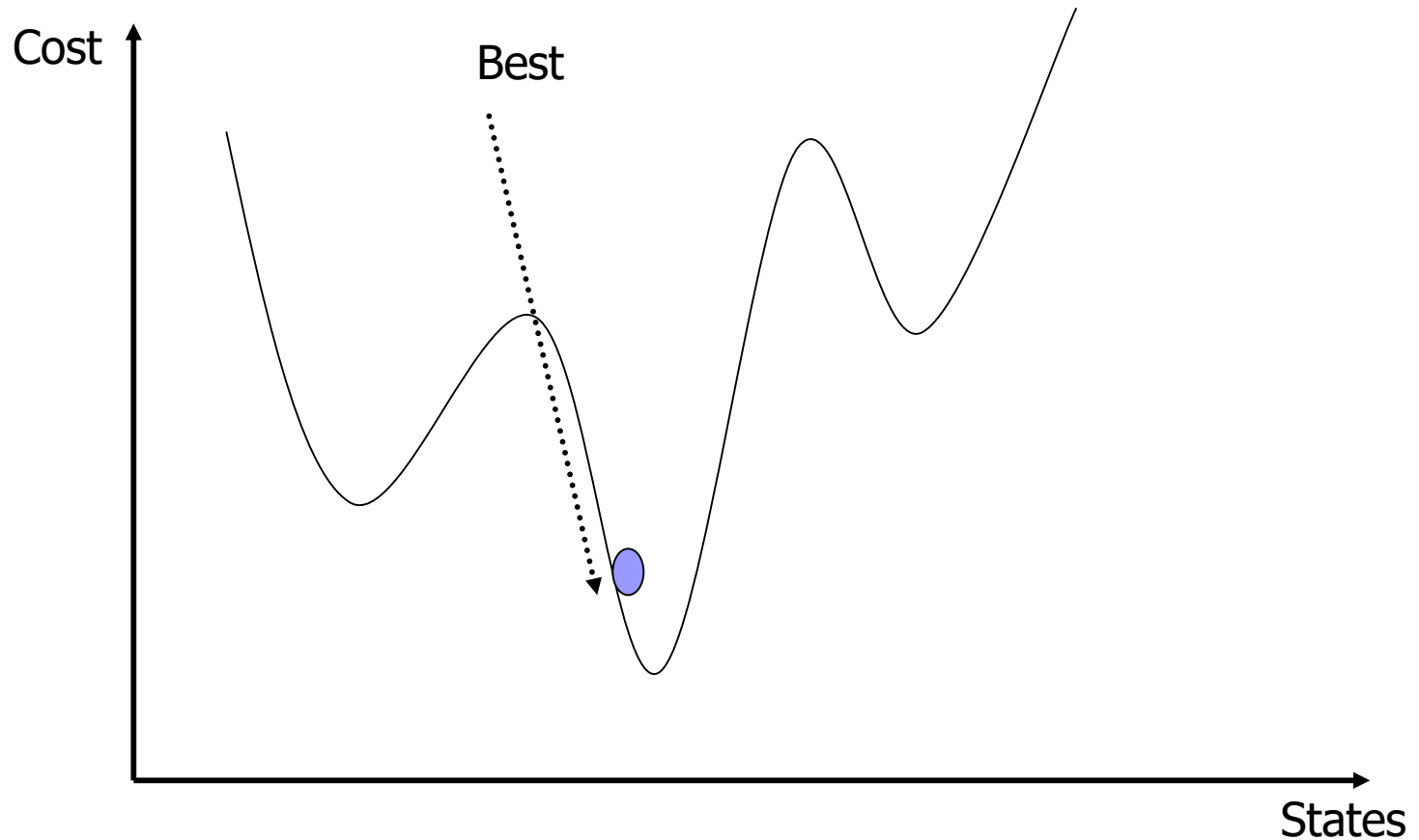
Simulated Annealing



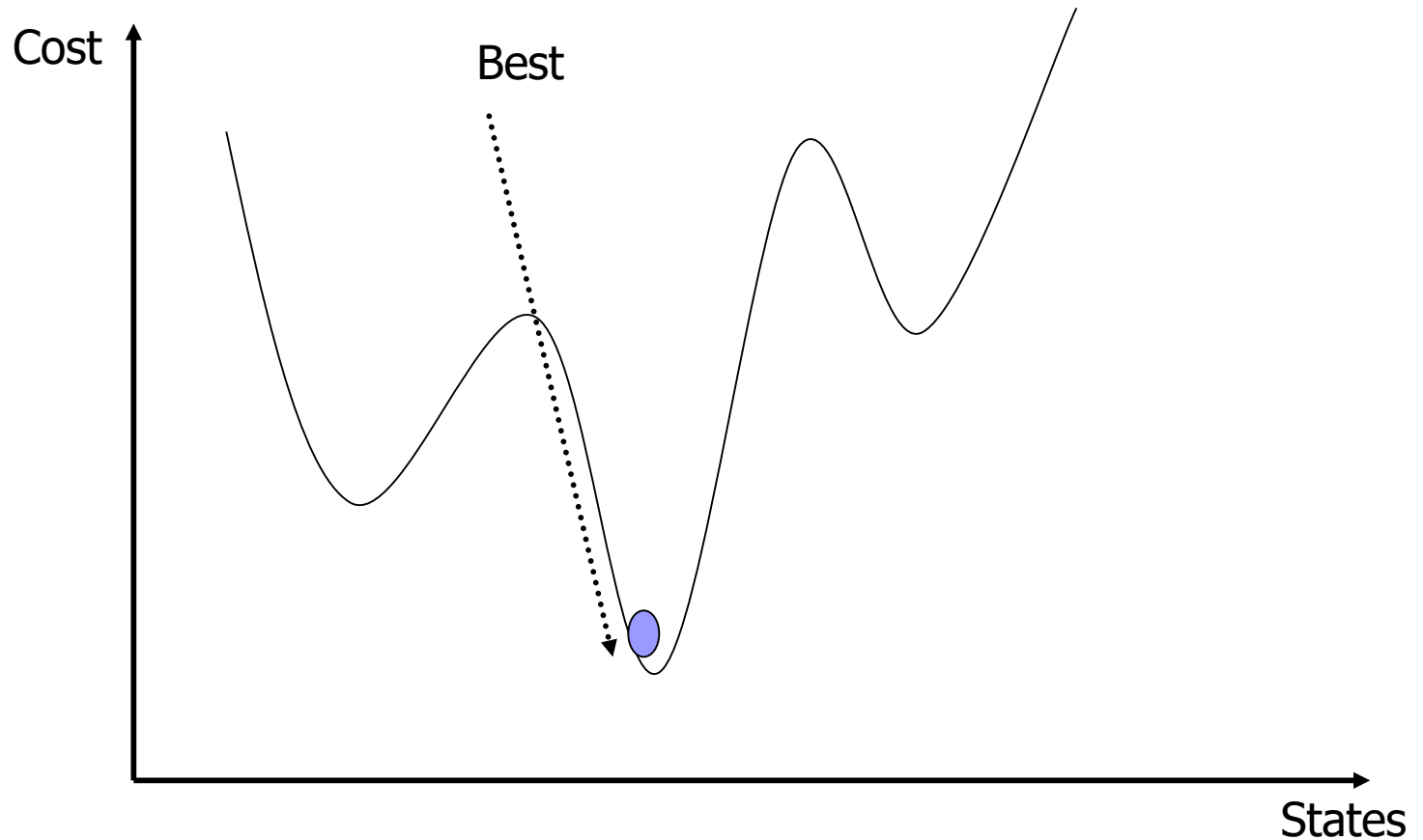
Simulated Annealing



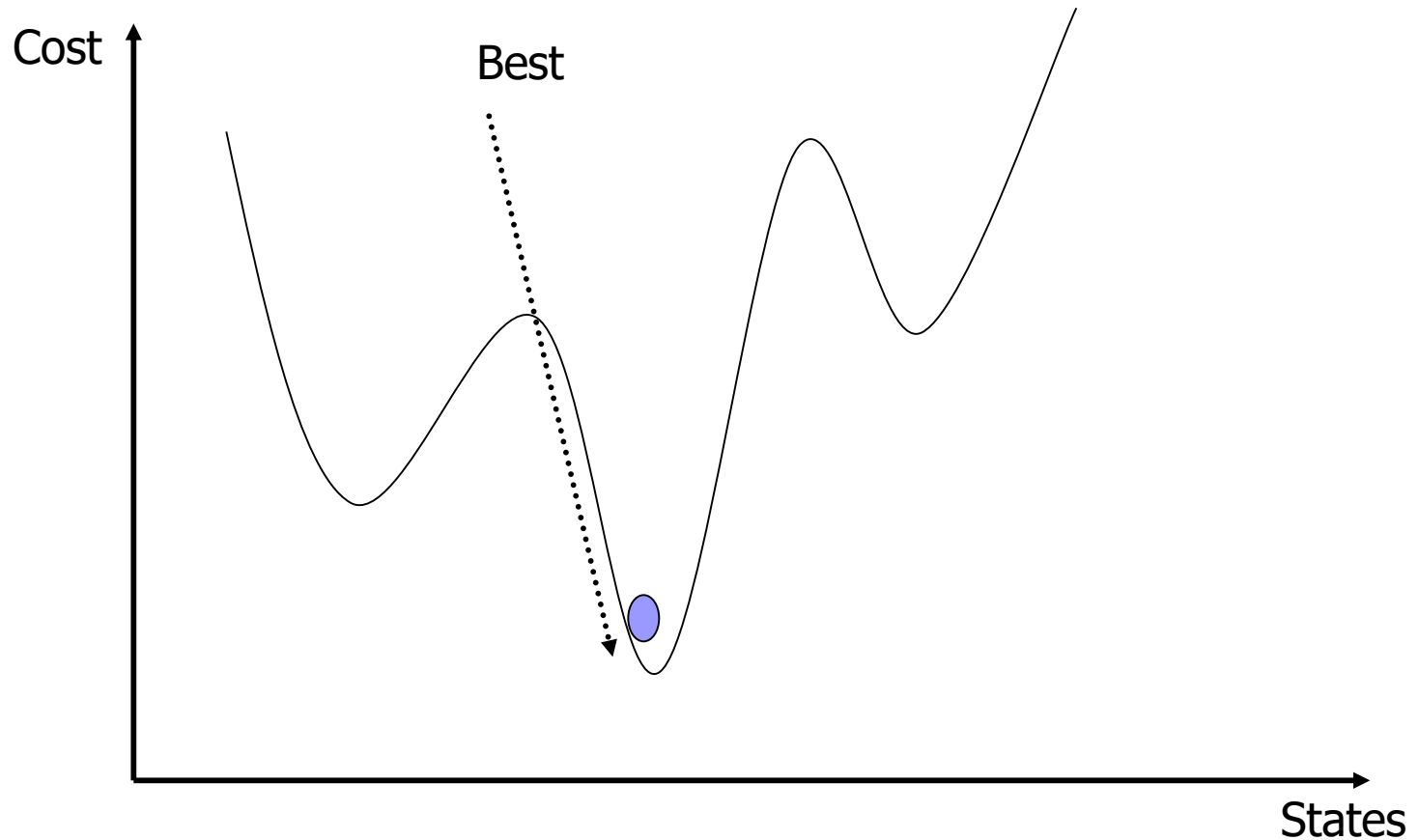
Simulated Annealing



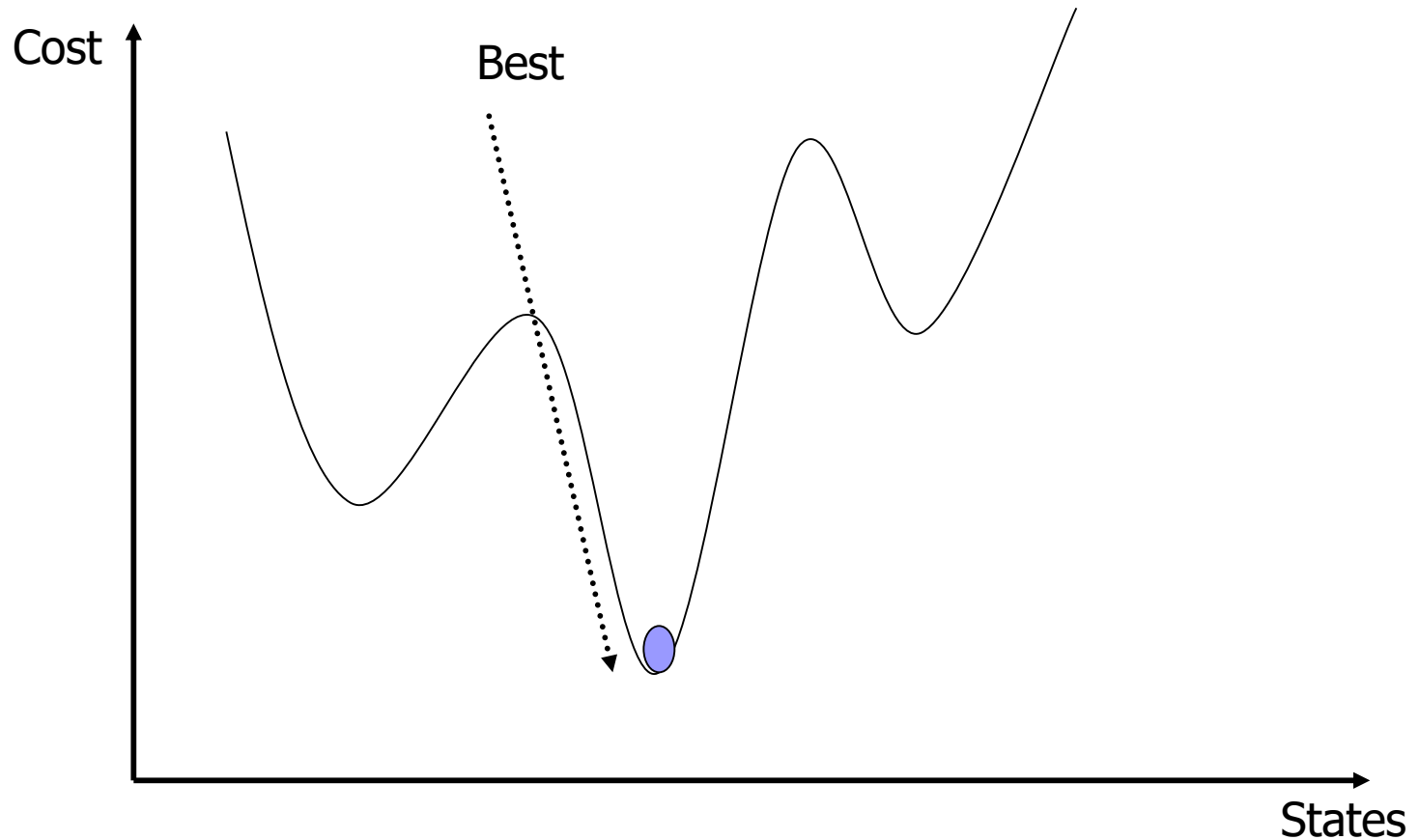
Simulated Annealing



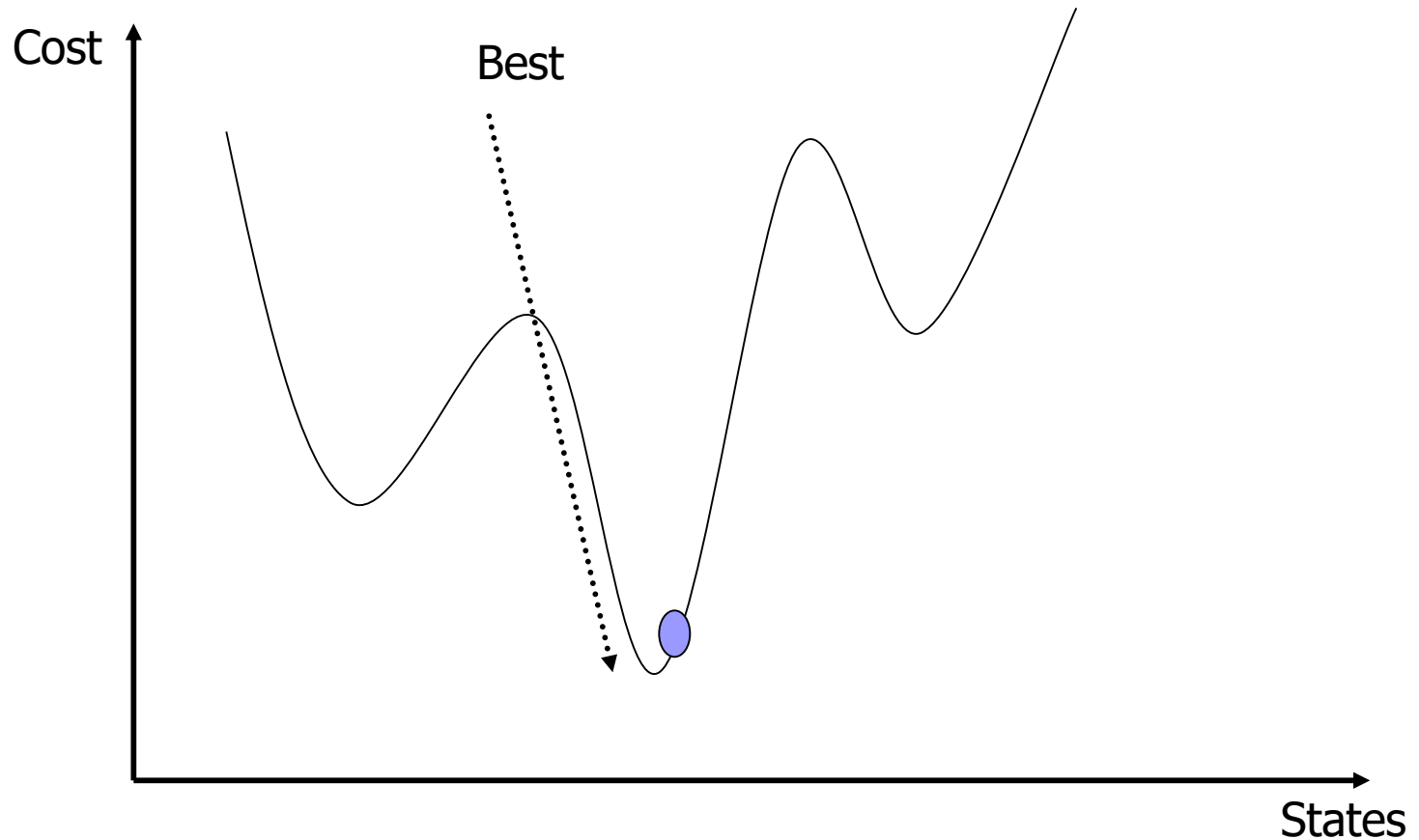
Simulated Annealing



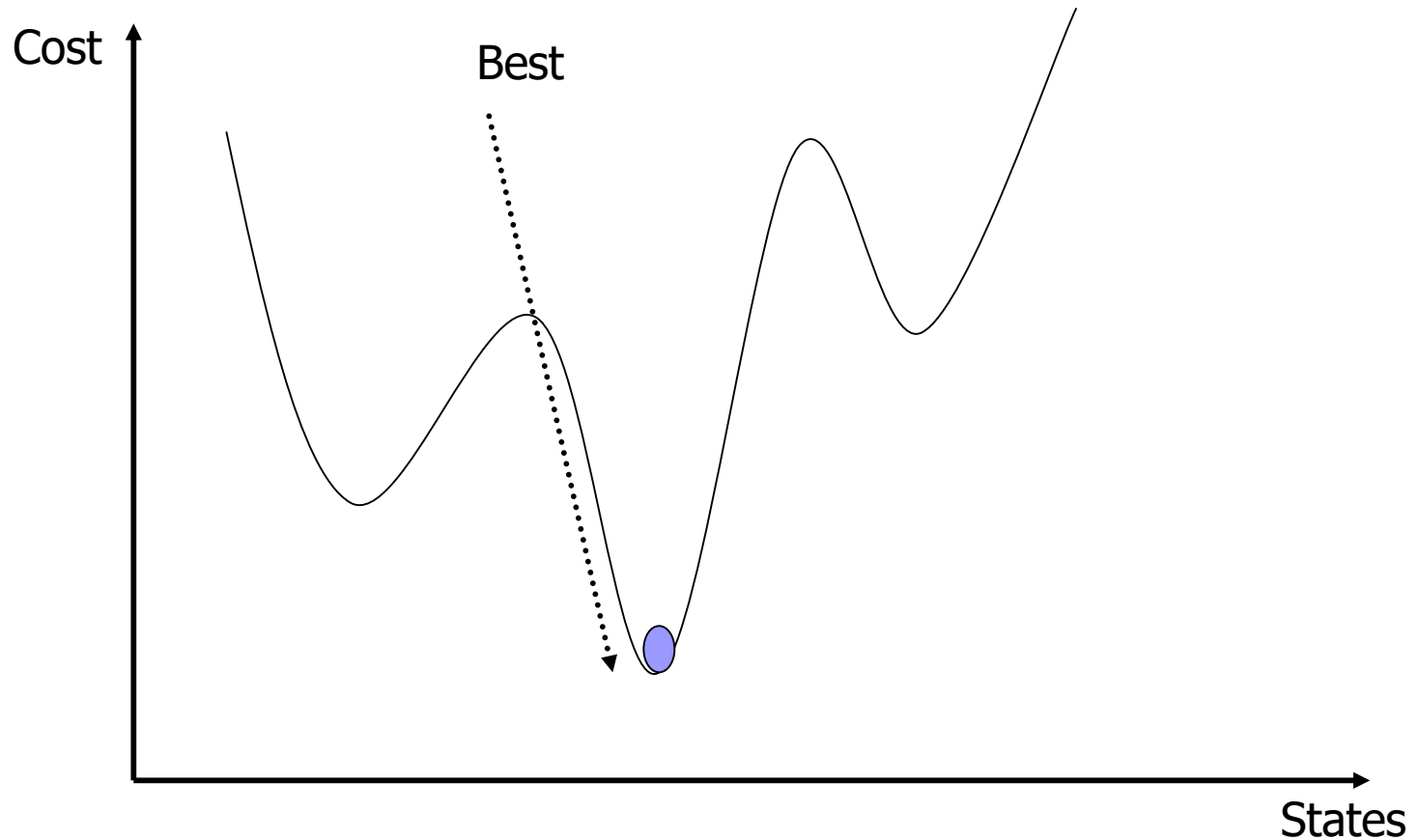
Simulated Annealing



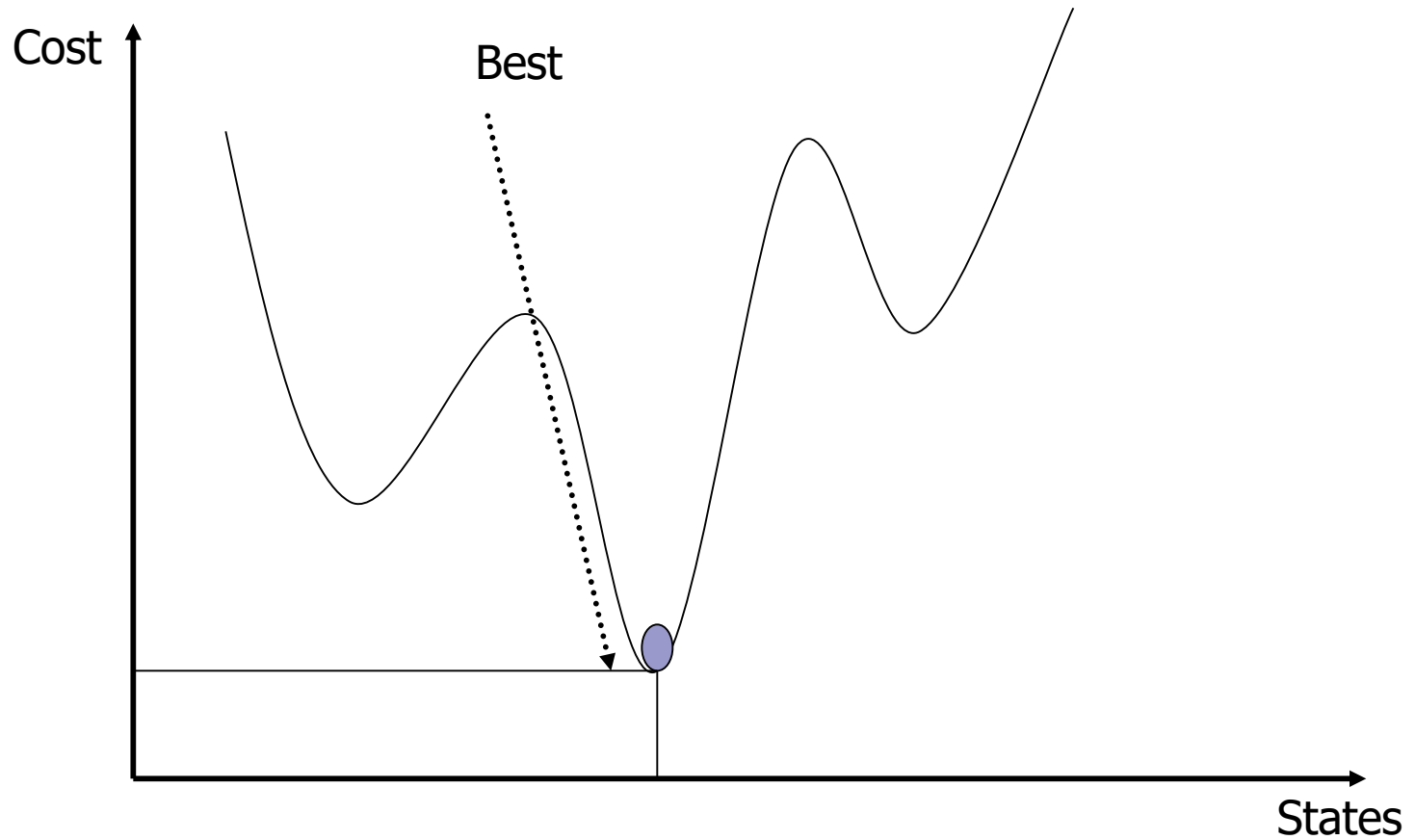
Simulated Annealing



Simulated Annealing



Simulated Annealing



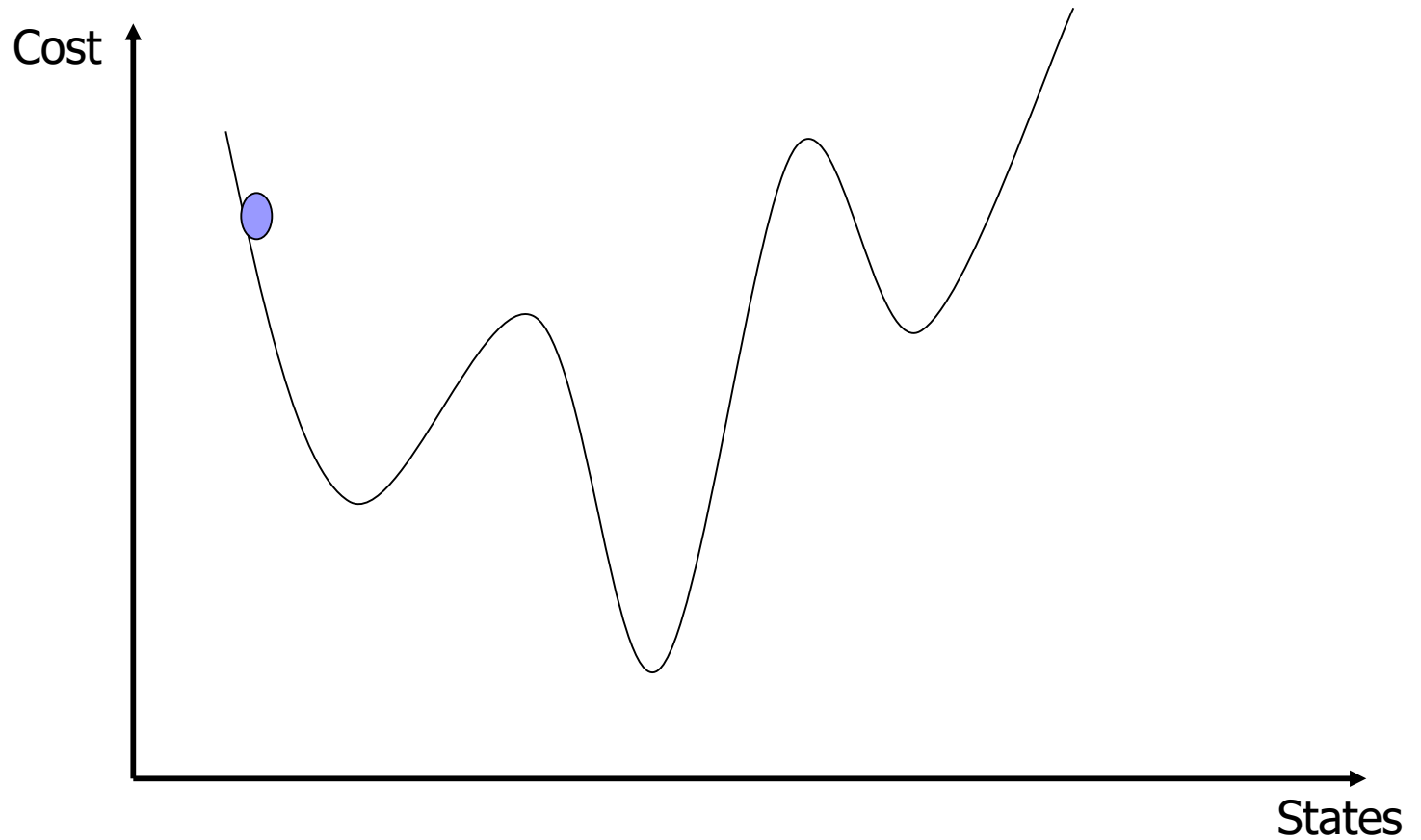
Xuất xứ, ứng dụng

- Được đề xuất bởi nhóm nghiên cứu của IBM năm 1983 để giải bài toán bố trí (layout) VLSI
 - Về lý thuyết sẽ tìm được tối ưu toàn cục
- Các ứng dụng khác
 - TSP, Graph partitioning, Graph coloring, Scheduling, Facility Layout, Image Processing,...
- Hữu ích nhưng tốc độ chậm
 - Do giảm T chậm để đạt được tối ưu

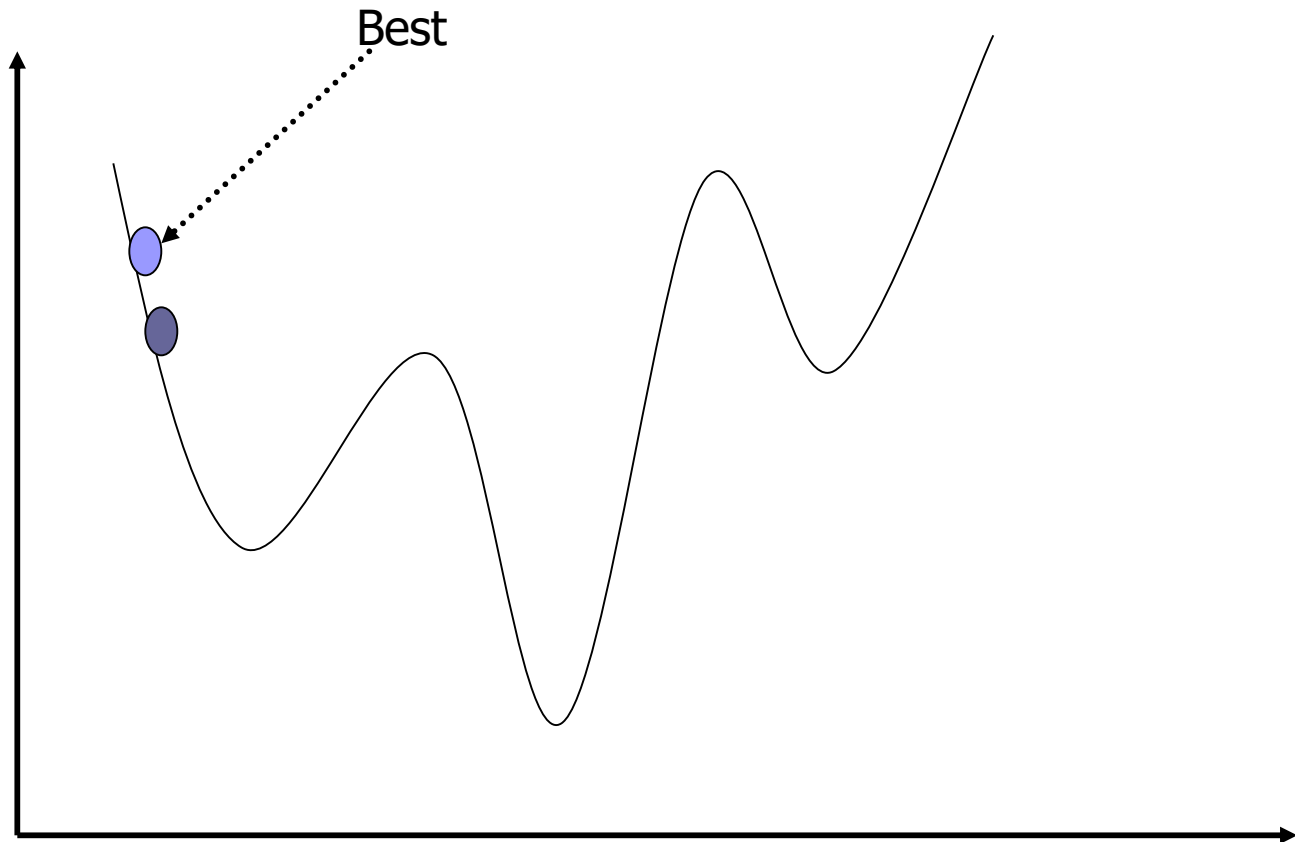
Tabu Search

- Cải tiến hill-climbing bằng cách cấm việc quay lại trạng thái đã duyệt
- Lập một hàng đợi trạng thái đã duyệt gần nhất (“tabu list” – danh sách cấm)
 - Thêm trạng thái mới nhất, xóa cũ nhất
 - Không quay lại trạng thái trong tabu list
- Tính chất
 - Khi danh sách đủ lớn thì hill-climbing tiệm cận tới duyệt không lặp
 - Với kích thước danh sách phù hợp thì áp dụng tốt cho nhiều bài toán

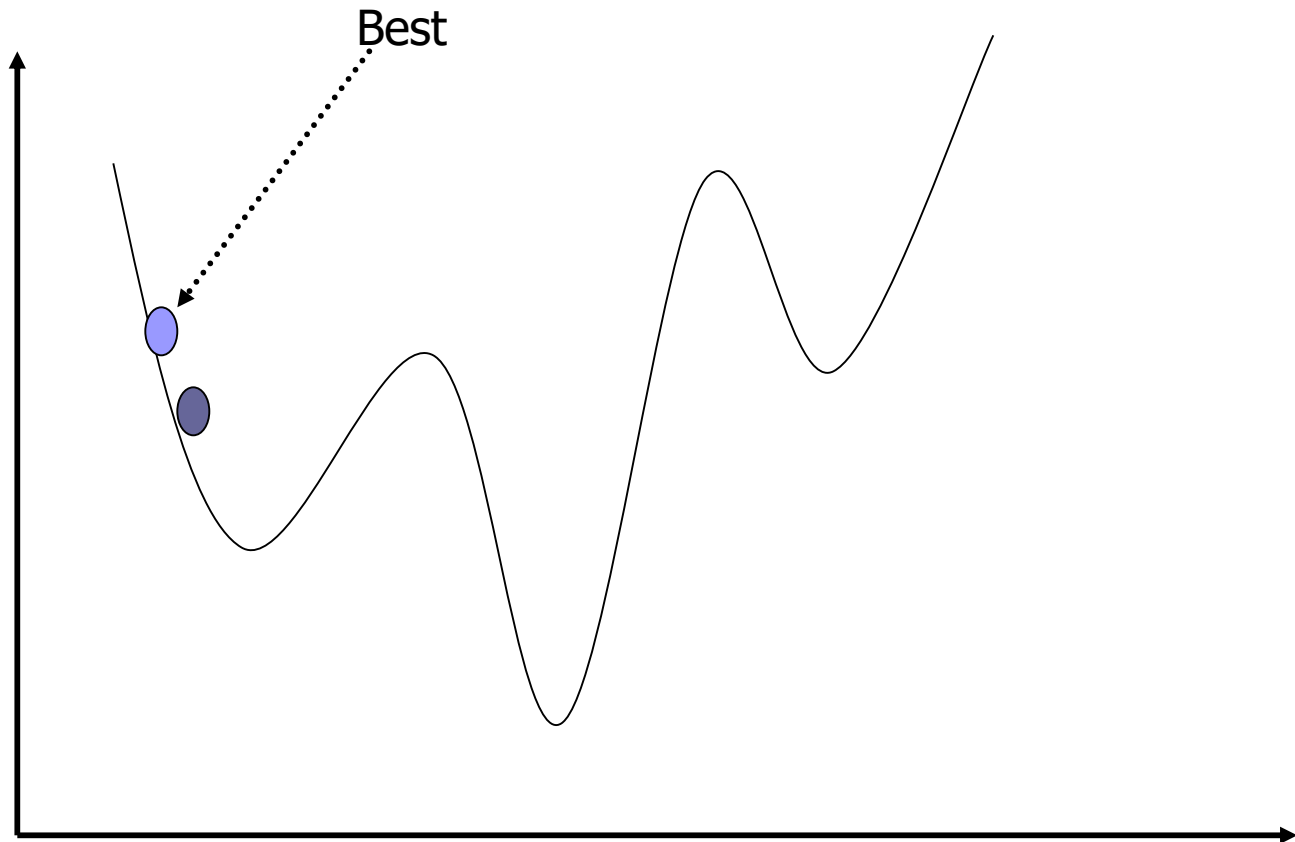
Tabu Search: TS



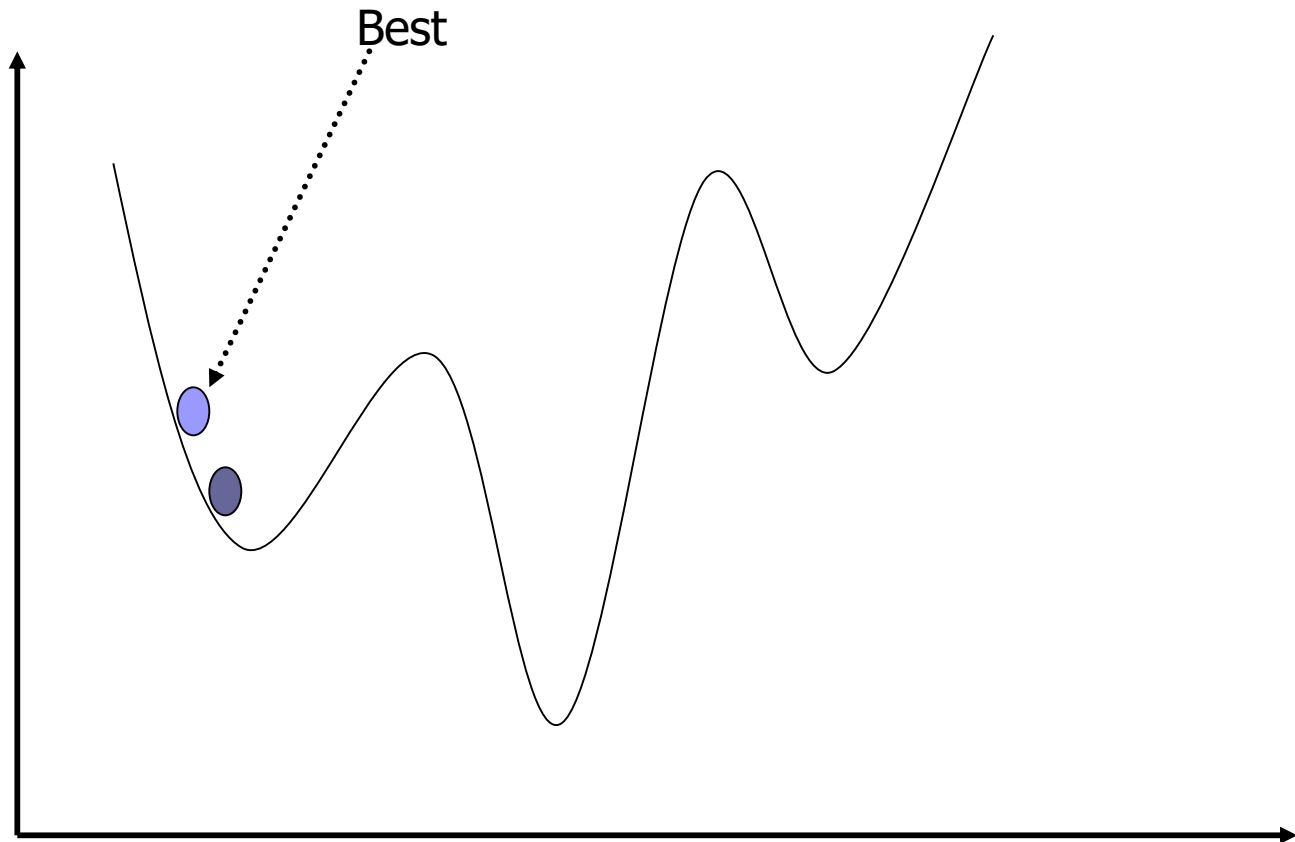
Tabu Search: TS



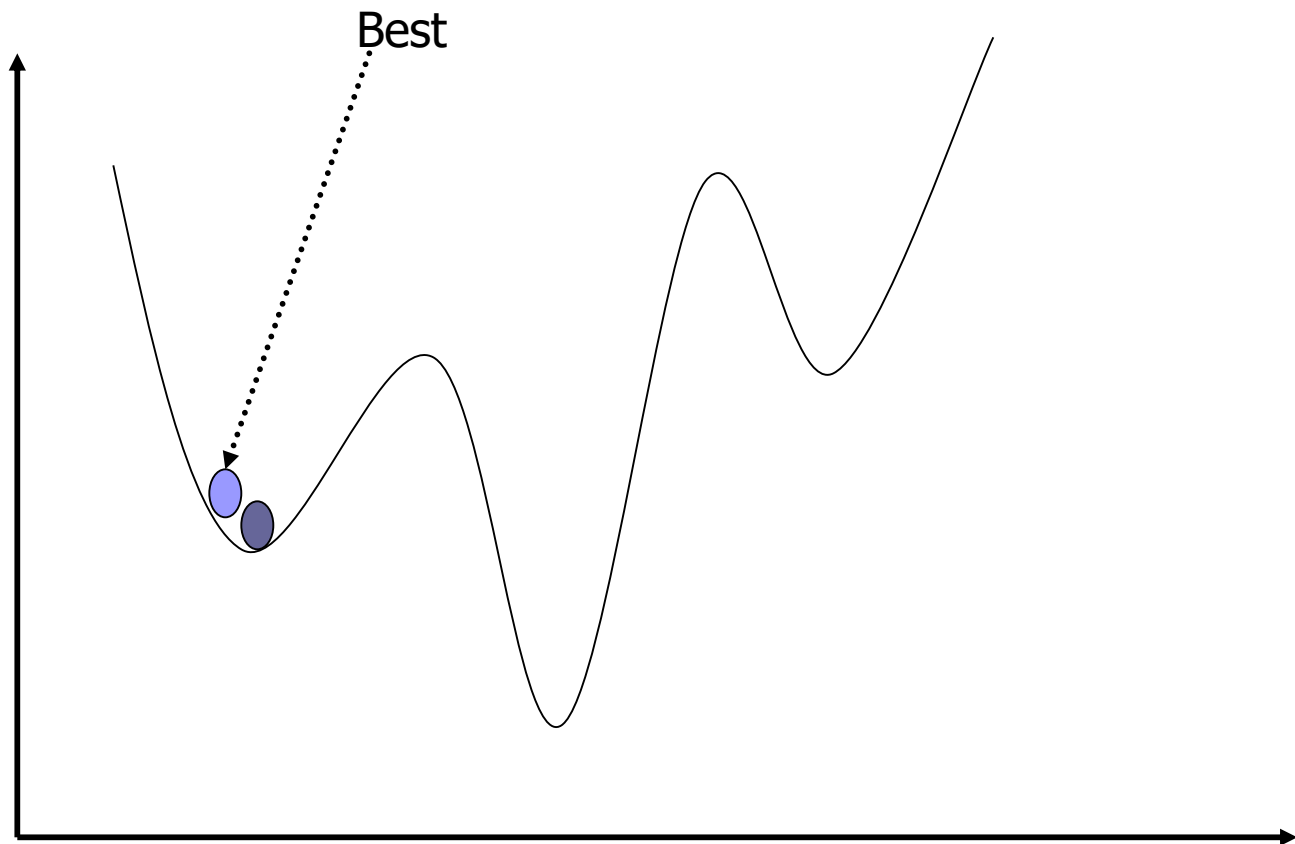
Tabu Search: TS



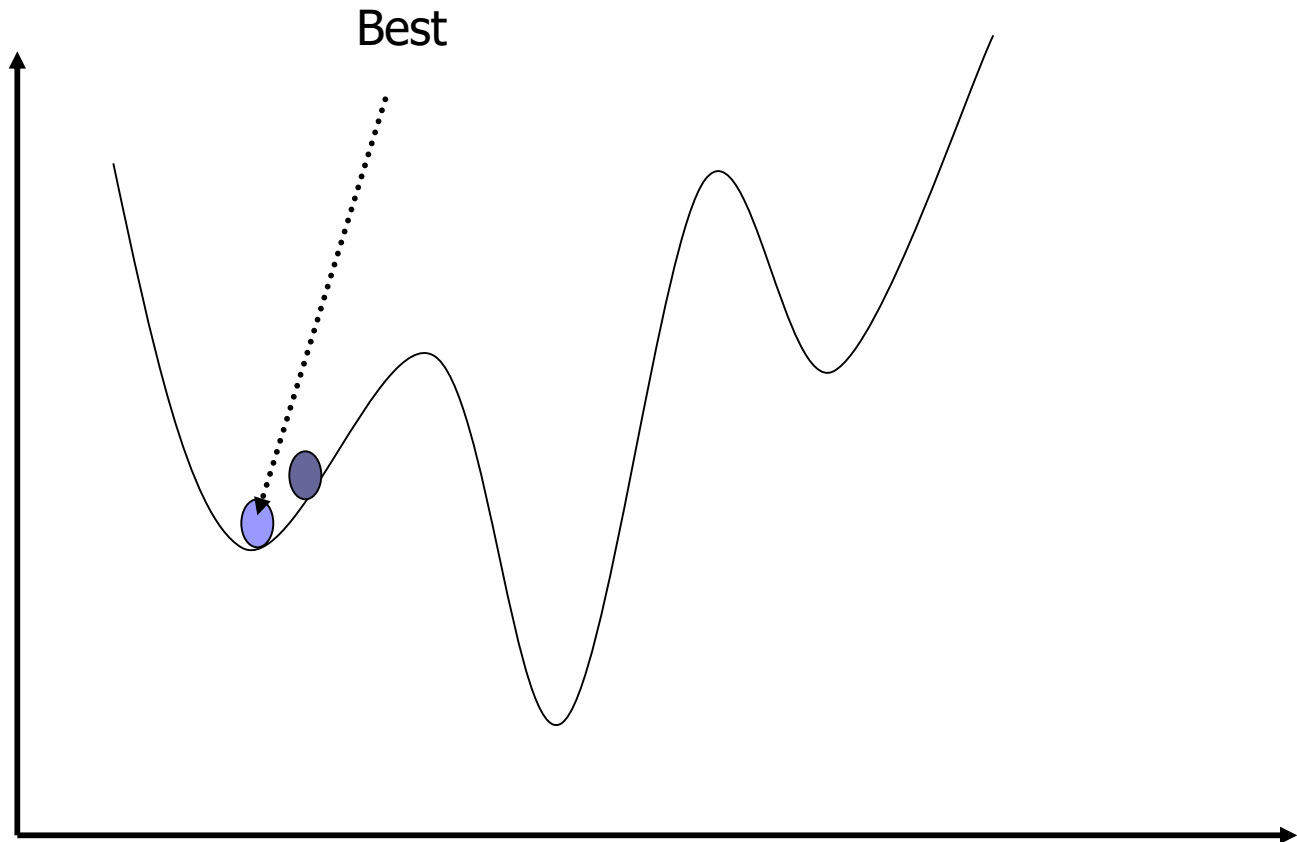
Tabu Search: TS



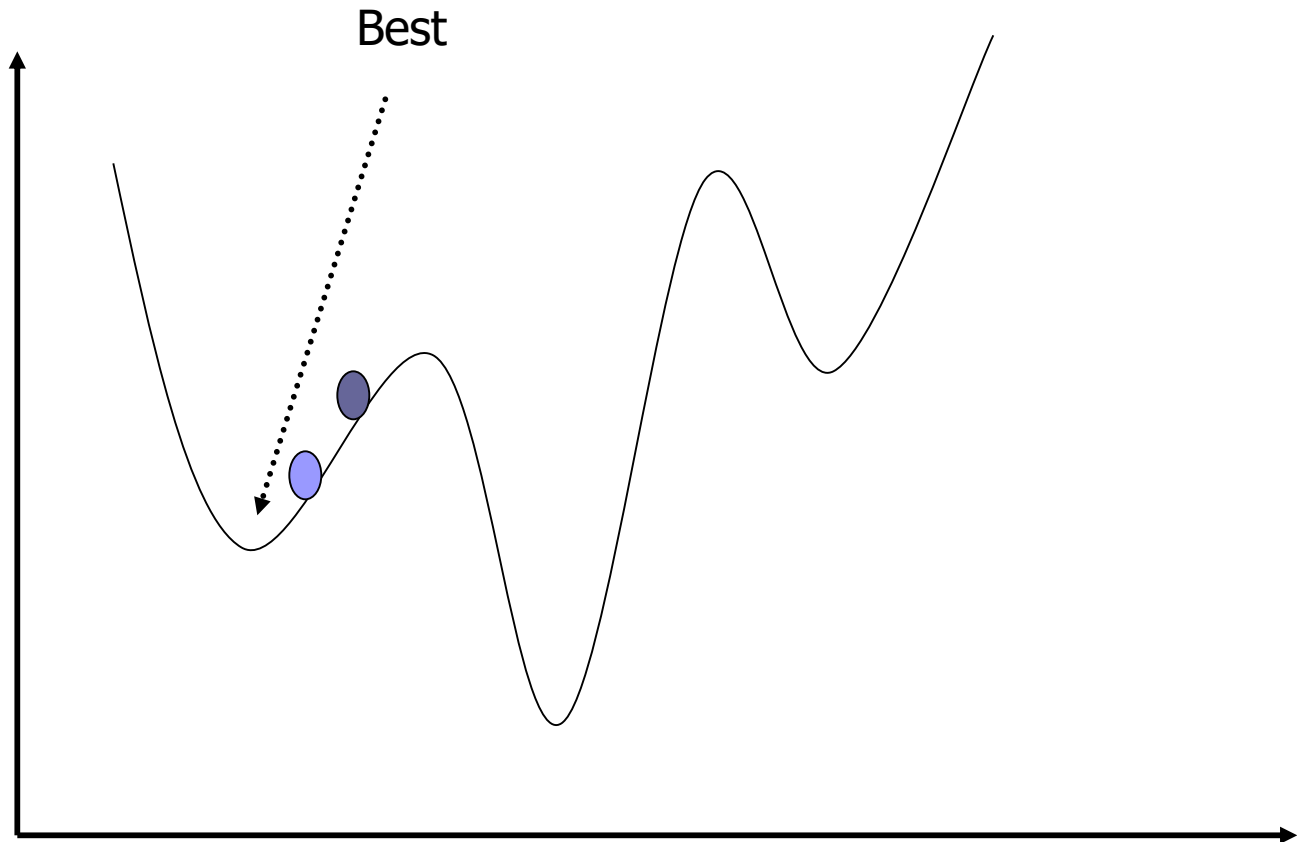
Tabu Search: TS



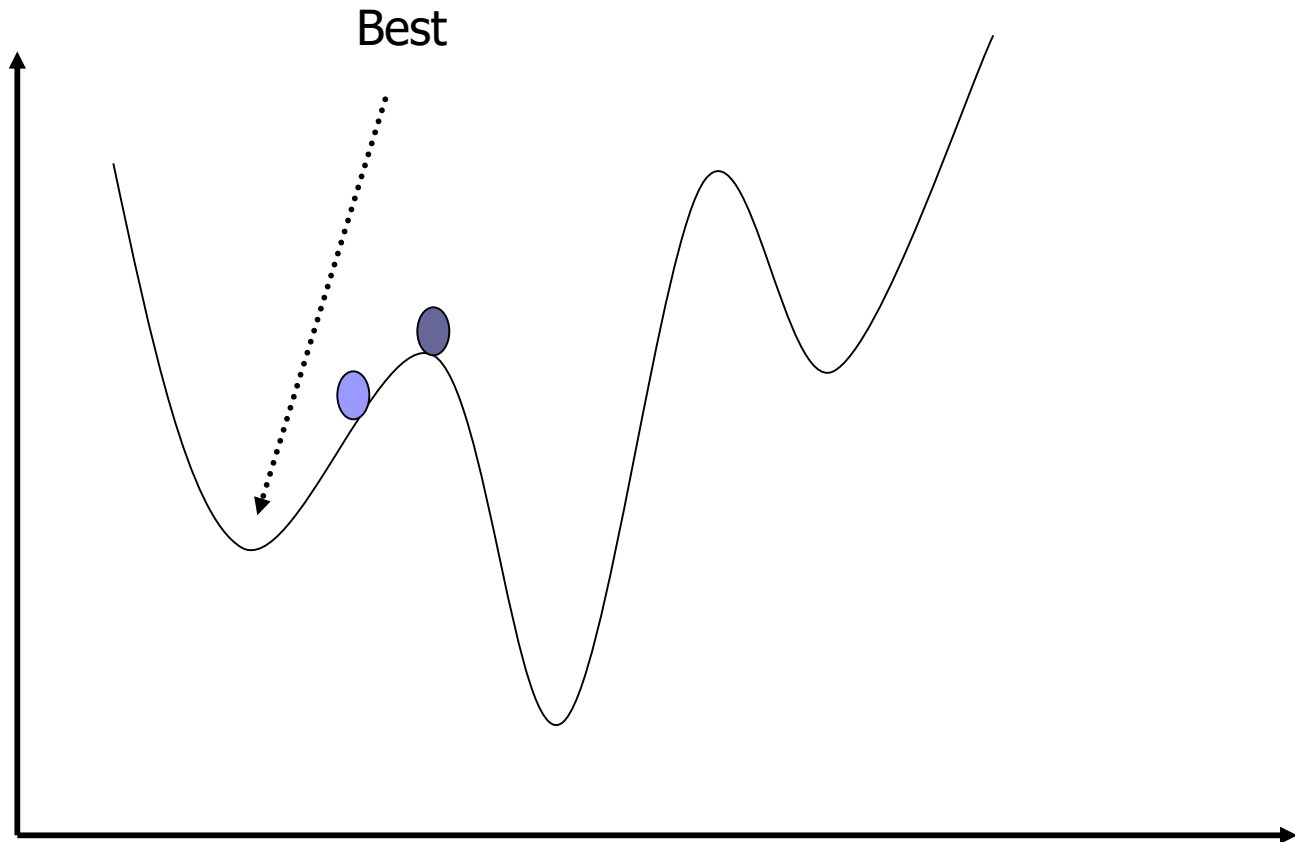
Tabu Search: TS



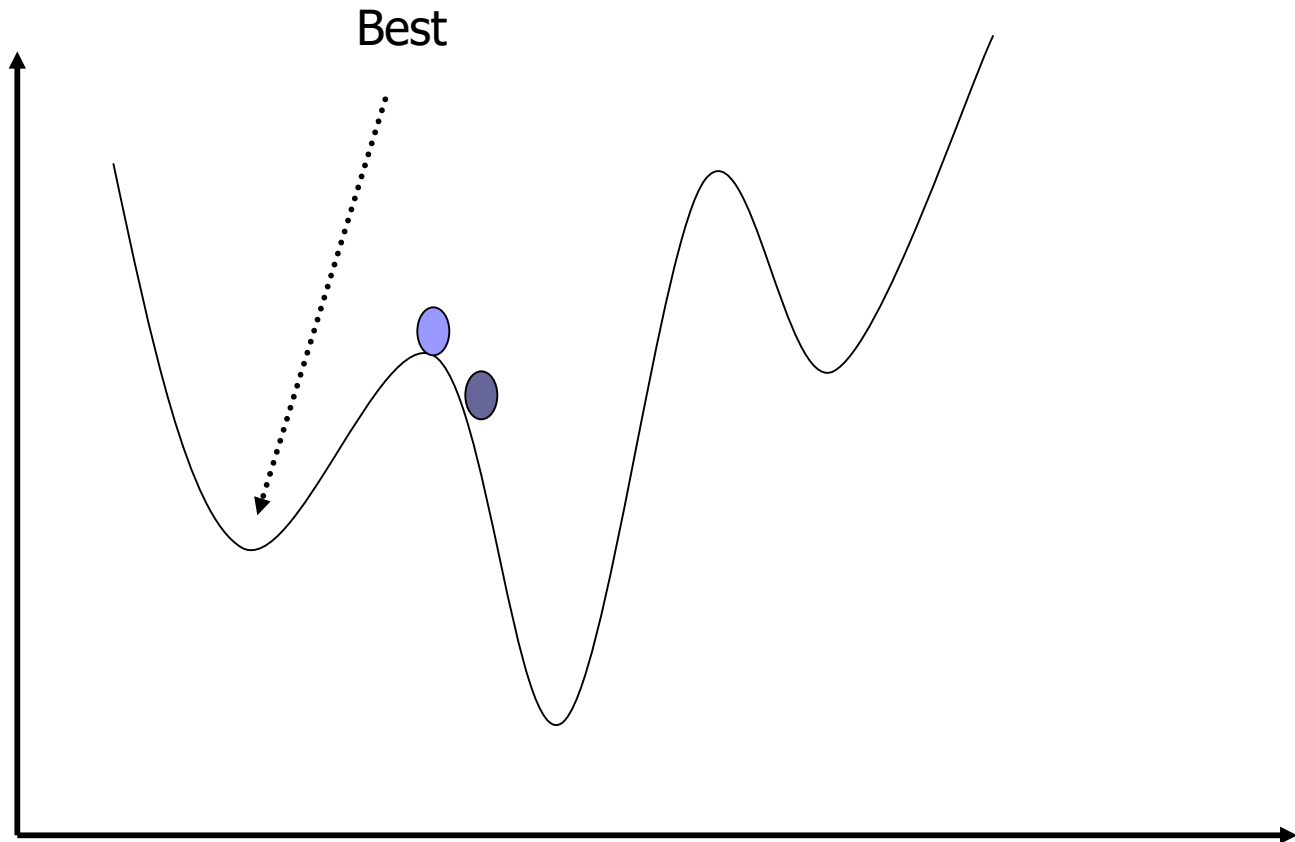
Tabu Search: TS



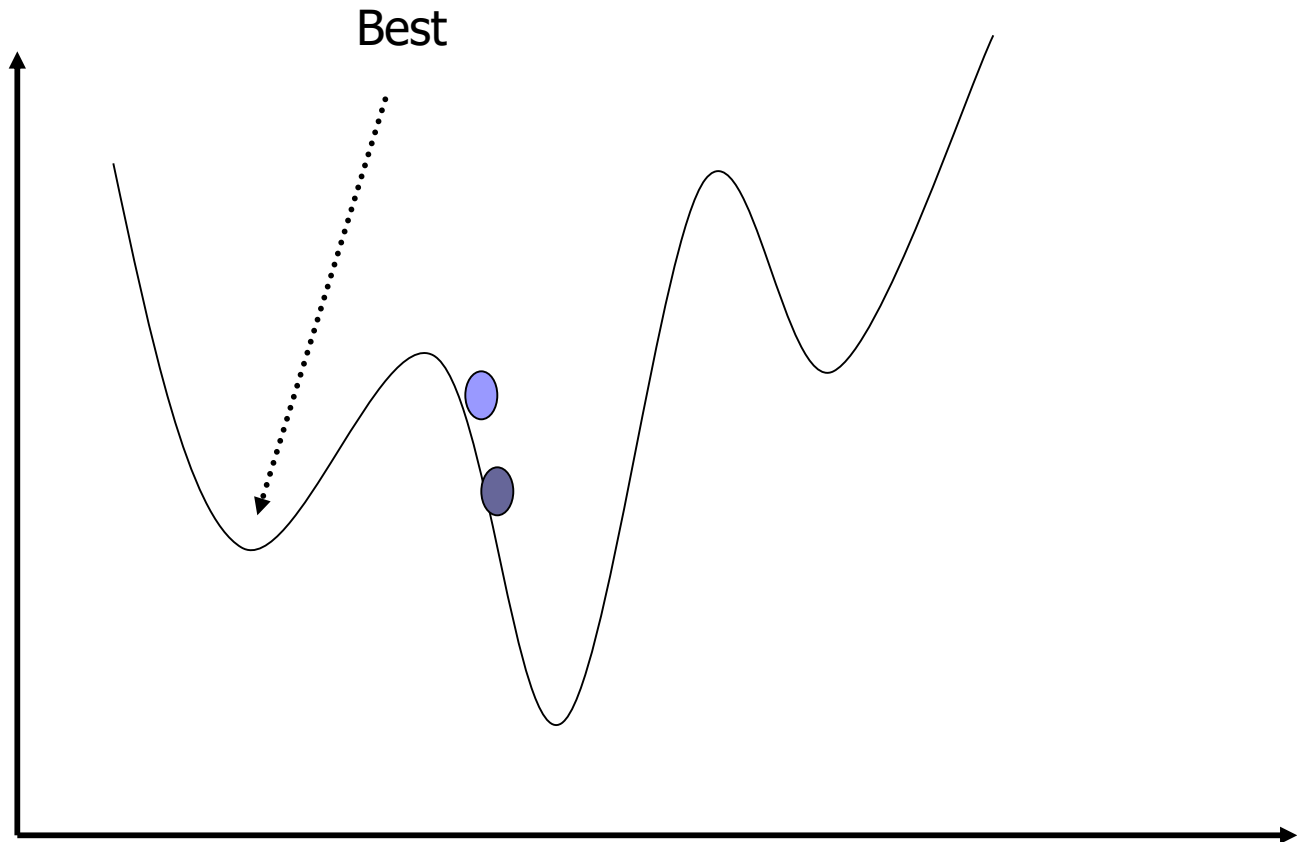
Tabu Search: TS



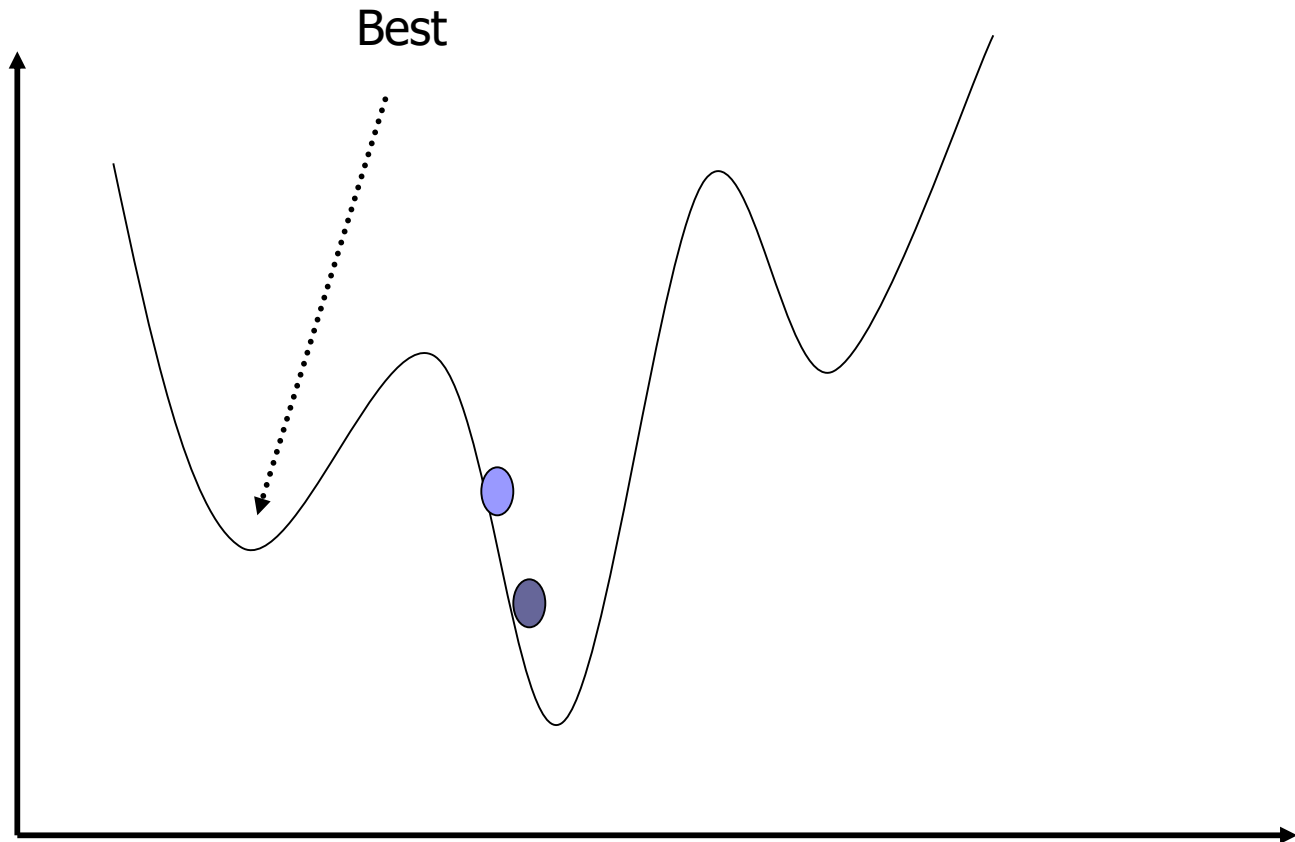
Tabu Search: TS



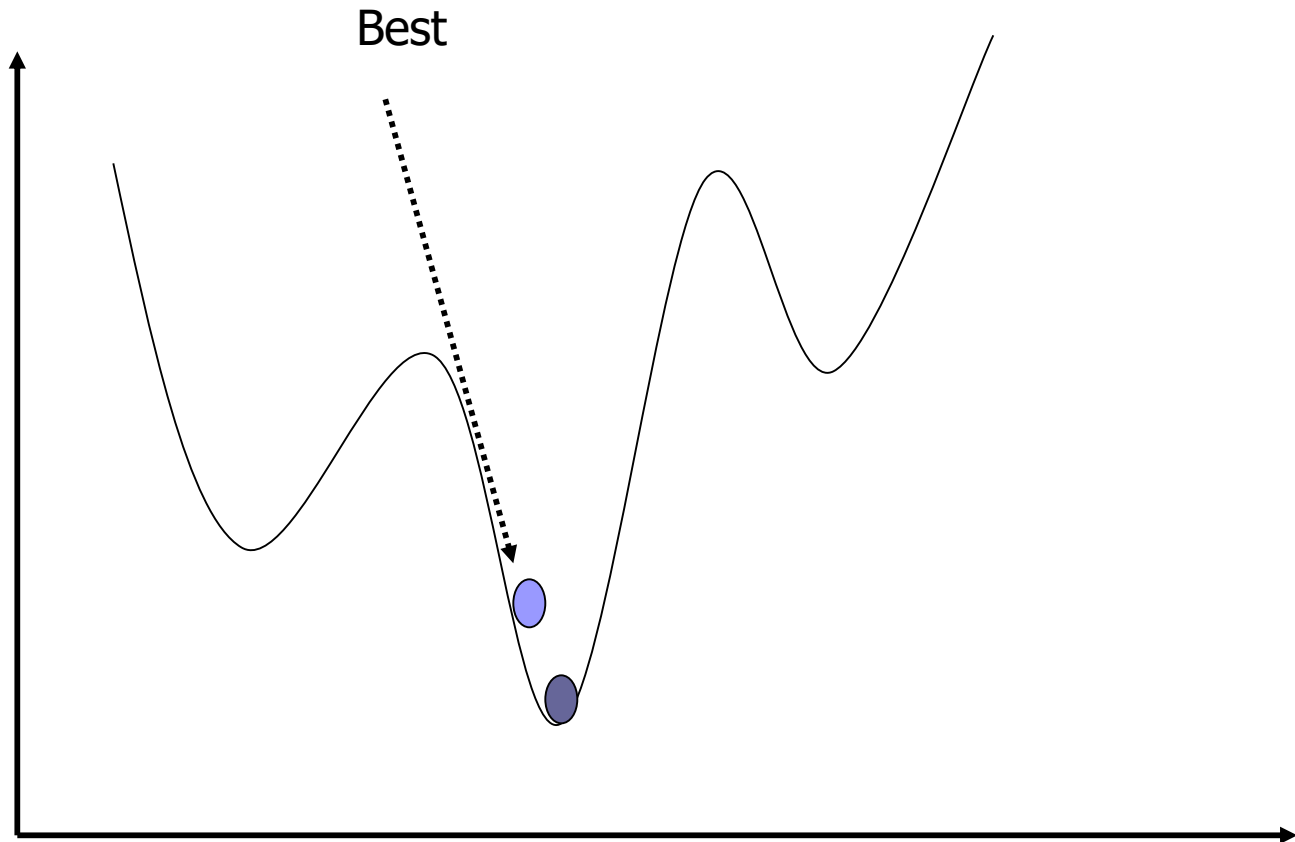
Tabu Search: TS



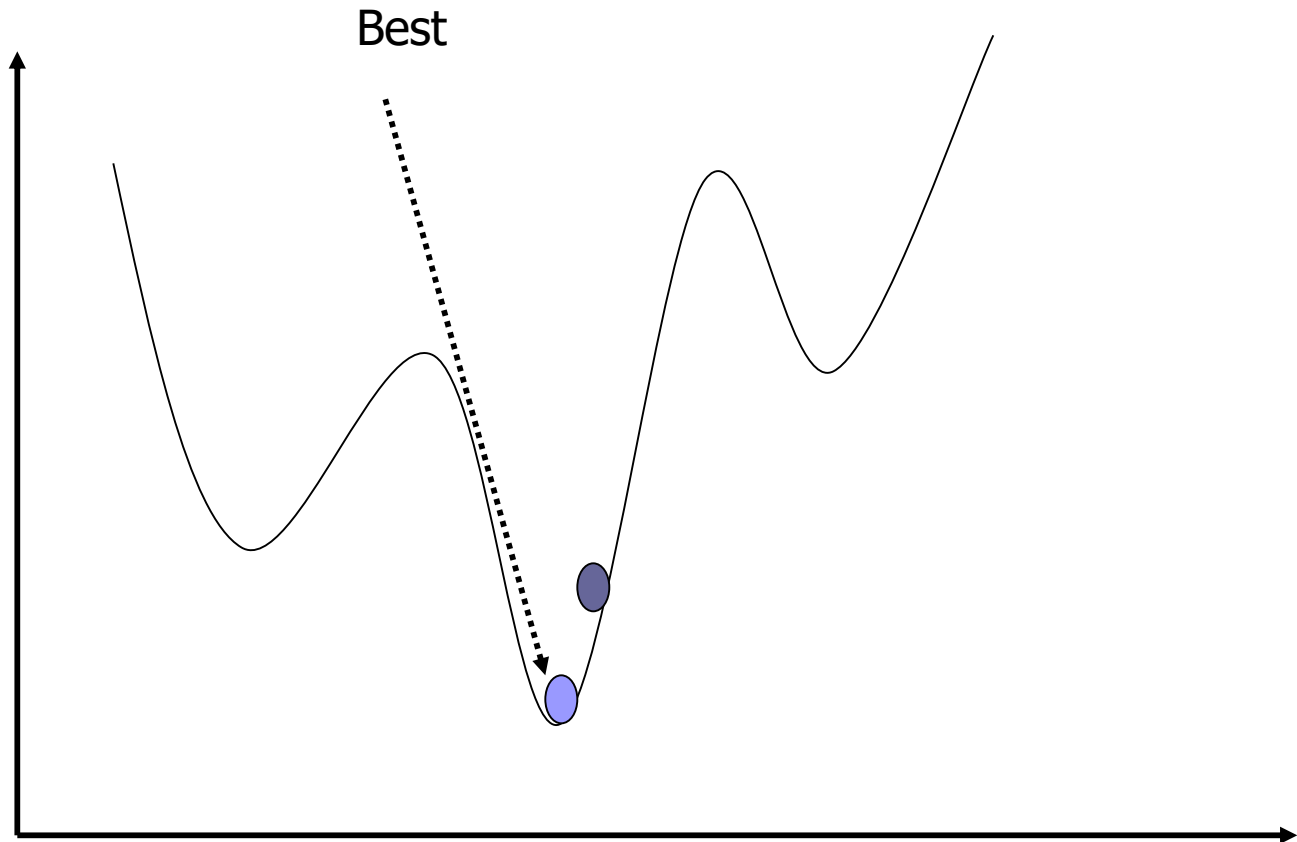
Tabu Search: TS



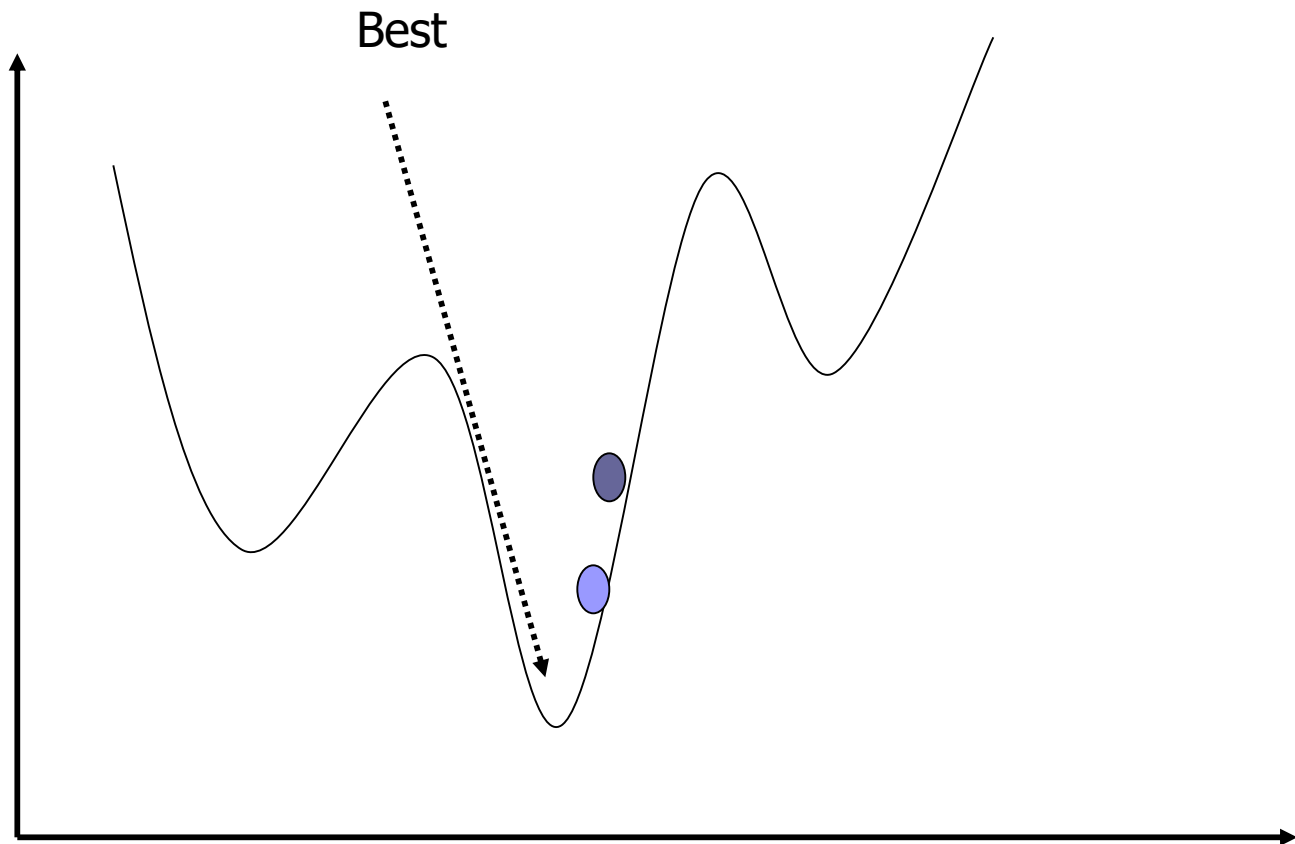
Tabu Search: TS



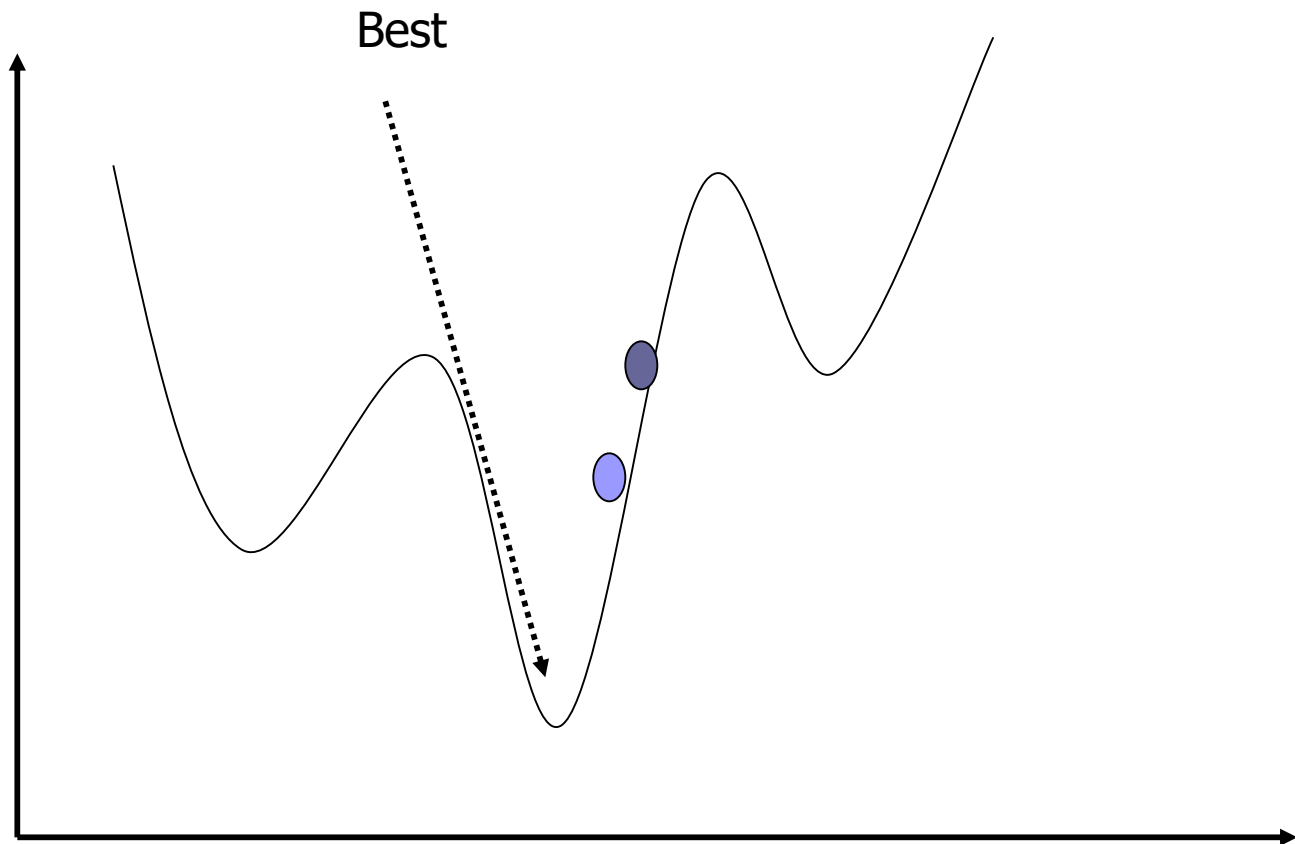
Tabu Search: TS



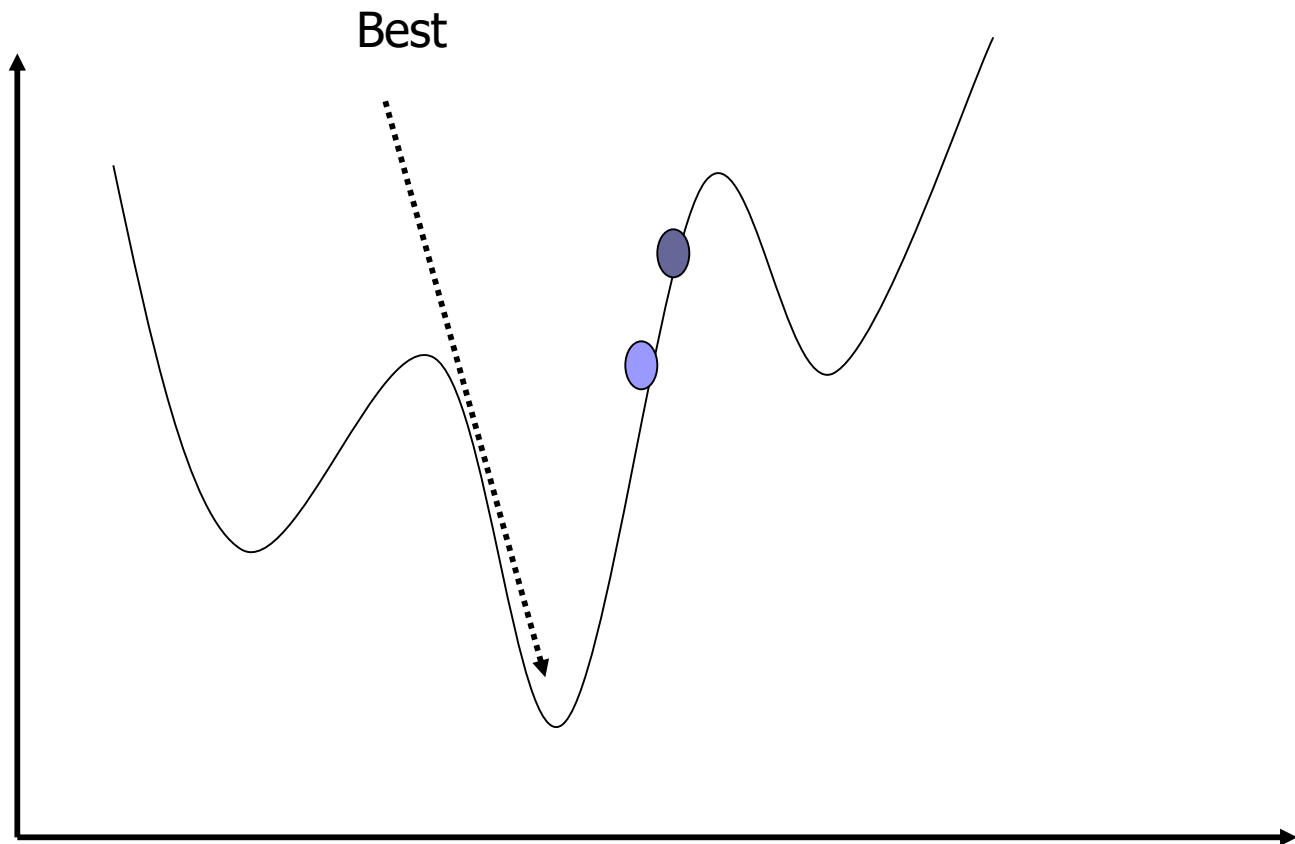
Tabu Search: TS



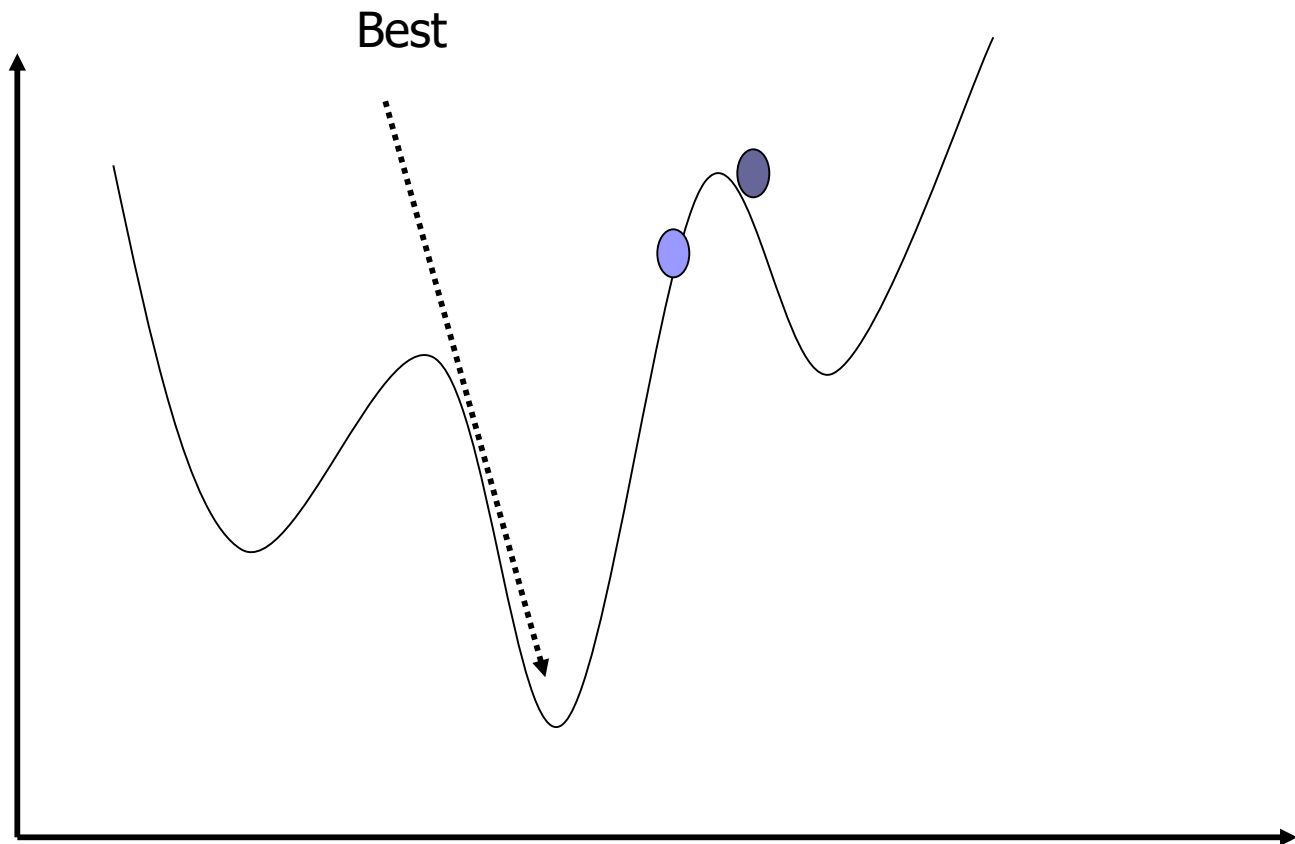
Tabu Search: TS



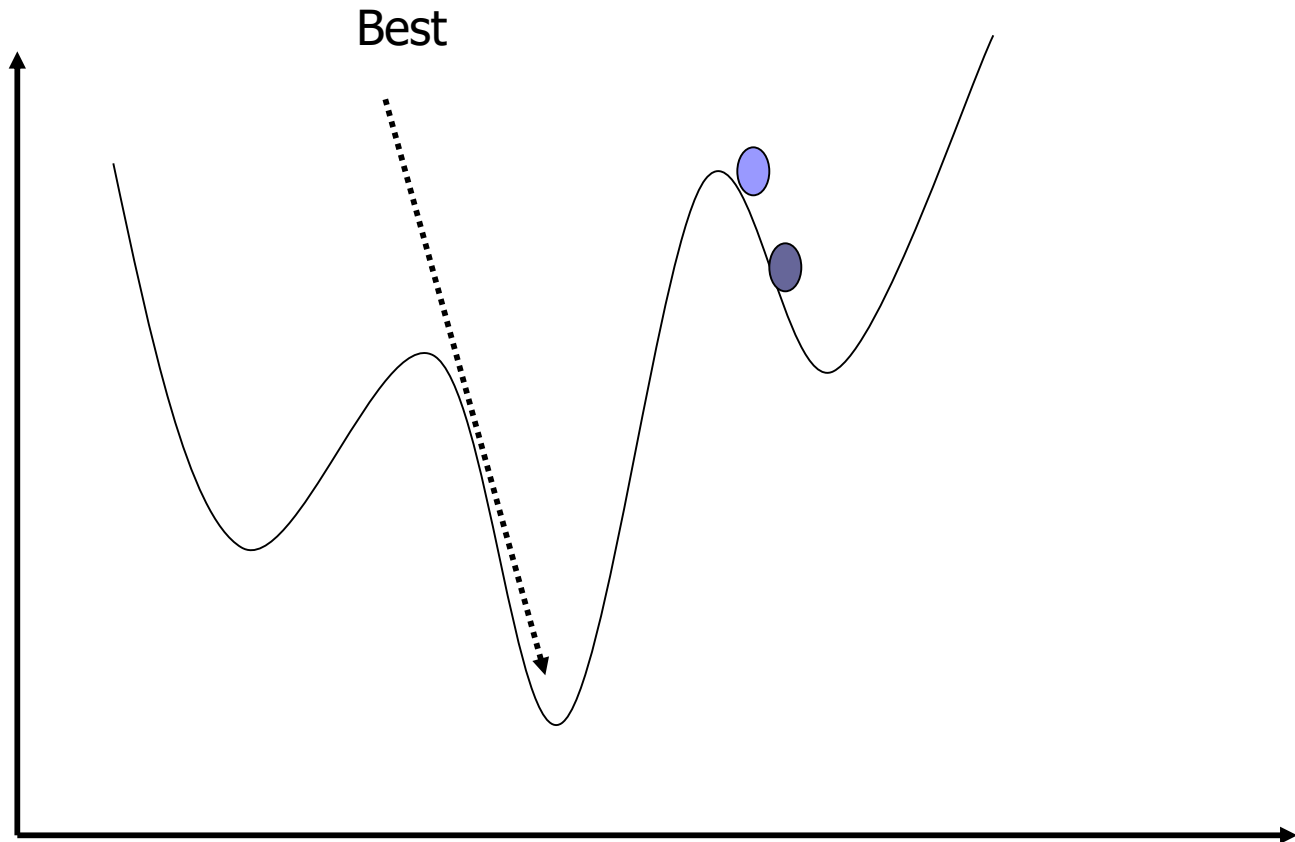
Tabu Search: TS



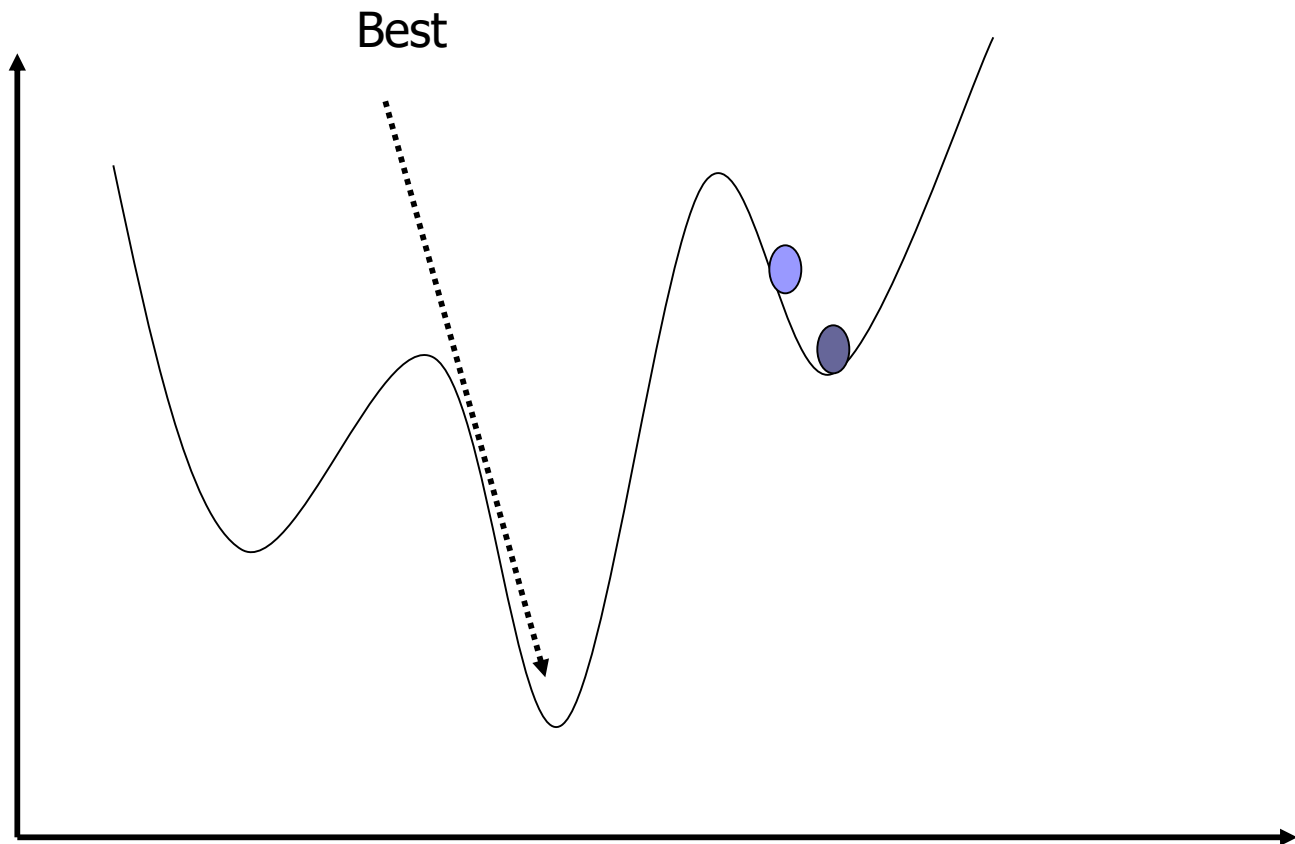
Tabu Search: TS



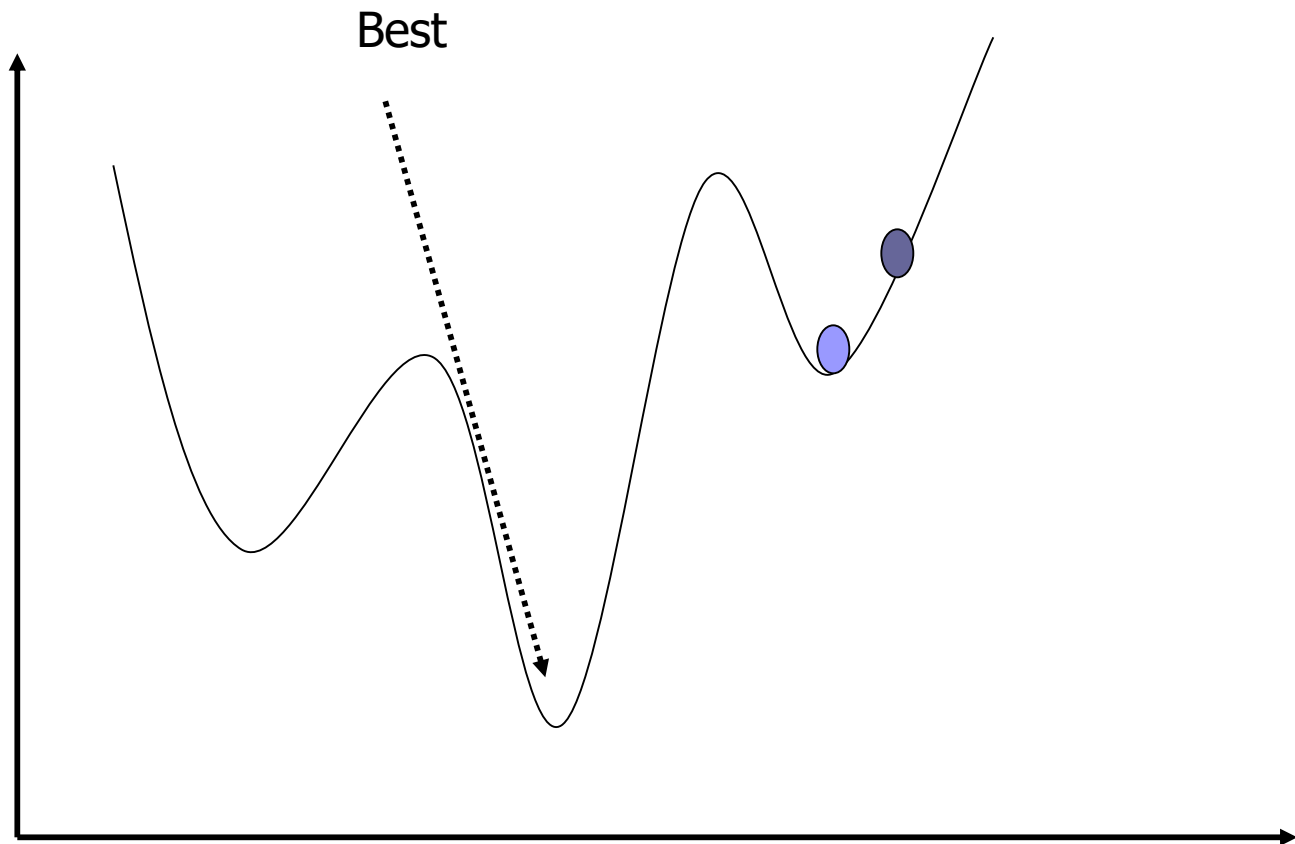
Tabu Search: TS



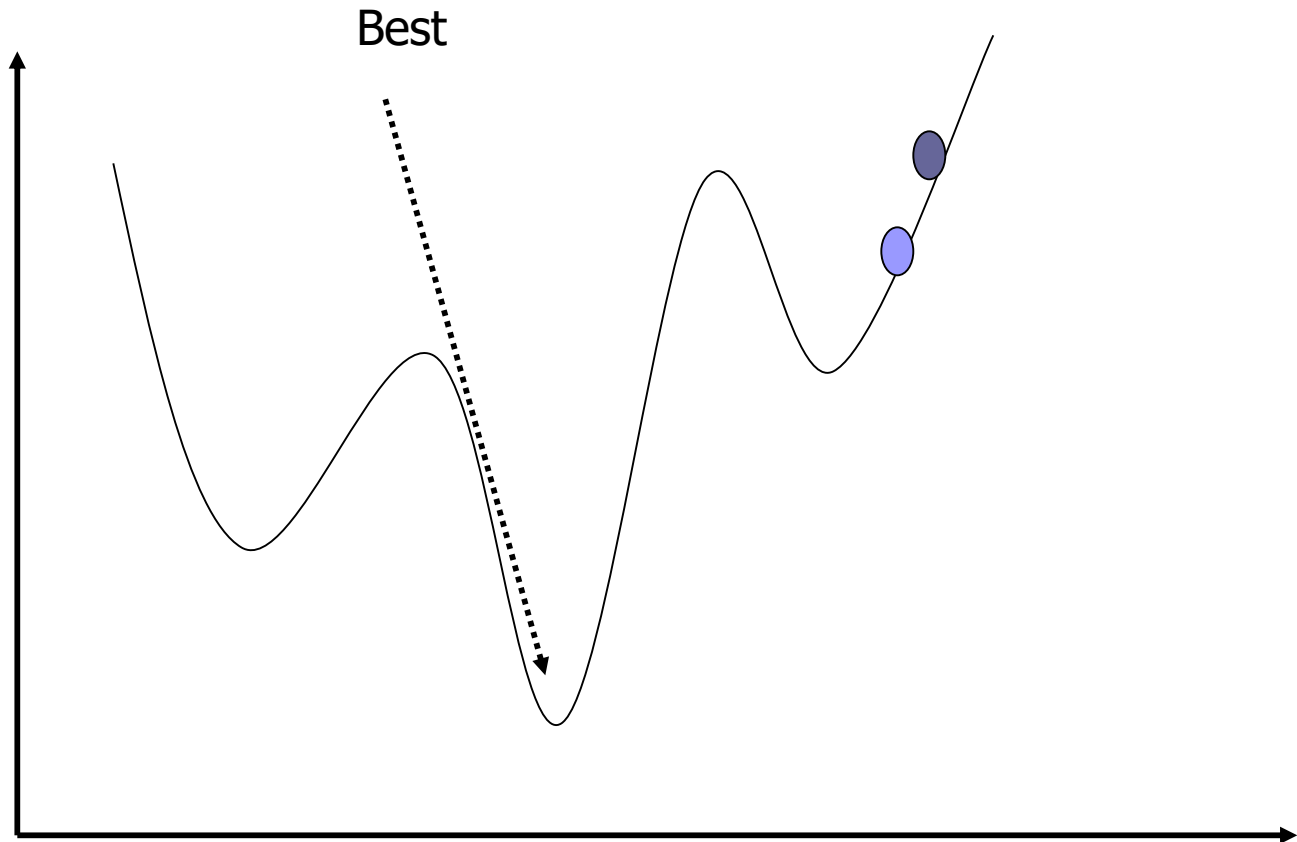
Tabu Search: TS



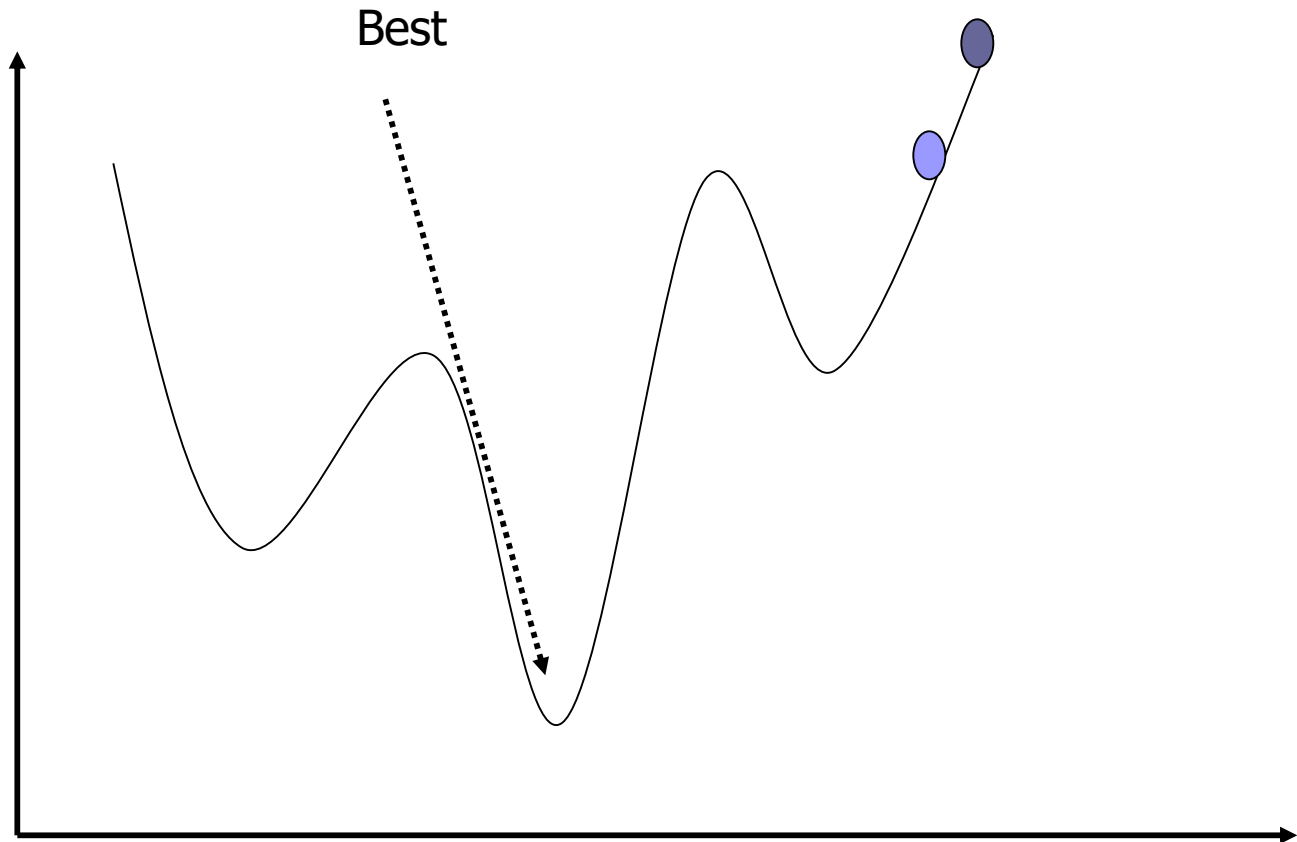
Tabu Search: TS



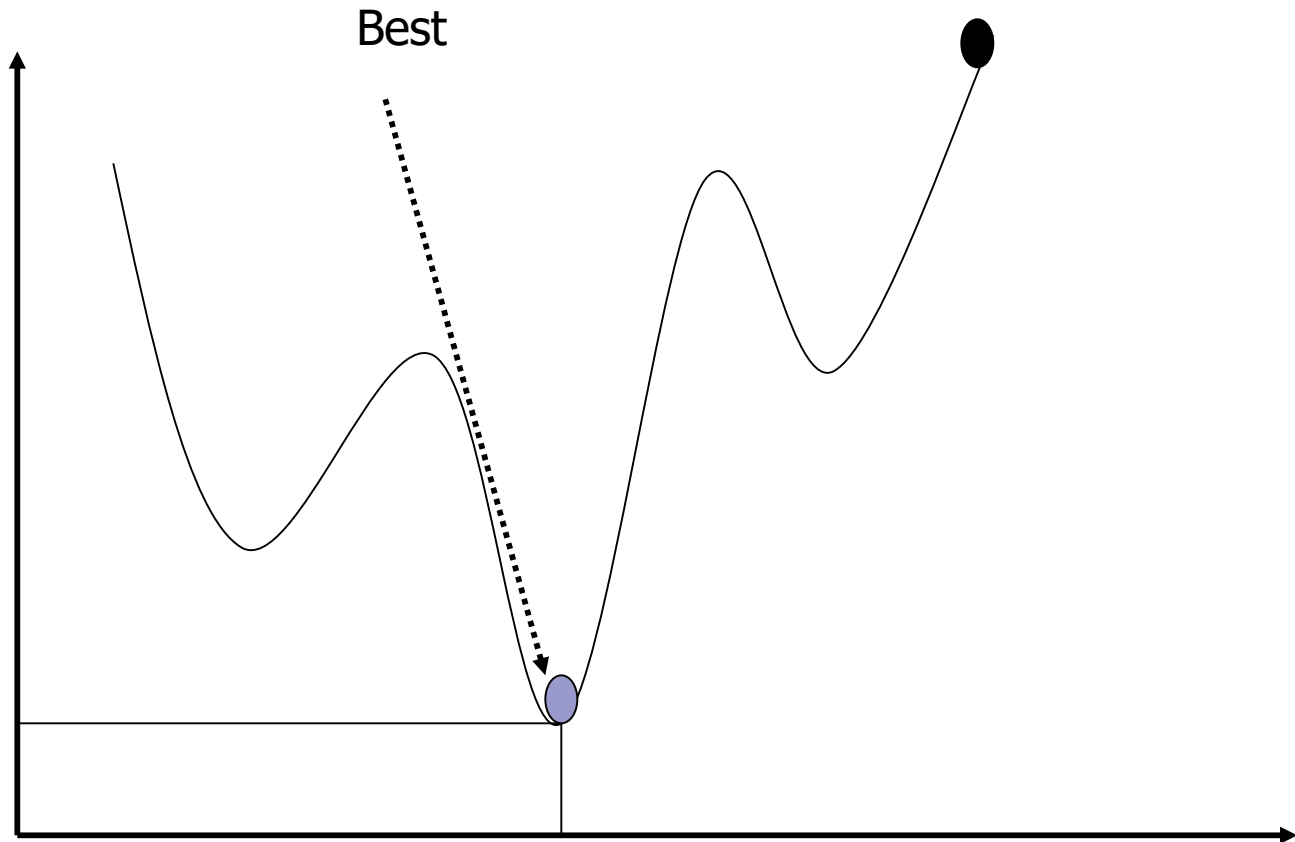
Tabu Search: TS

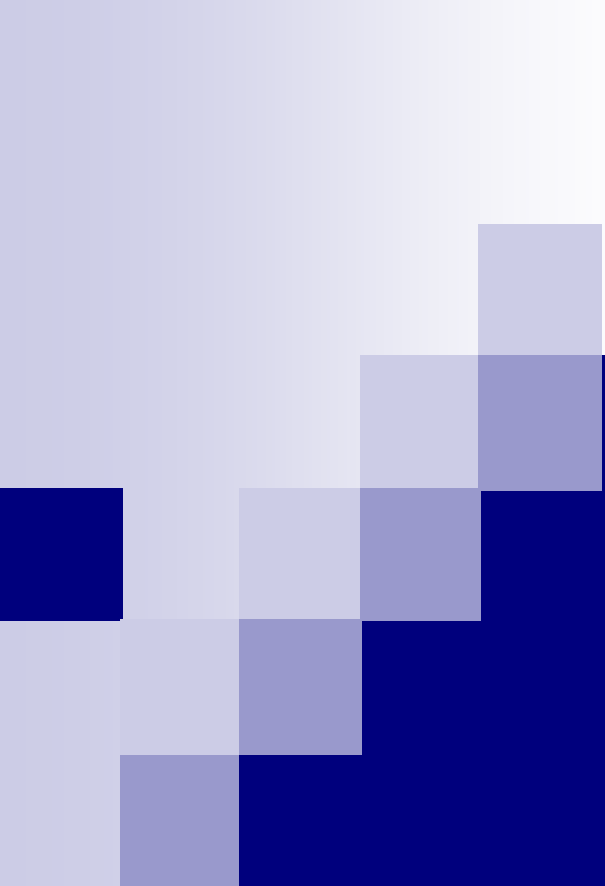


Tabu Search: TS



Tabu Search: TS





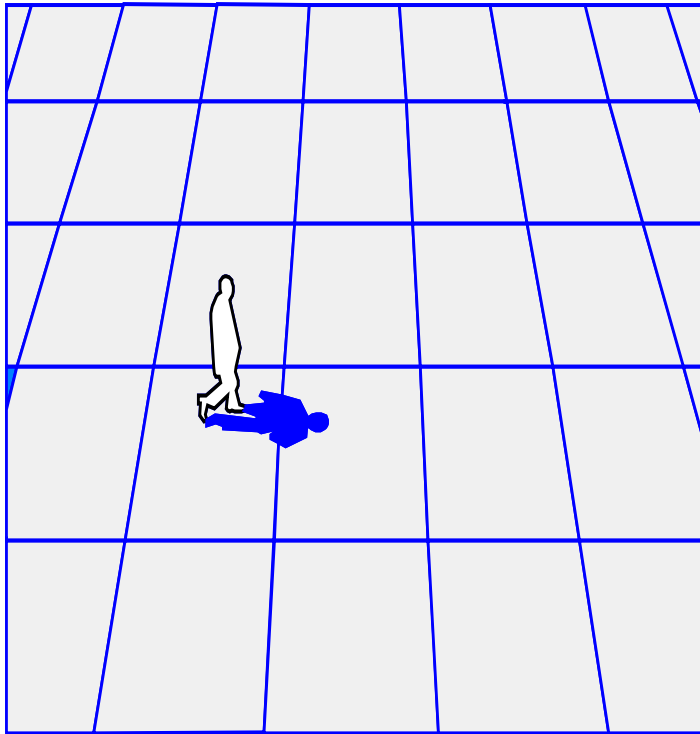
Population based Algorithms

Beam search

Genetic Algorithms

Genetic programming

Genetic Algorithms



“Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, solutions you might not otherwise find in a lifetime.”

- Salvatore Mangano
Computer Design, May 1995



Genetic Algorithm

Giải thuật di truyền

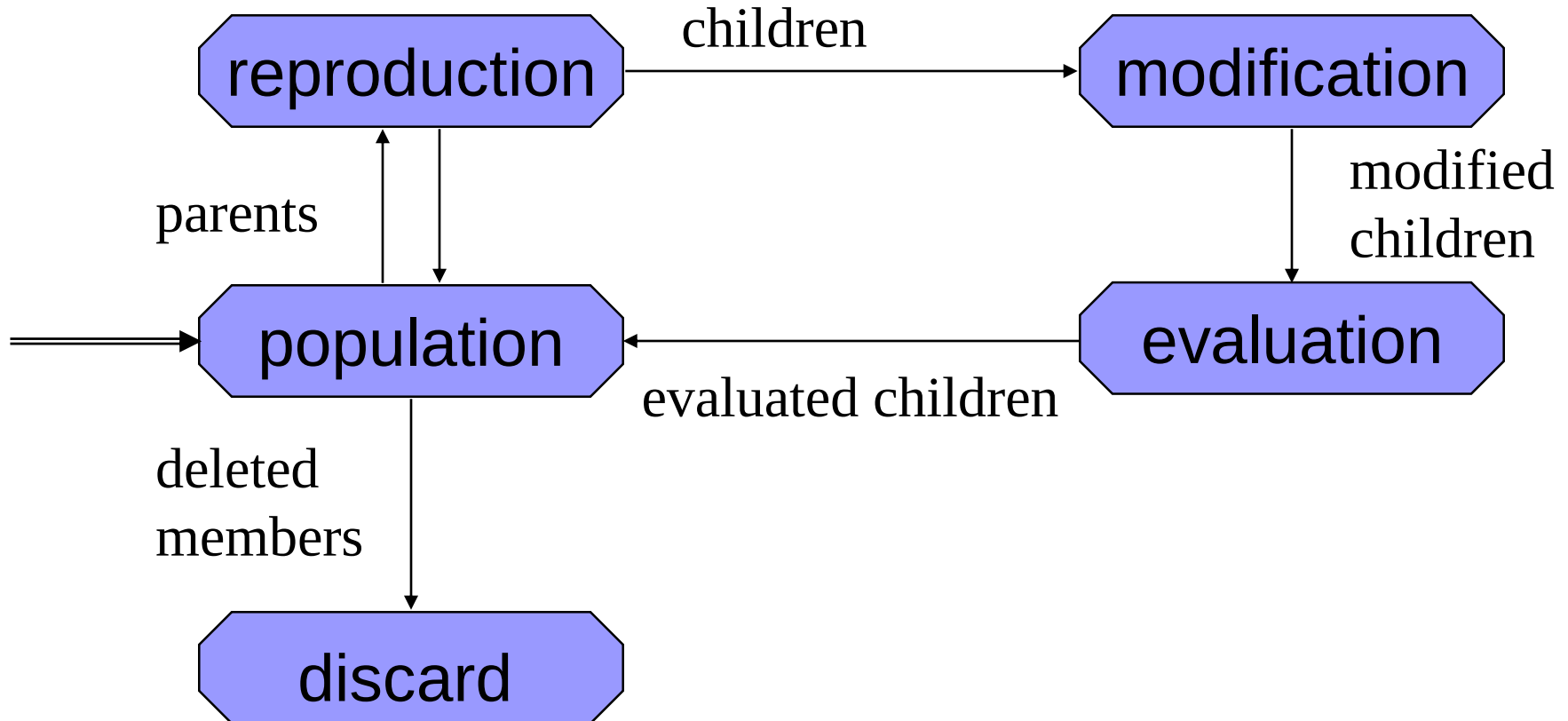
- Directed search algorithms based on the mechanics of biological evolution
- Developed by John Holland, Univ. of Michigan (1970's)
 - To understand the adaptive processes of natural systems
 - To design artificial systems software that retains the robustness of natural systems
- Provide efficient, effective techniques for optimization and machine learning applications
- Widely-used today in business, scientific and engineering circles

Components of a GA

A problem to solve, and ...

- Encoding technique *(gene, chromosome)*
- Initialization procedure *(creation)*
- Evaluation function *(environment)*
- Selection of parents *(reproduction)*
- Genetic operators *(mutation, recombination)*
- Parameter settings *(practice and art)*

The GA Cycle of Reproduction



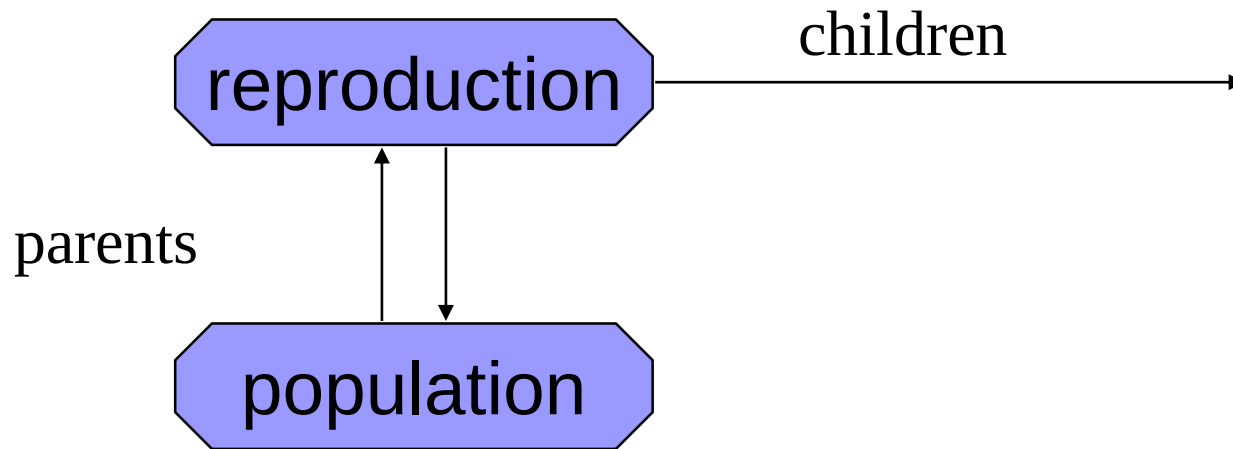
Population



Chromosomes could be:

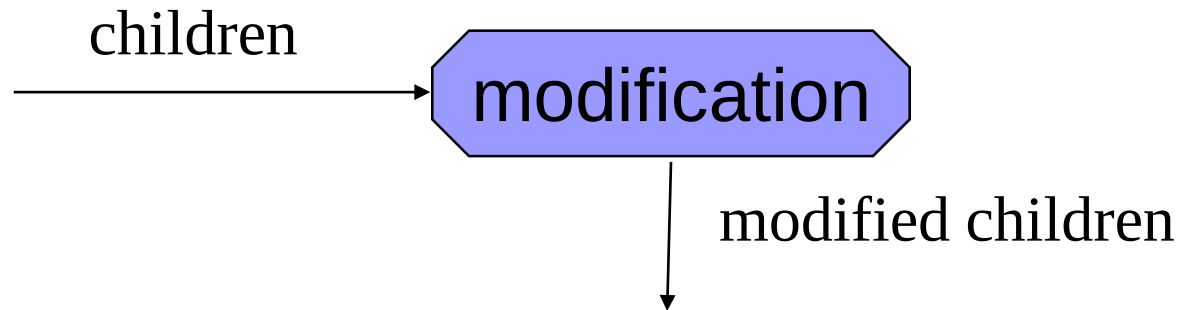
- ☐ Bit strings (0101 ... 1100)
- ☐ Real numbers (43.2 -33.1 ... 0.0 89.2)
- ☐ Permutations of element (E11 E3 E7 ... E1 E15)
- ☐ Lists of rules (R1 R2 R3 ... R22 R23)
- ☐ Program elements (genetic programming)
- ☐ ... any data structure ...

Reproduction



Parents are selected at random with selection chances biased in relation to chromosome evaluations.

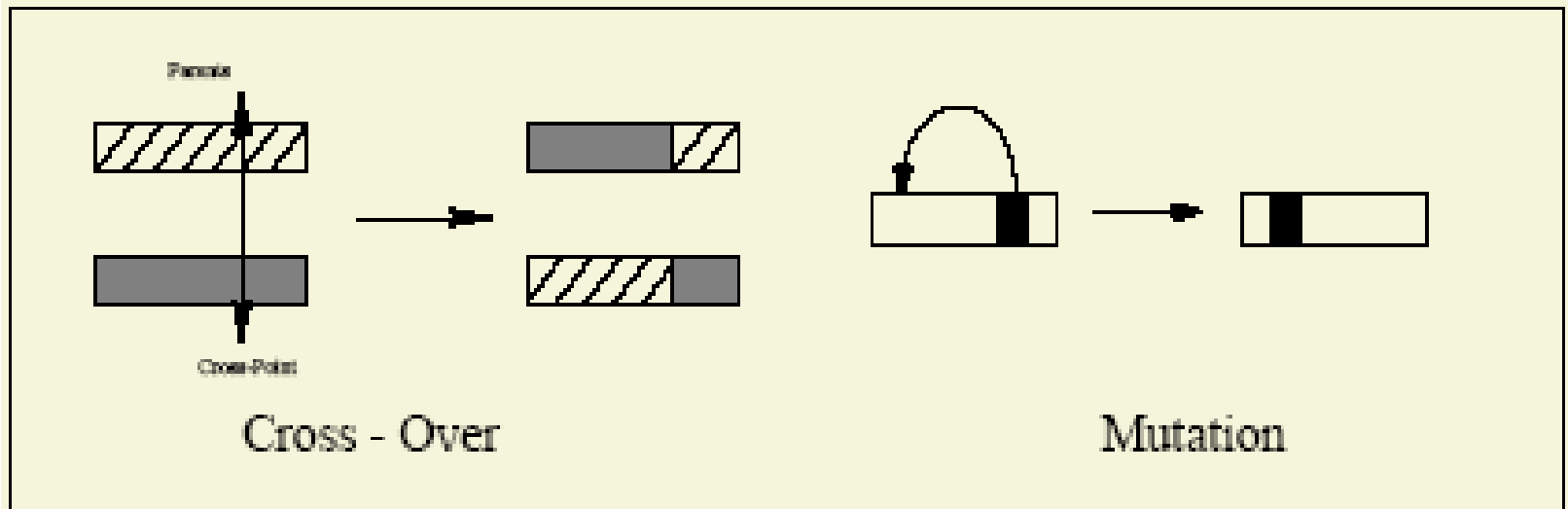
Chromosome Modification



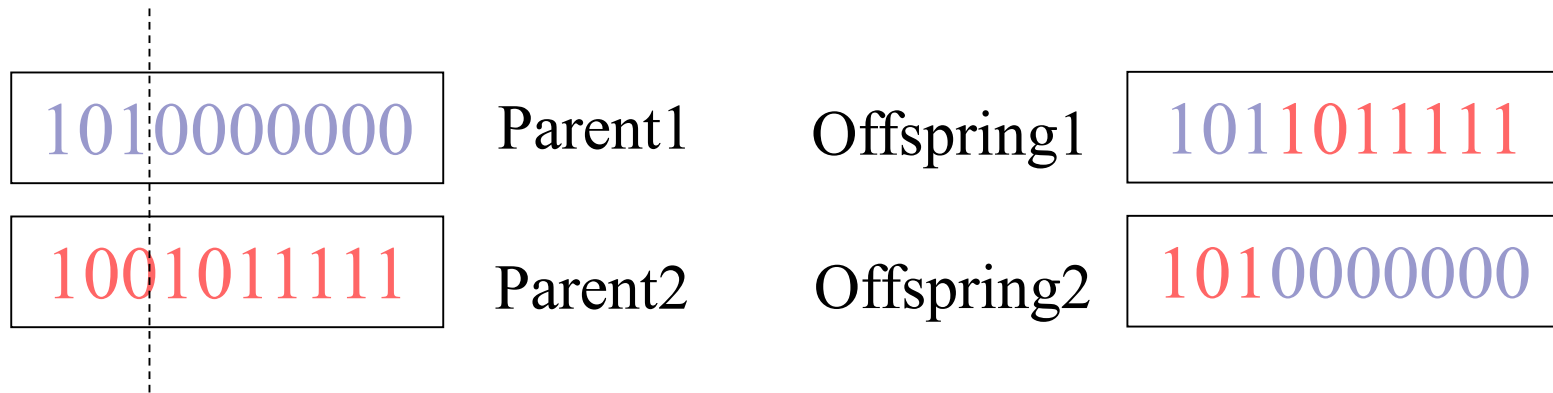
- Modifications are stochastically triggered
- Operator types are:
 - Mutation
 - Crossover (recombination)

Genetic Algorithms: Crossover and Mutation

How do individuals reproduce ?



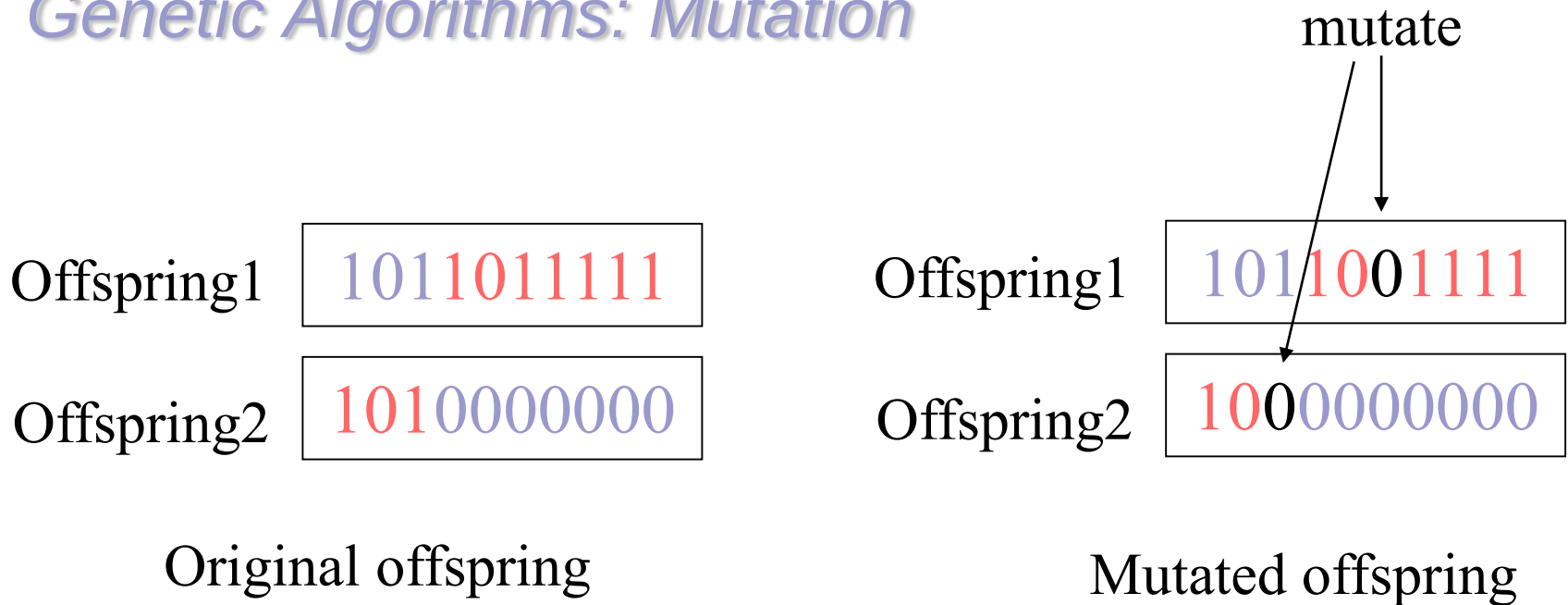
Genetic Algorithms: Crossover - Recombination



Crossover
single point -
random

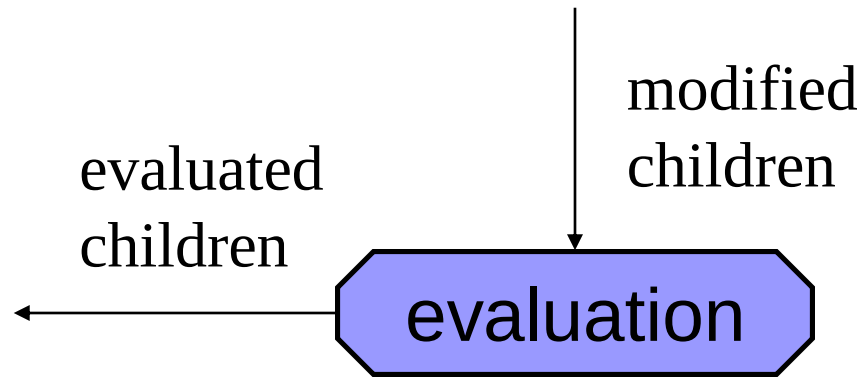
With some high probability (*crossover rate*) apply crossover to the parents.
(*typical values are 0.8 to 0.95*)

Genetic Algorithms: Mutation



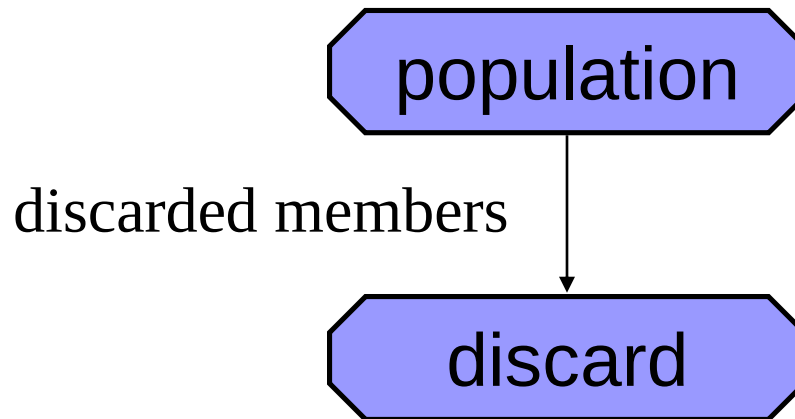
With some small probability (the *mutation rate*) flip each bit in the offspring (*typical values between 0.1 and 0.001*)

Evaluation



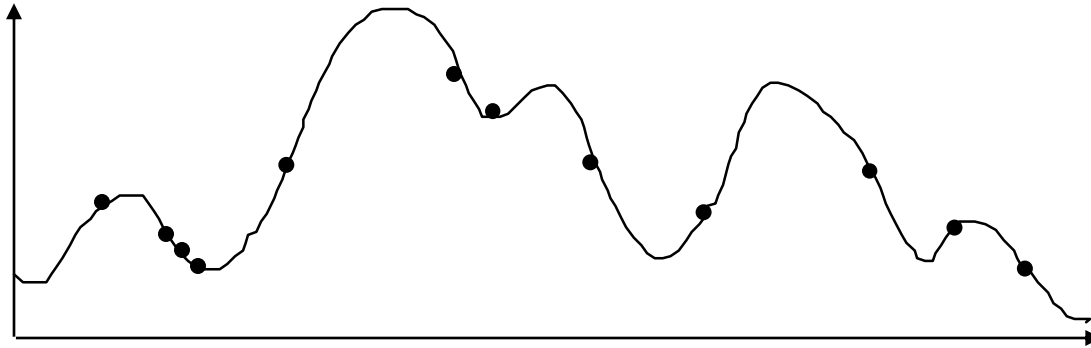
- The evaluator decodes a chromosome and assigns it a fitness measure
- The evaluator is the only link between a classical GA and the problem it is solving

Deletion

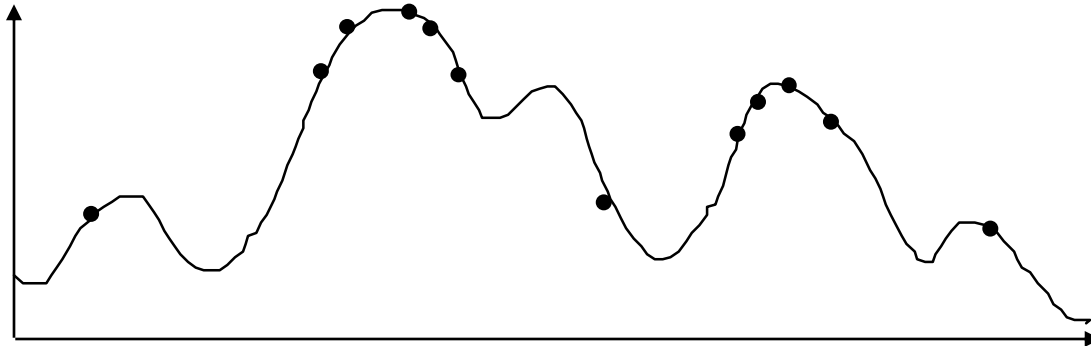


- *Generational GA*:
entire populations replaced with each iteration
- *Steady-state GA*:
a few members replaced each generation

An Abstract Example

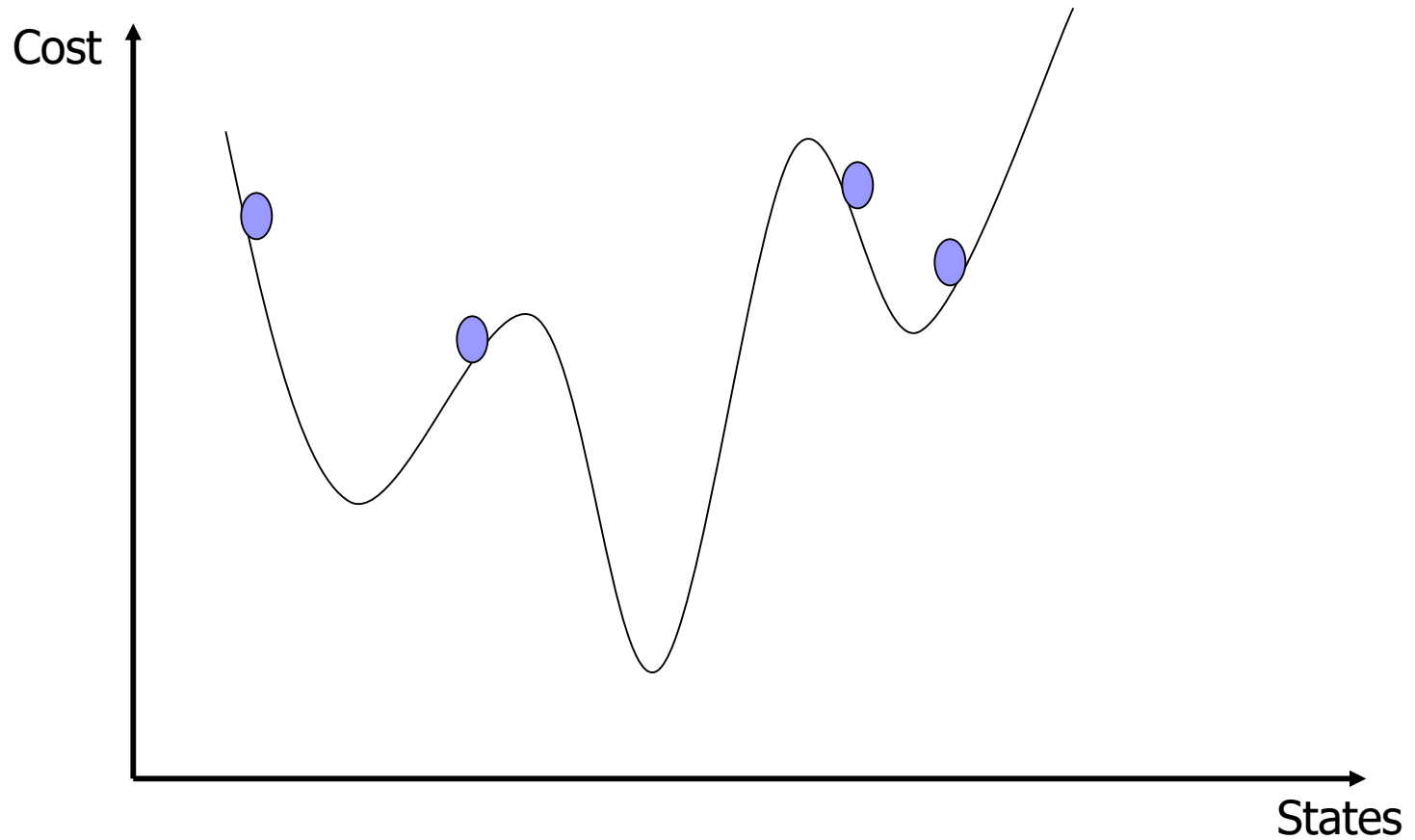


Distribution of Individuals in Generation 0

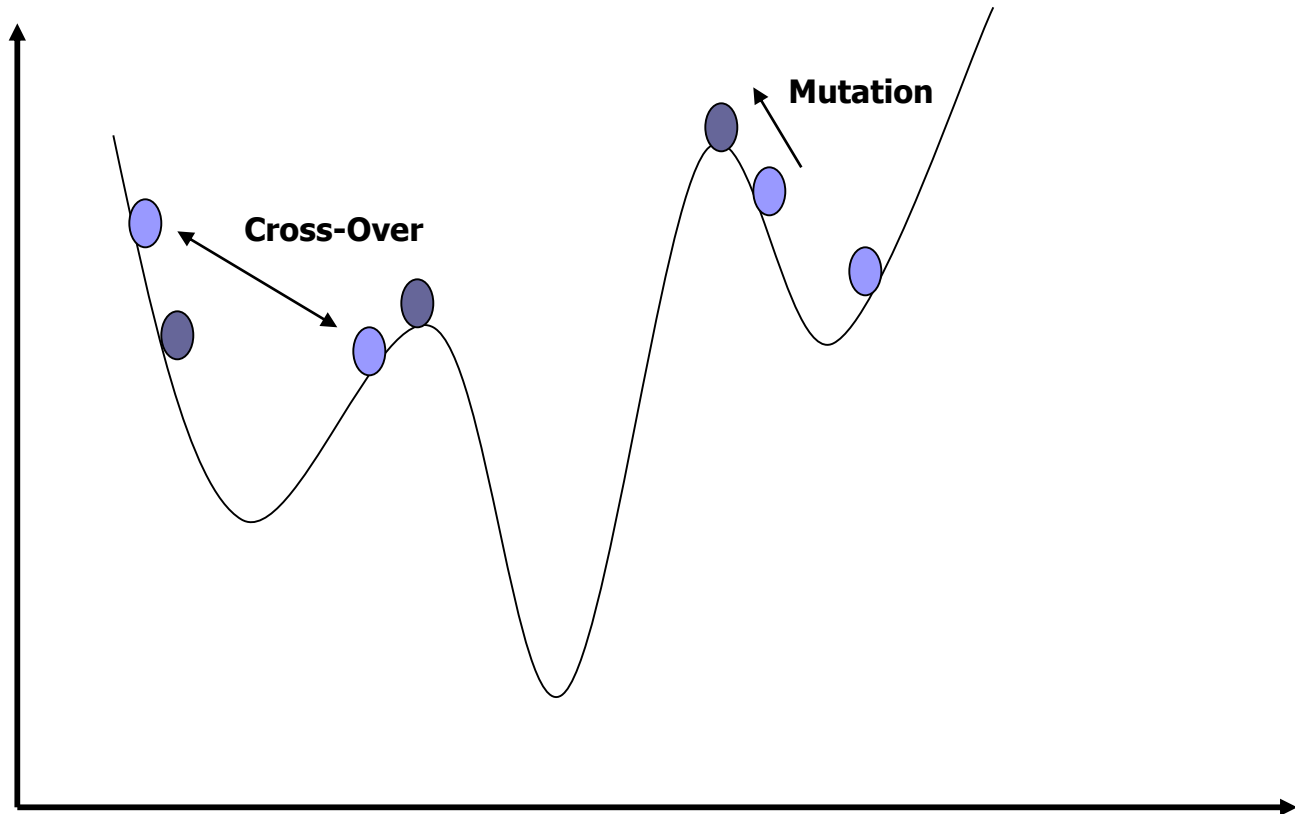


Distribution of Individuals in Generation N

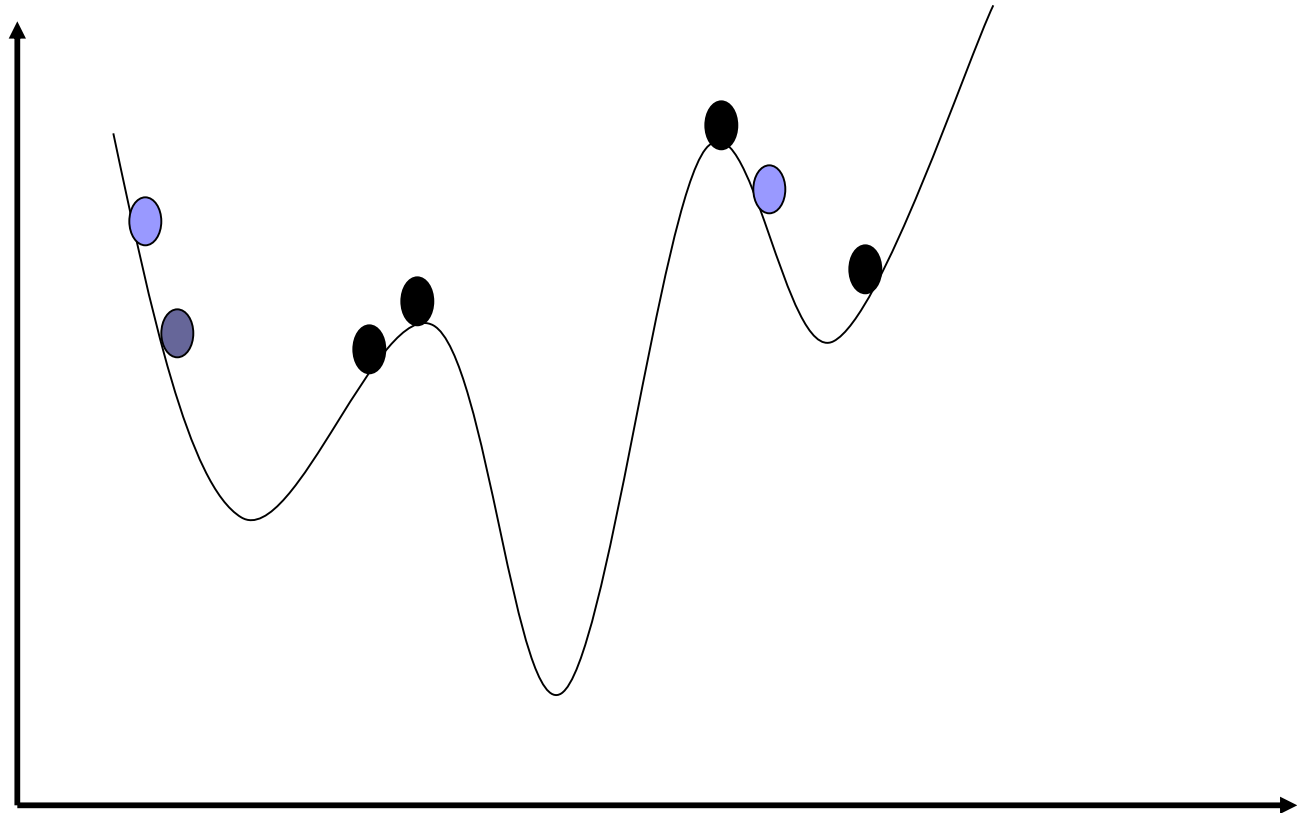
Genetic Algorithms



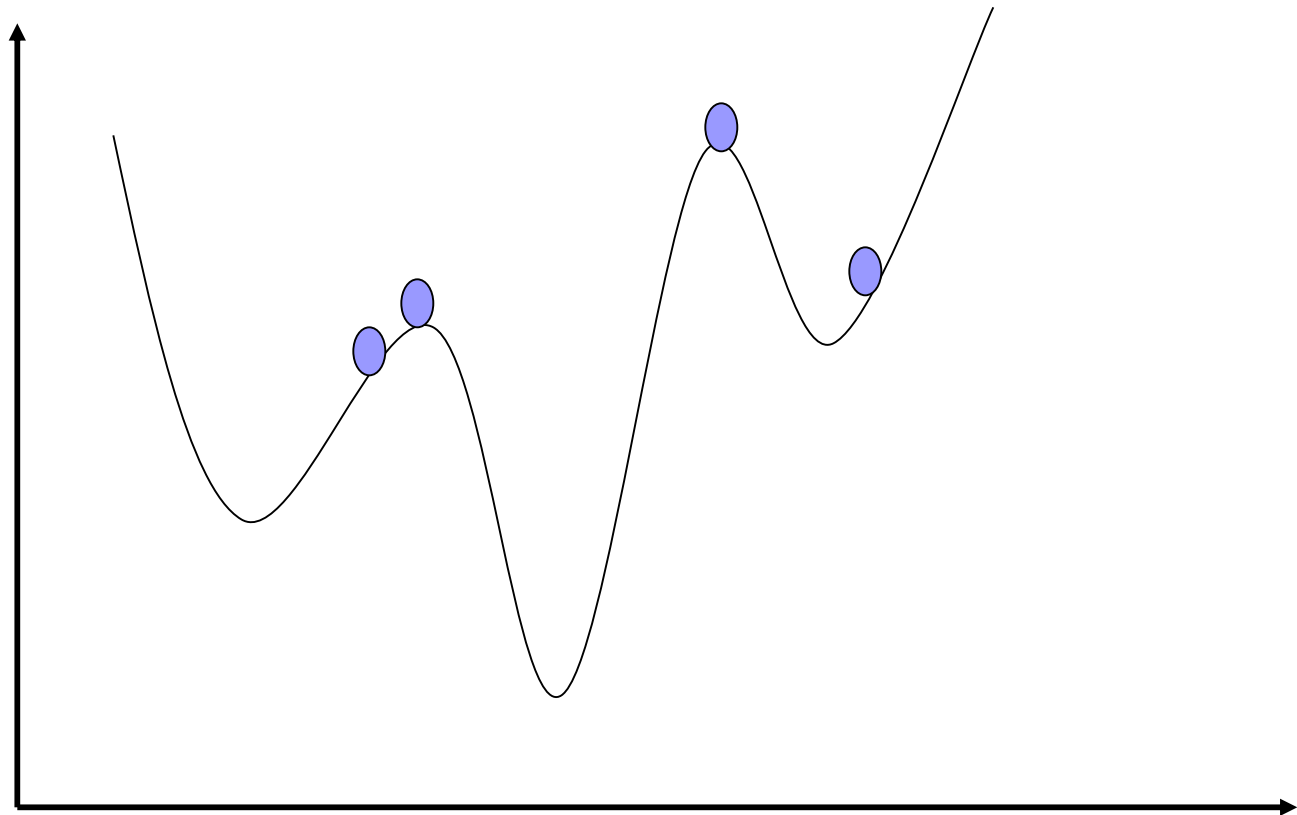
Genetic Algorithms



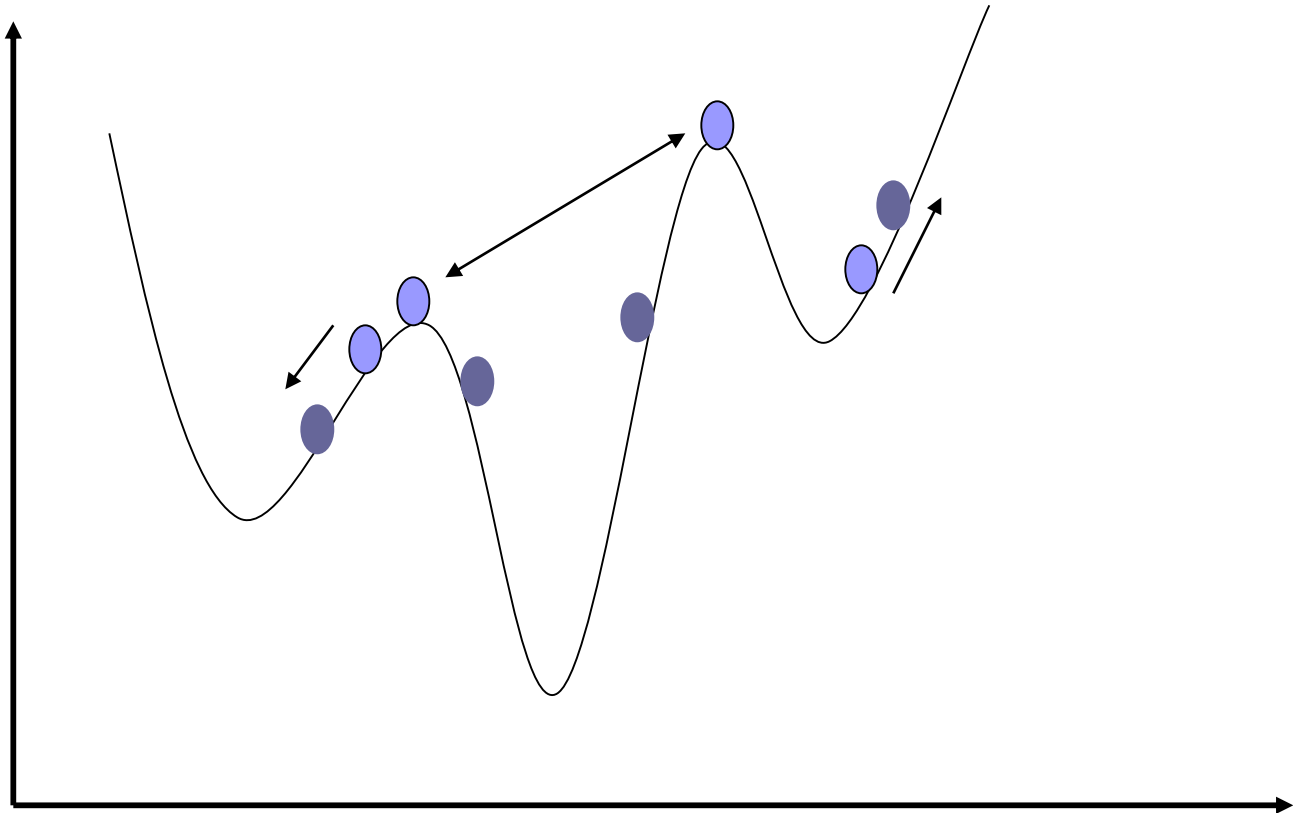
Genetic Algorithms



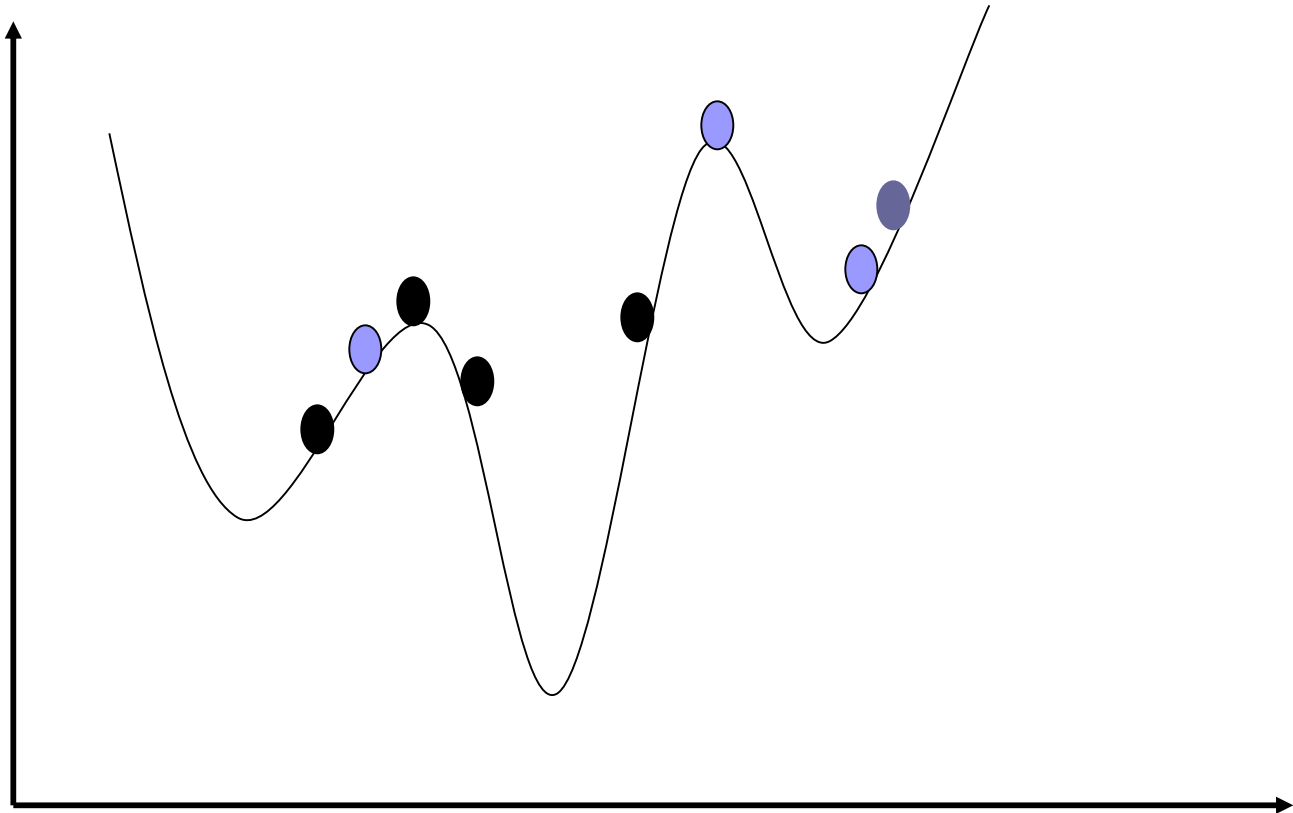
Genetic Algorithms



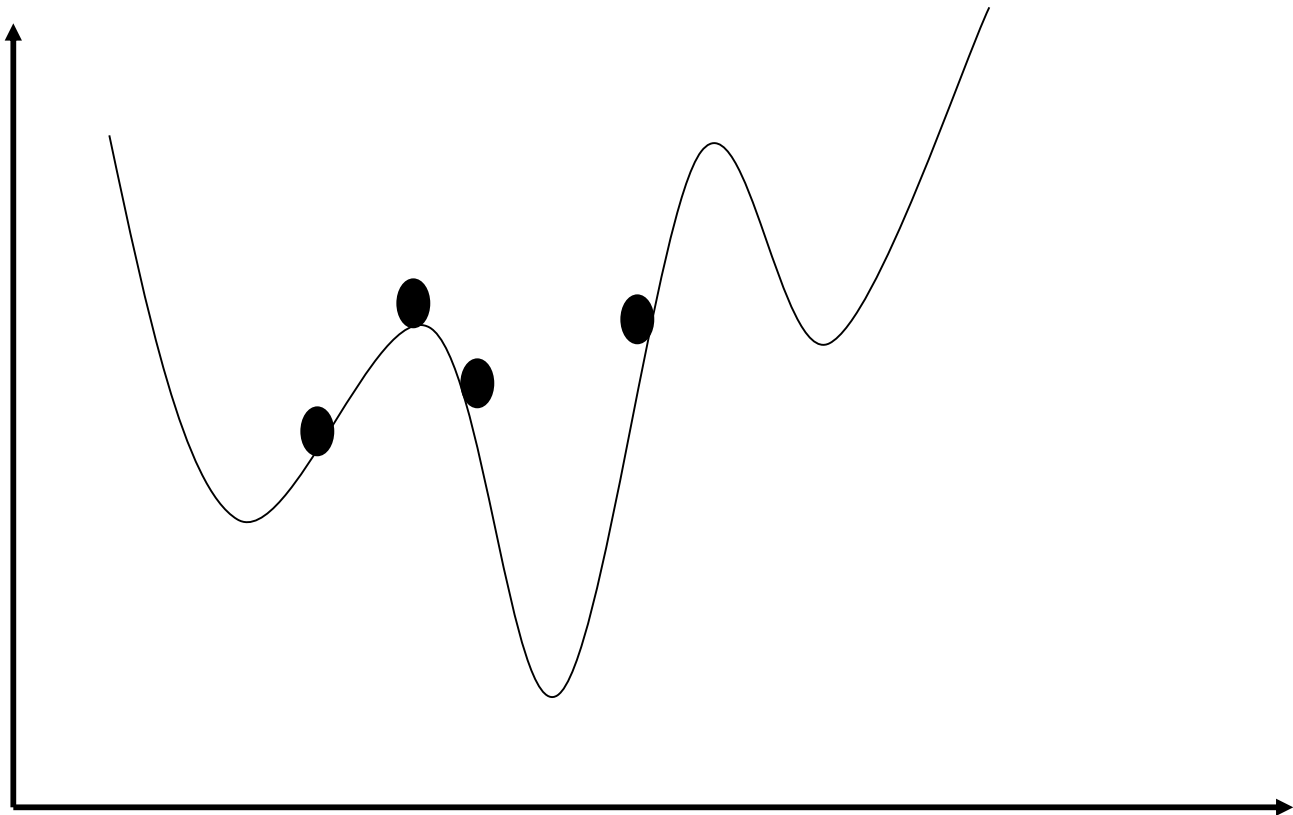
Genetic Algorithms



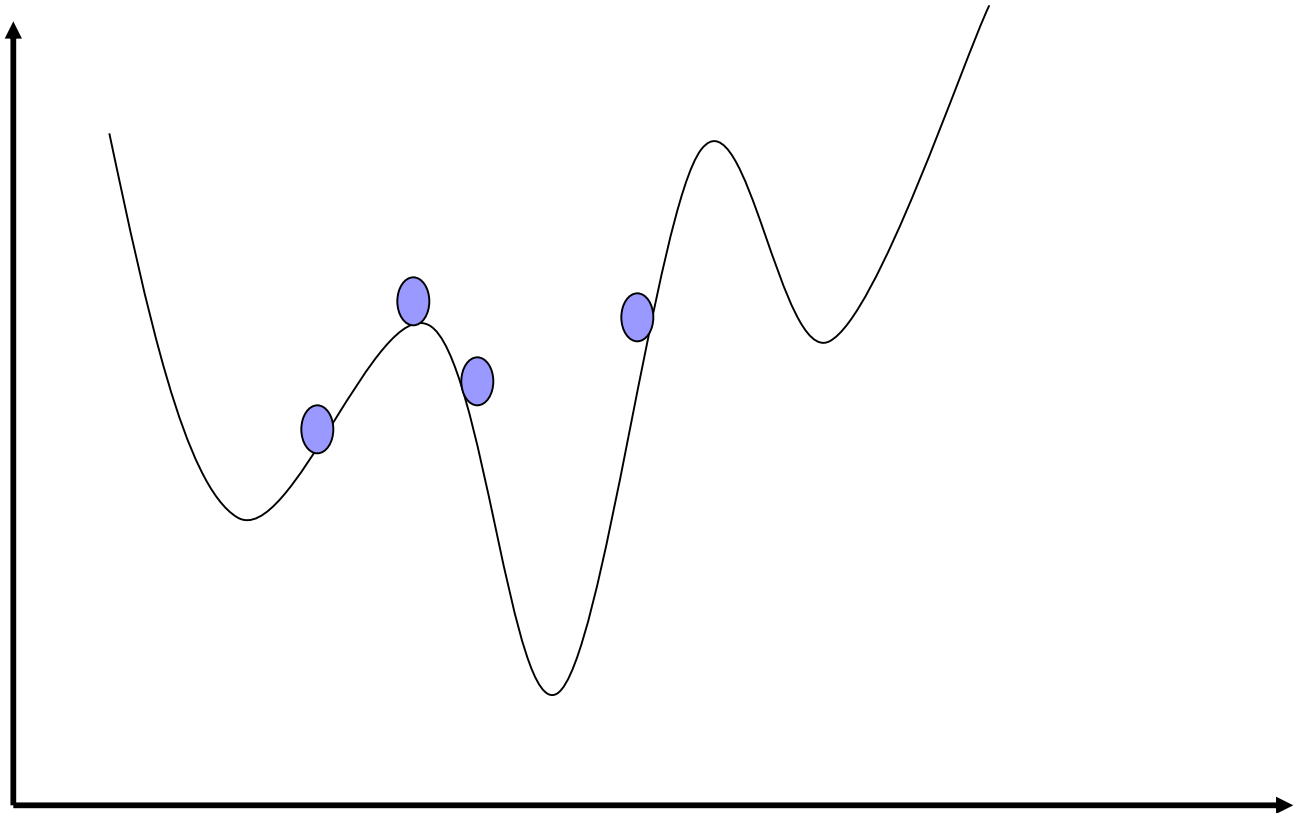
Genetic Algorithms



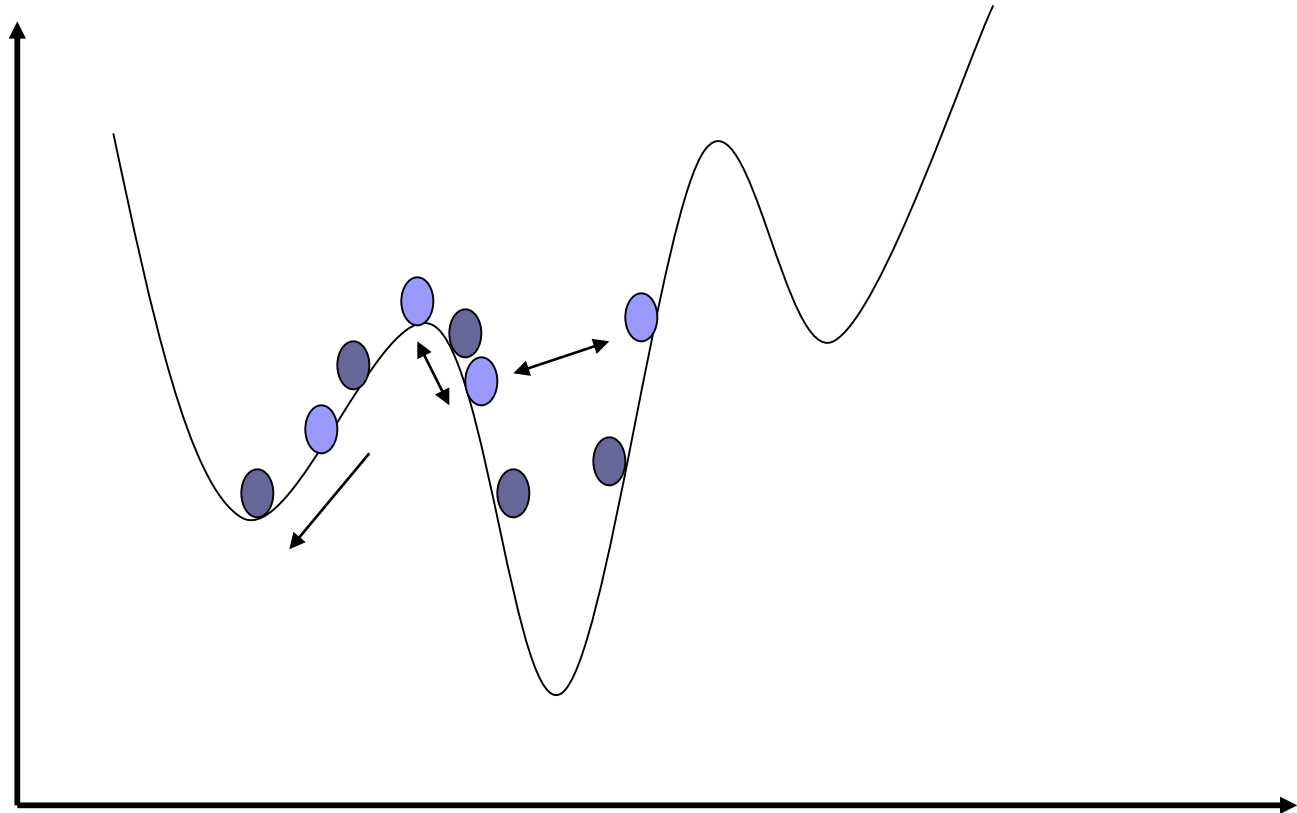
Genetic Algorithms



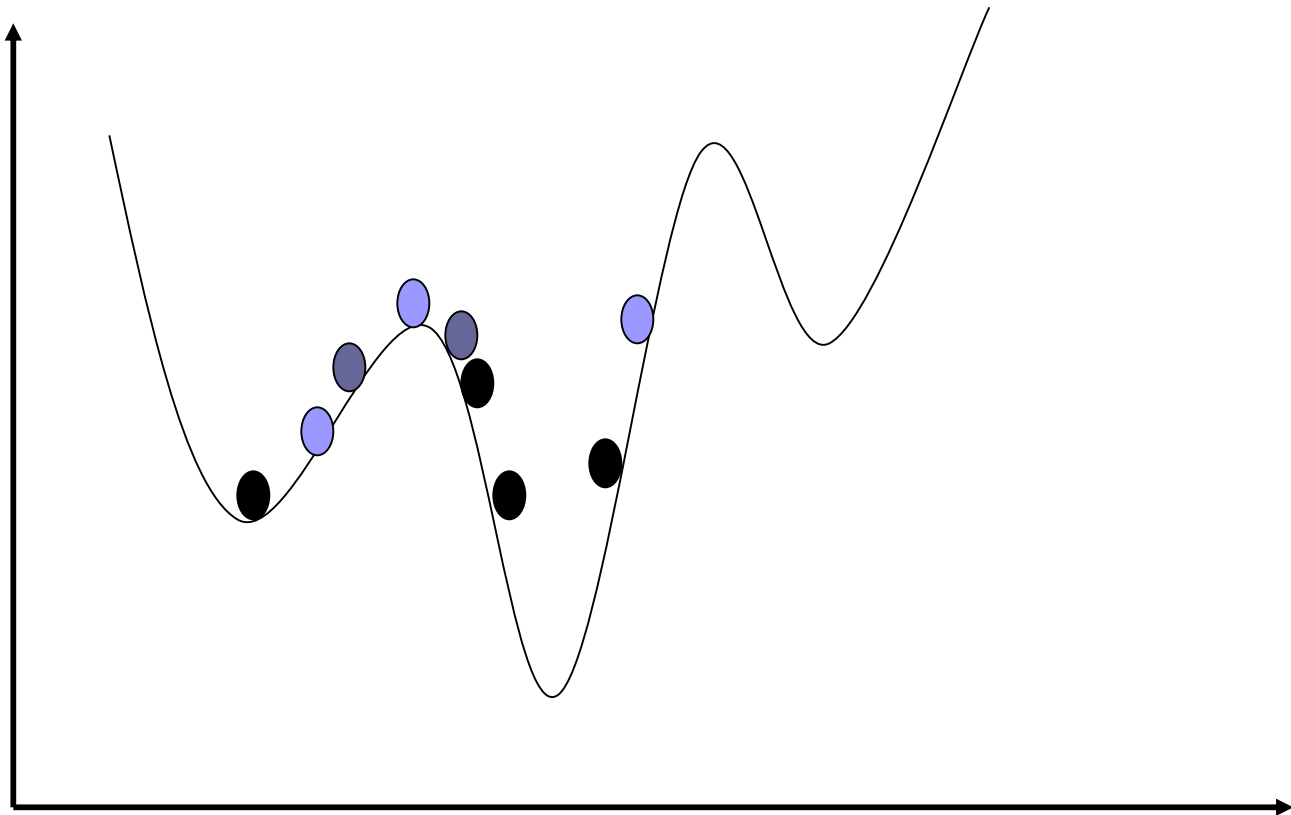
Genetic Algorithms



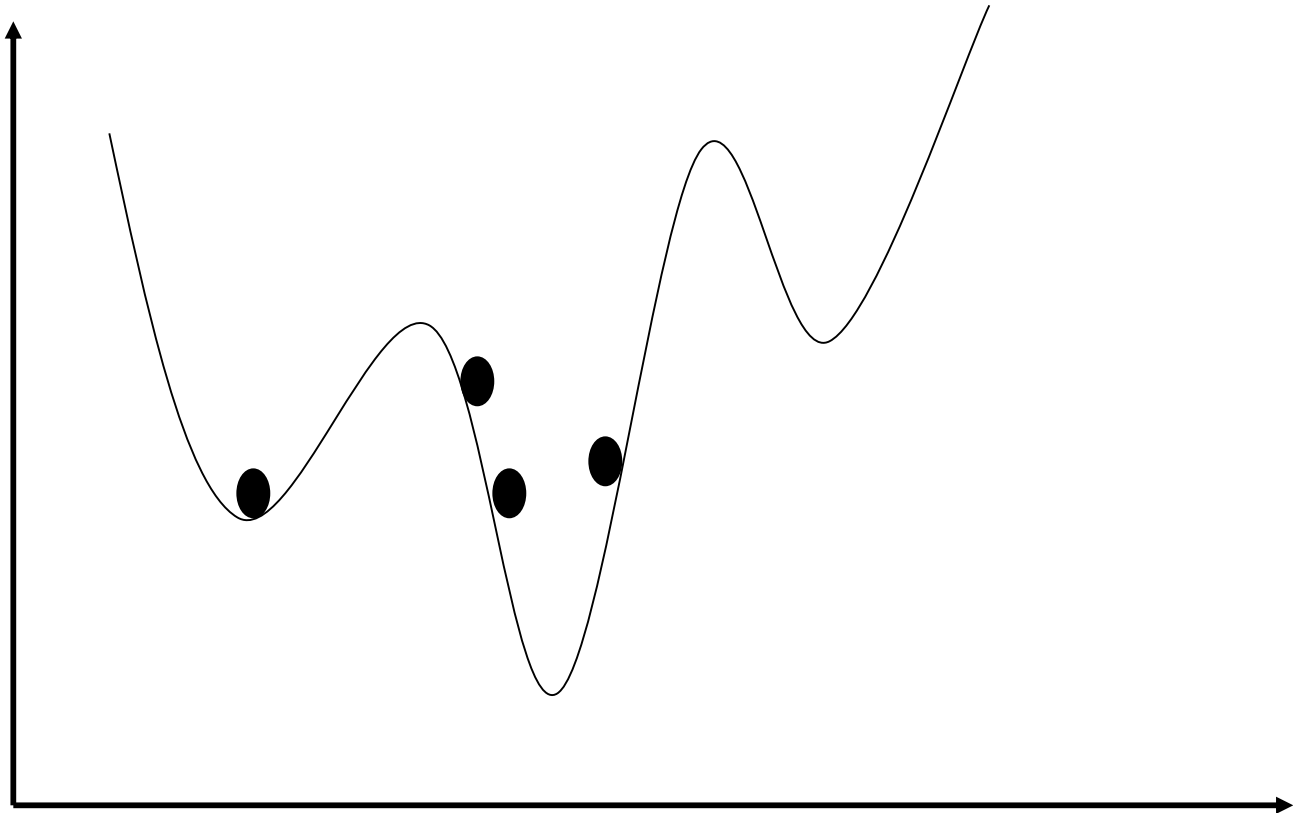
Genetic Algorithms



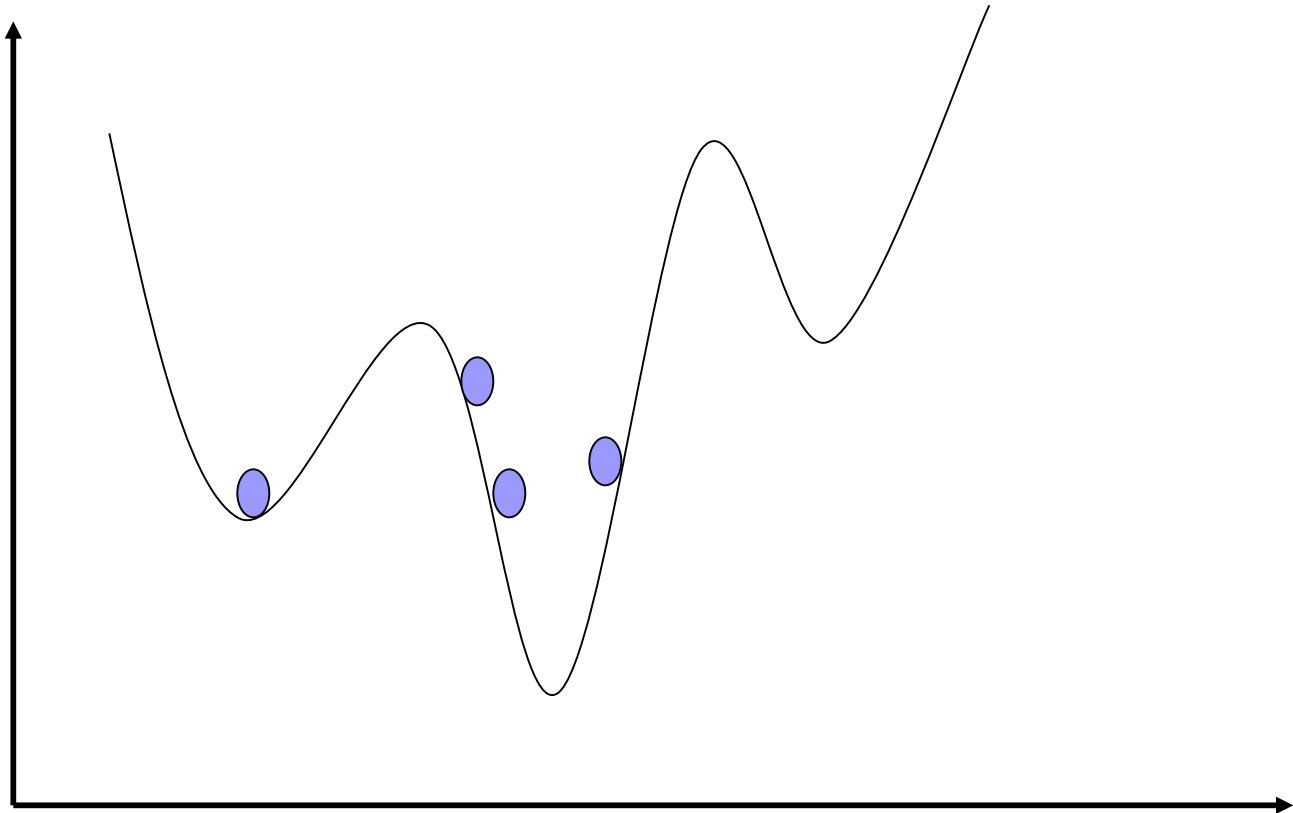
Genetic Algorithms



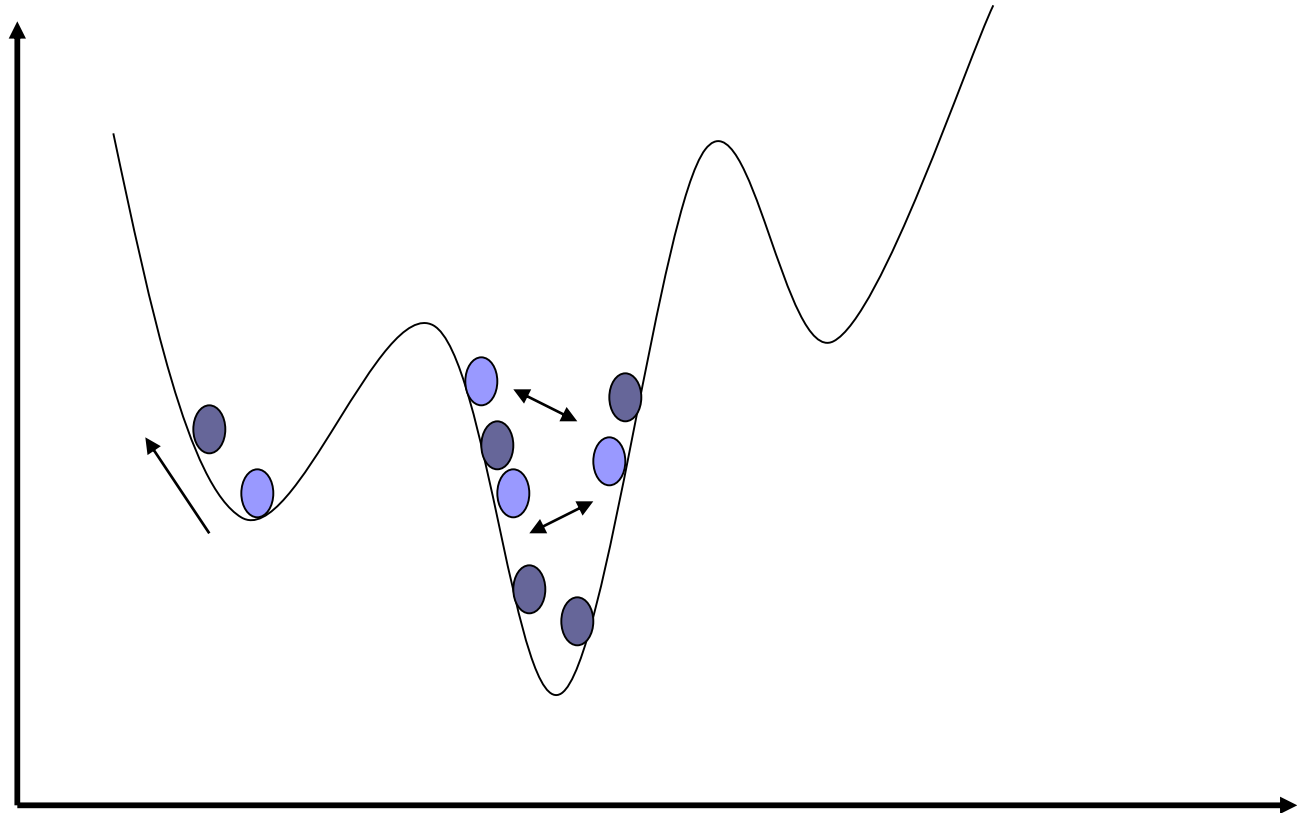
Genetic Algorithms



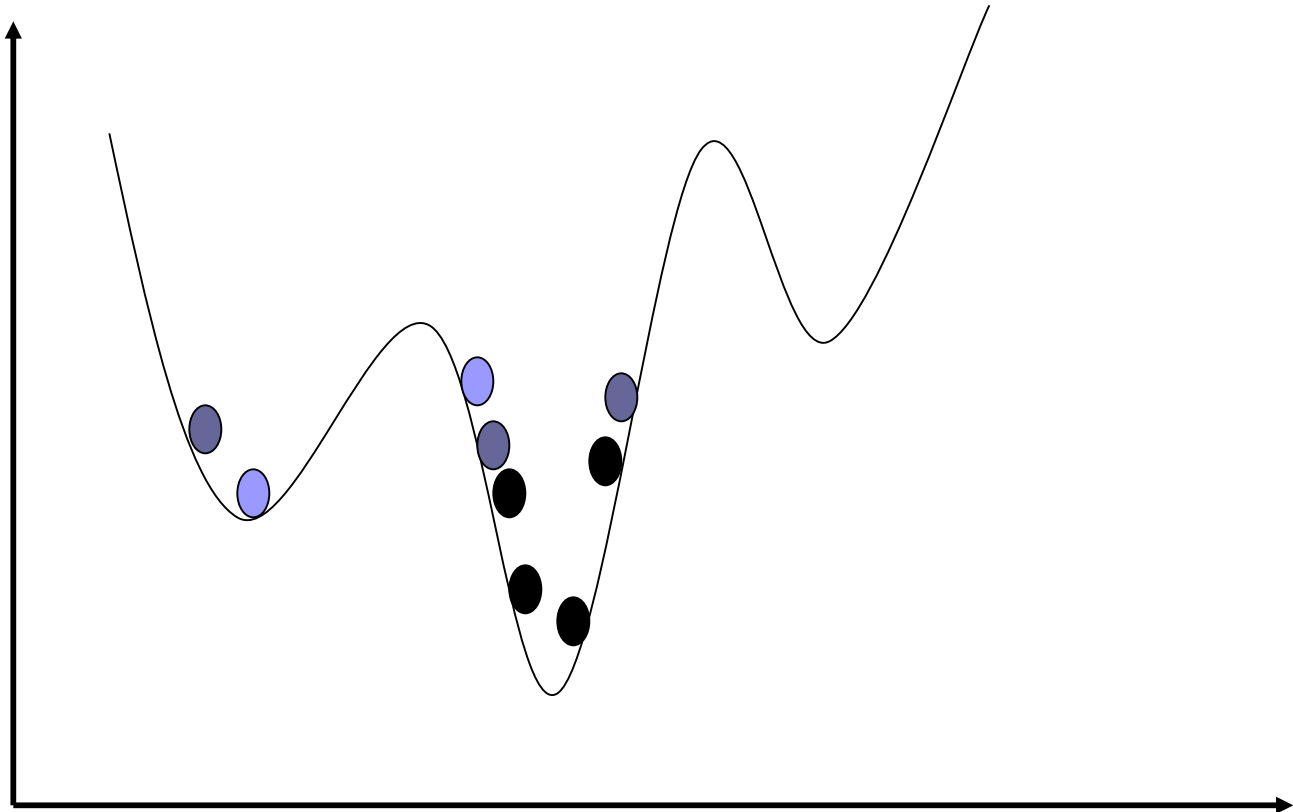
Genetic Algorithms



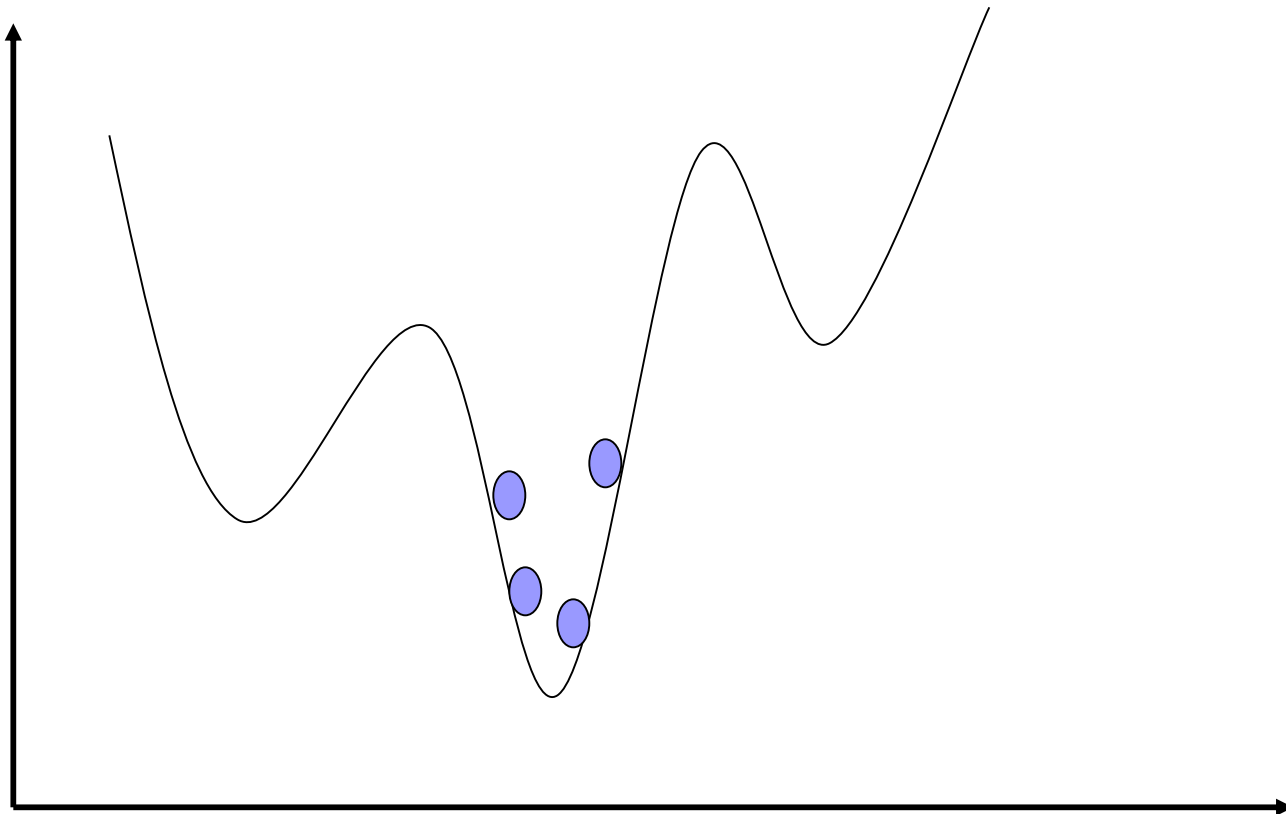
Genetic Algorithms



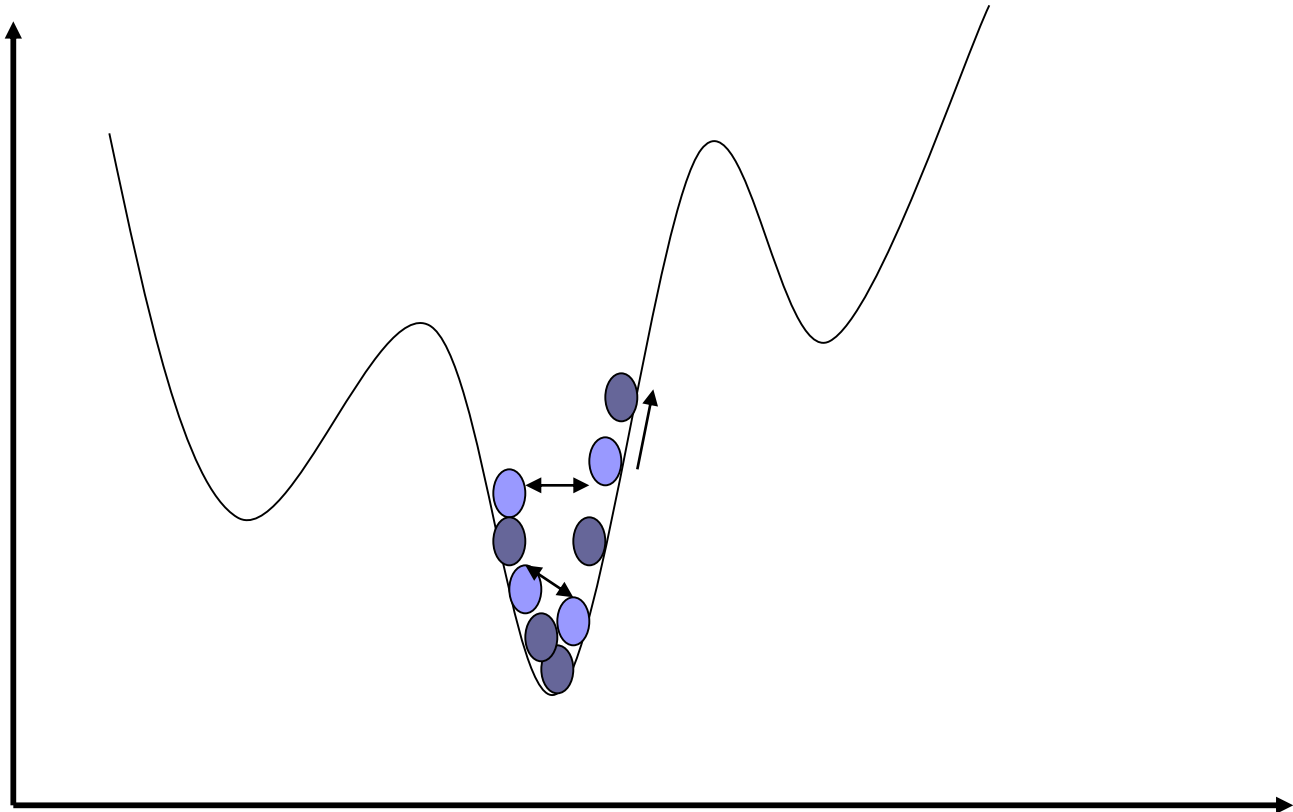
Genetic Algorithms



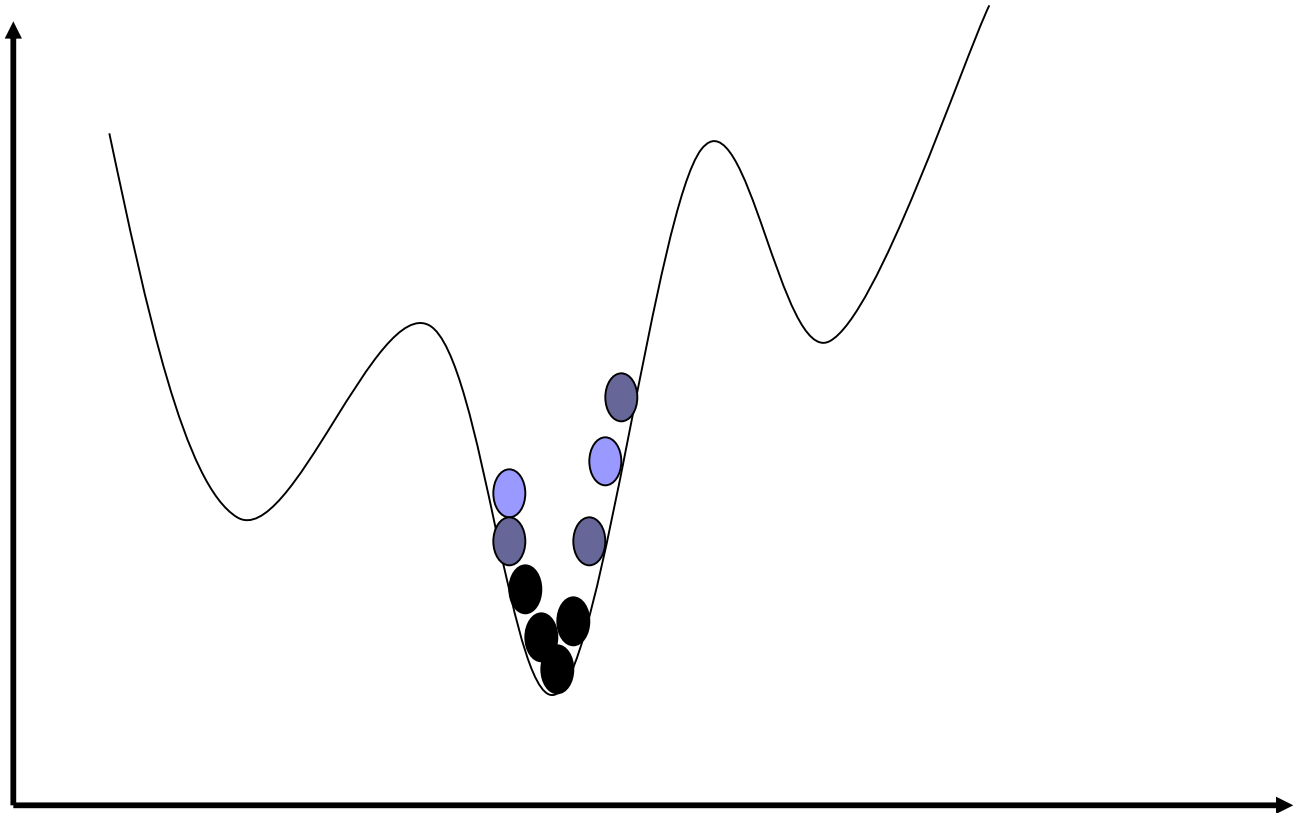
Genetic Algorithms



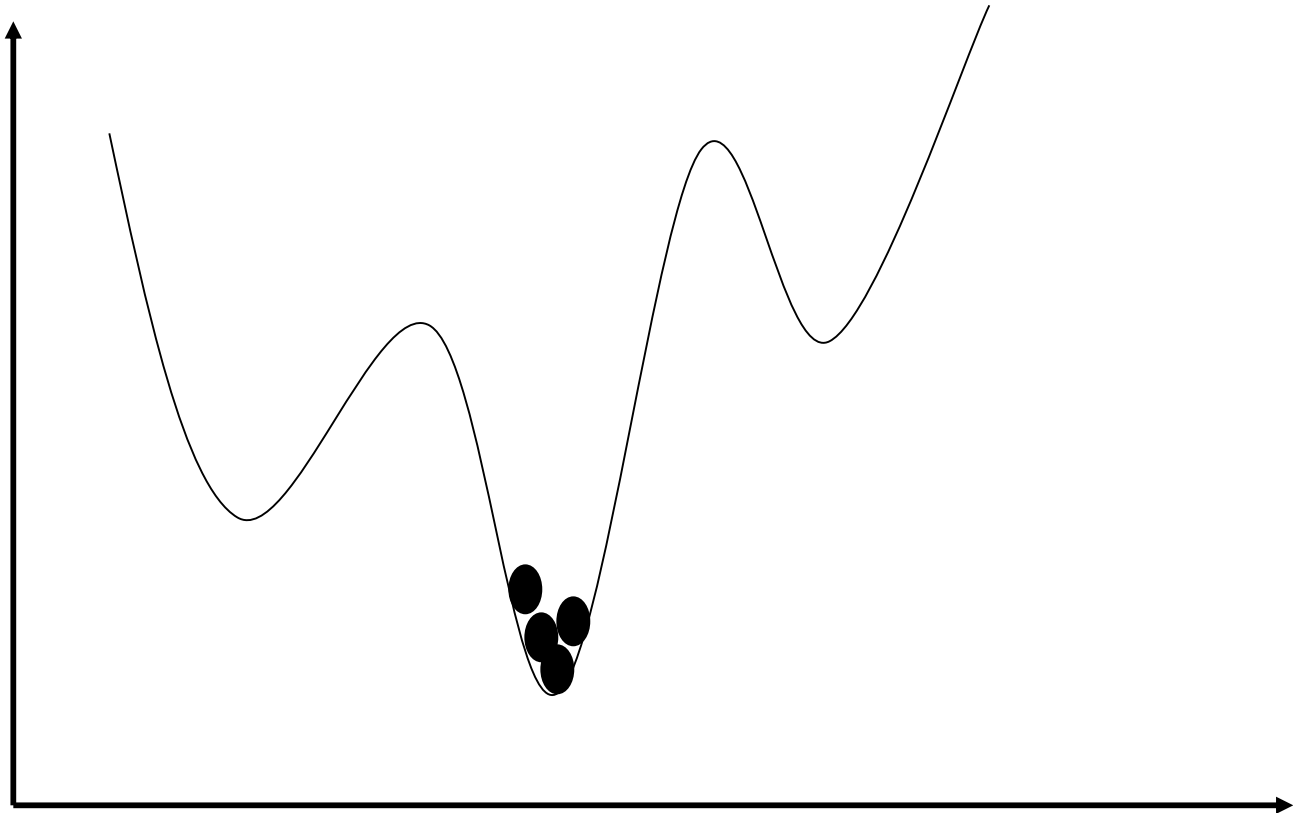
Genetic Algorithms



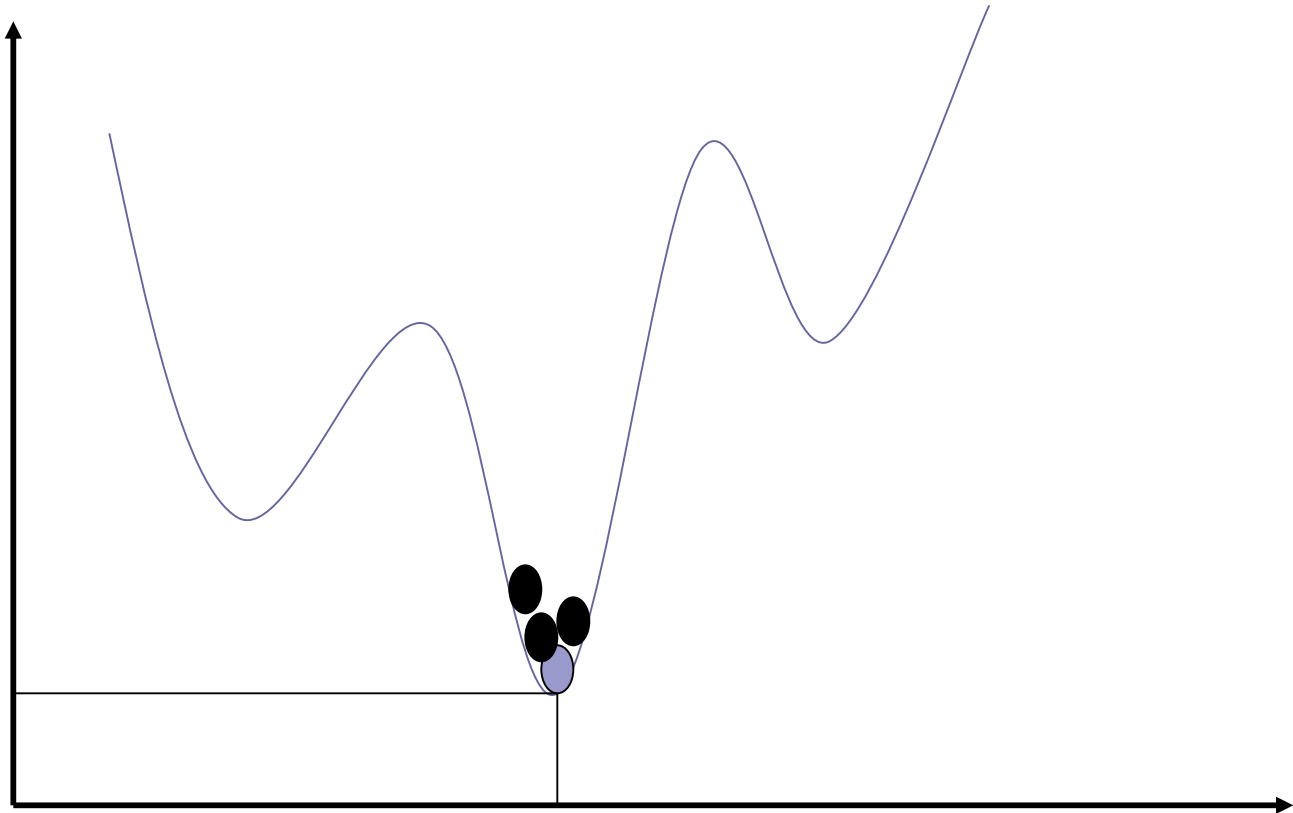
Genetic Algorithms



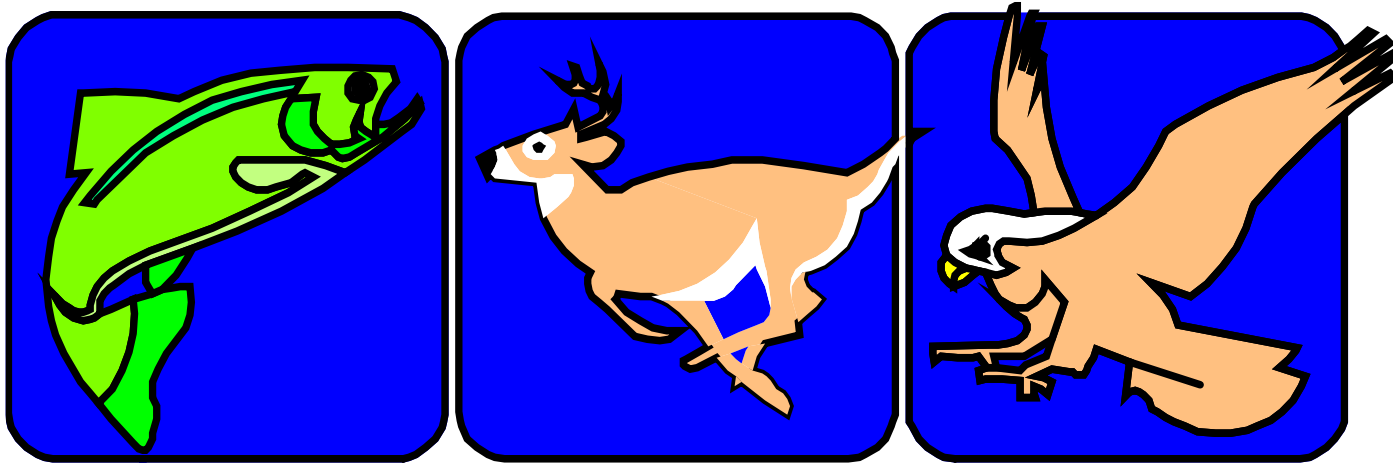
Genetic Algorithms



Genetic Algorithms



A Simple Example



“The Gene is by far the most sophisticated program around.”

- Bill Gates, *Business Week*, June 27, 1994



A Simple Example

The Traveling Salesman Problem:

Find a tour of a given set of cities so that

- each city is visited only once
- the total distance traveled is minimized

Representation

Representation is an ordered list of city numbers known as an *order-based GA*.

1) London	3) Dunedin	5) Beijing	7) Tokyo
2) Venice	4) Singapore	6) Phoenix	8) Victoria

CityList1 (3 5 7 2 1 6 4 8)

CityList2 (2 5 7 6 8 1 3 4)

Crossover

- Idea is to preserve relative order that elements occur
- Informal procedure:
 1. Choose an arbitrary part from the first parent
 2. Copy this part to the first child
 3. Copy the numbers that are not in the first part, to the first child:
 - starting right from cut point of the copied part,
 - using the **order** of the second parent
 - and wrapping around at the end
 4. Analogous for the second child, with parent roles reversed

Crossover

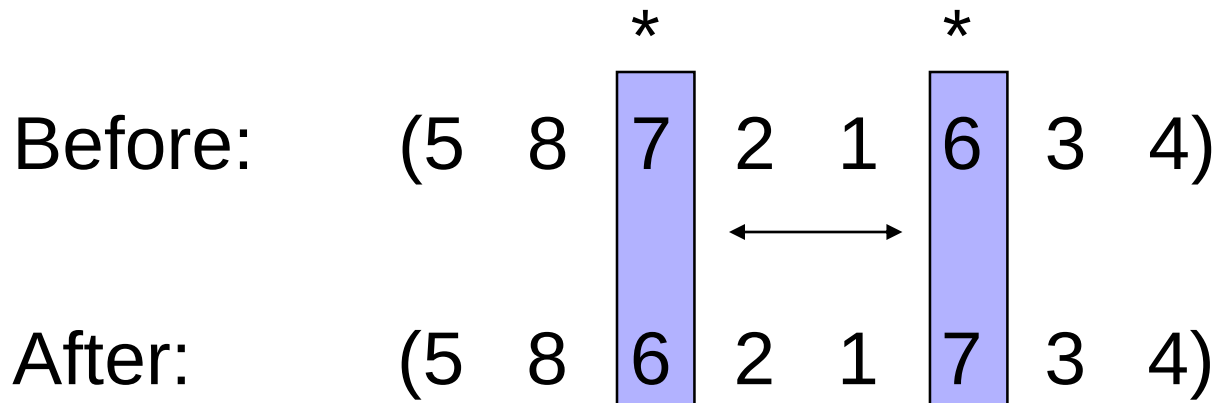
Crossover combines inversion and recombination:

			*		*		
Parent1	(3	5	7	2	1	6	4 8)
Parent2	(2	5	7	6	8	1	3 4)
Child	(5	8	7	2	1	6	3 4)

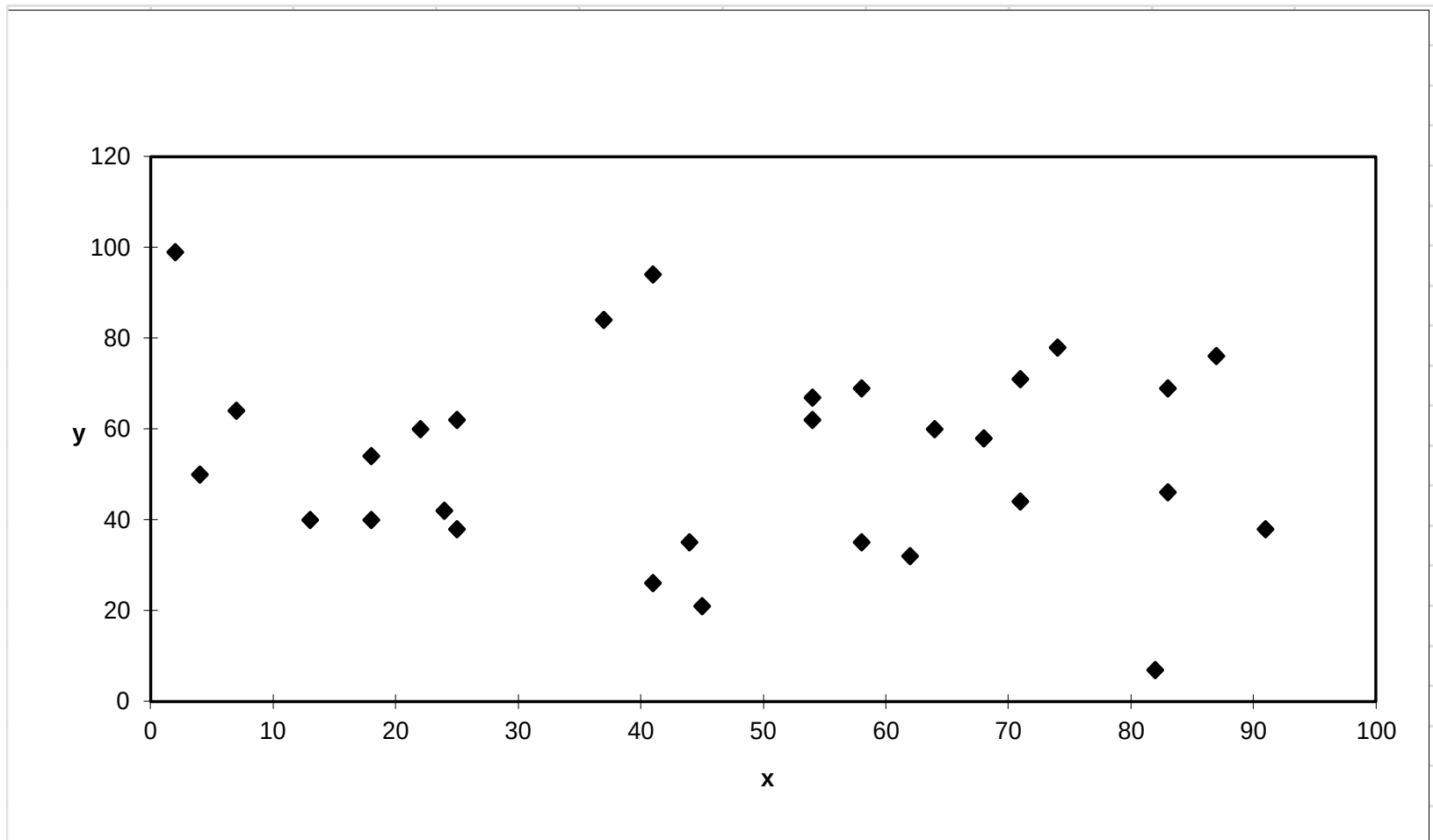
This operator is called the *Order1 crossover*.

Mutation

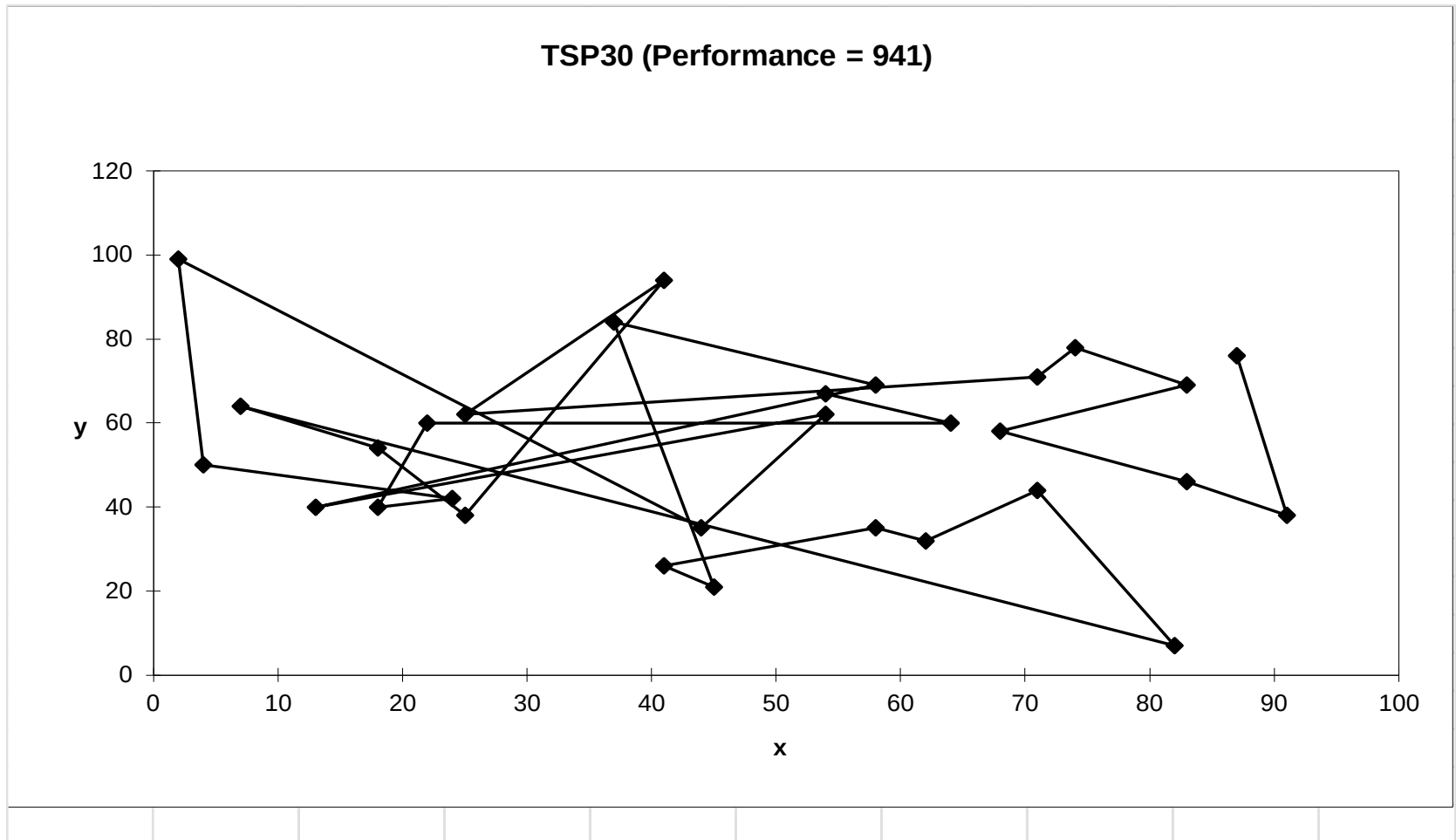
Mutation involves reordering of the list:



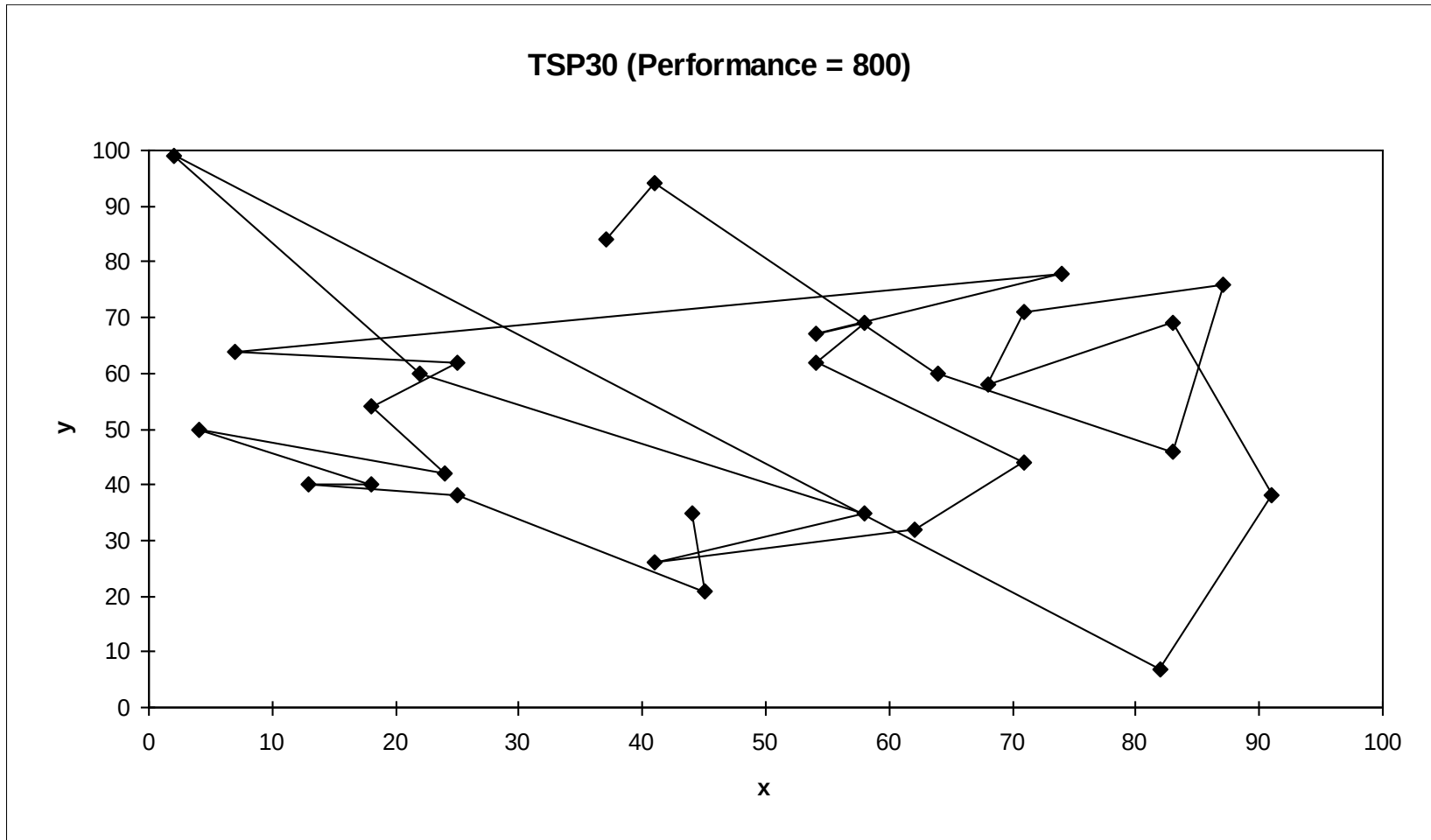
TSP Example: 30 Cities



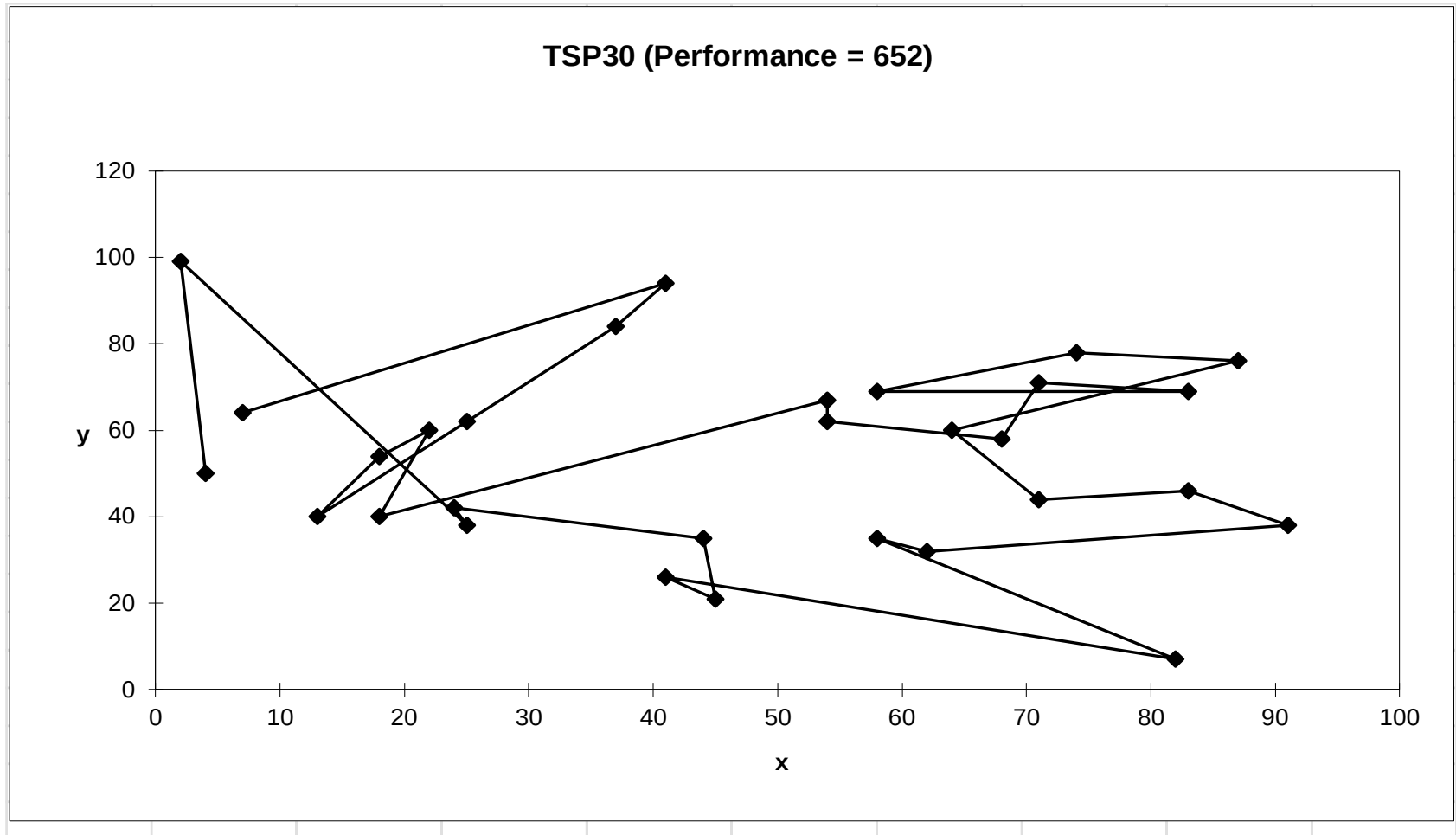
Solution _i (Distance = 941)



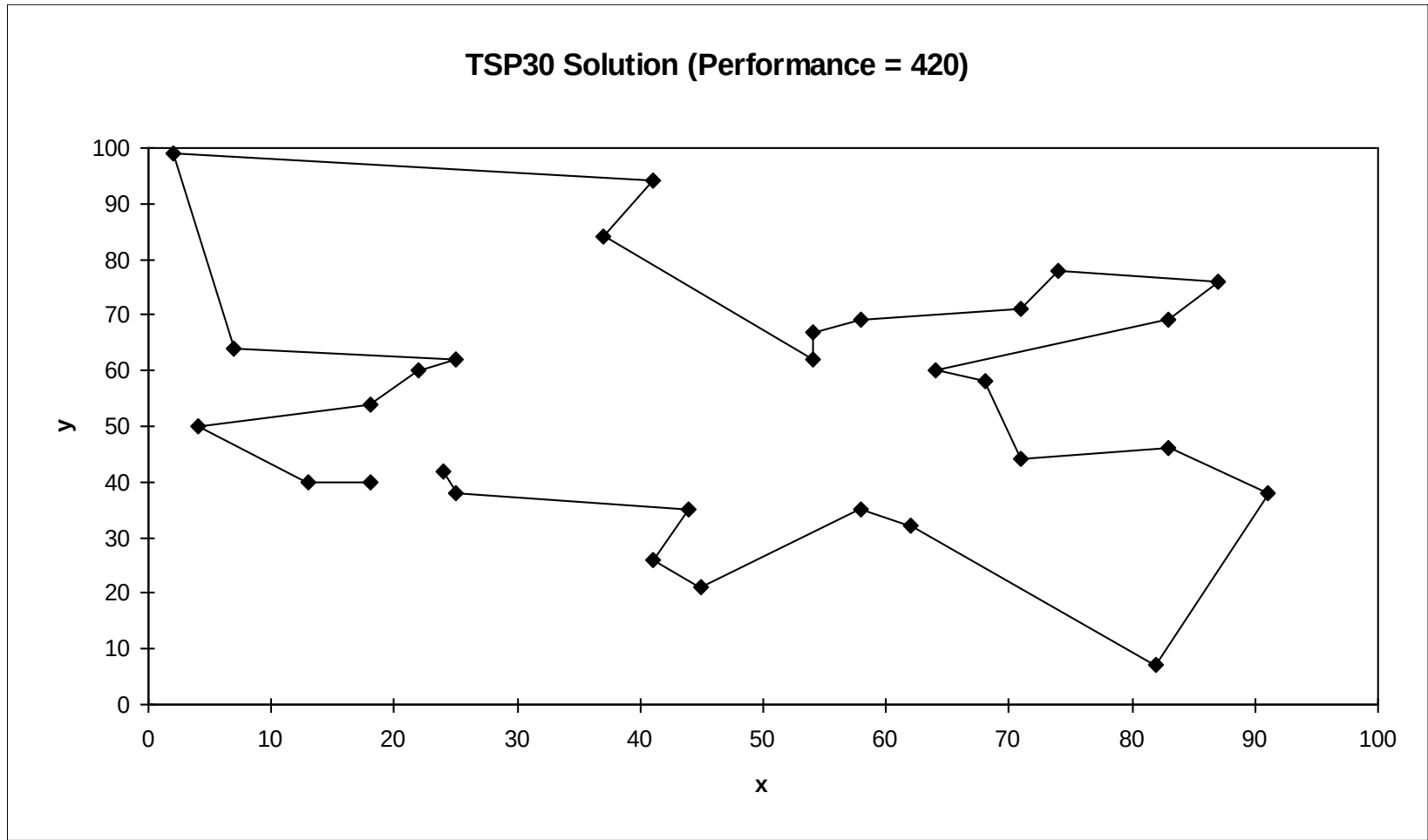
Solution j (Distance = 800)



Solution $_k$ (Distance = 652)

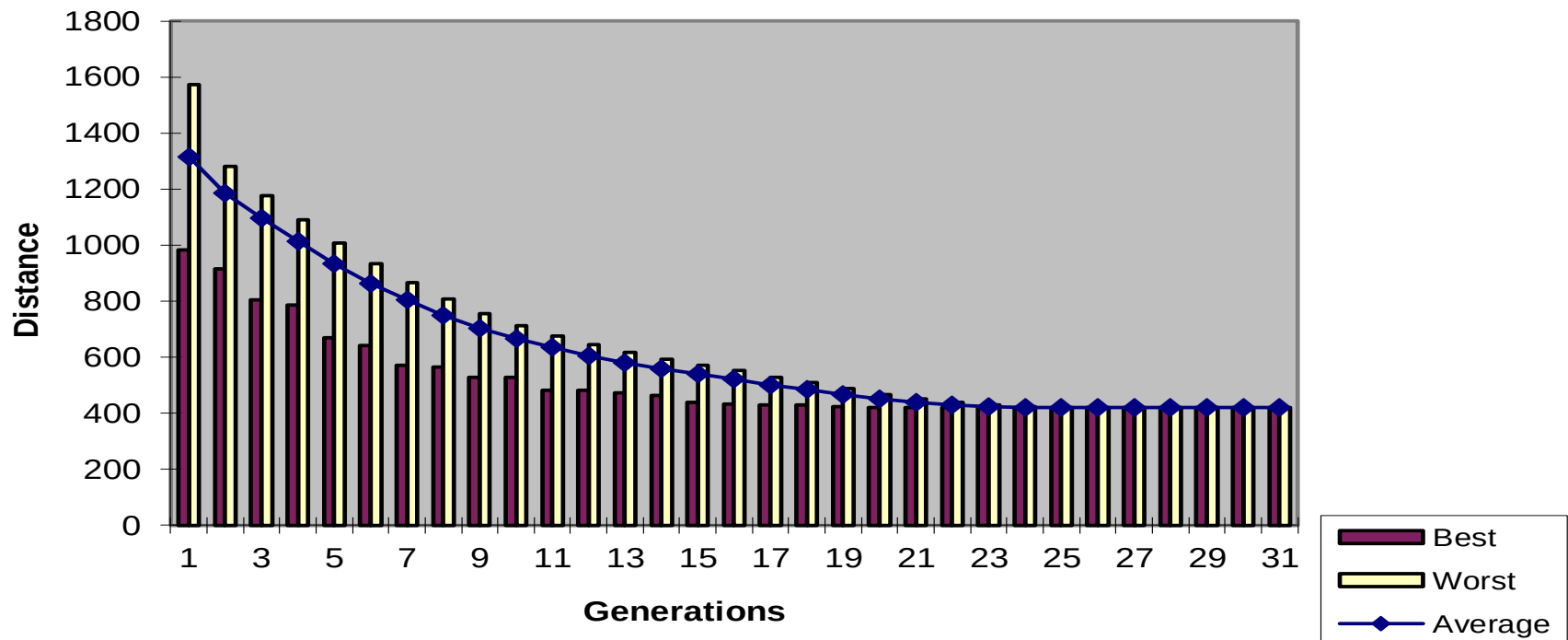


Best Solution (Distance = 420)

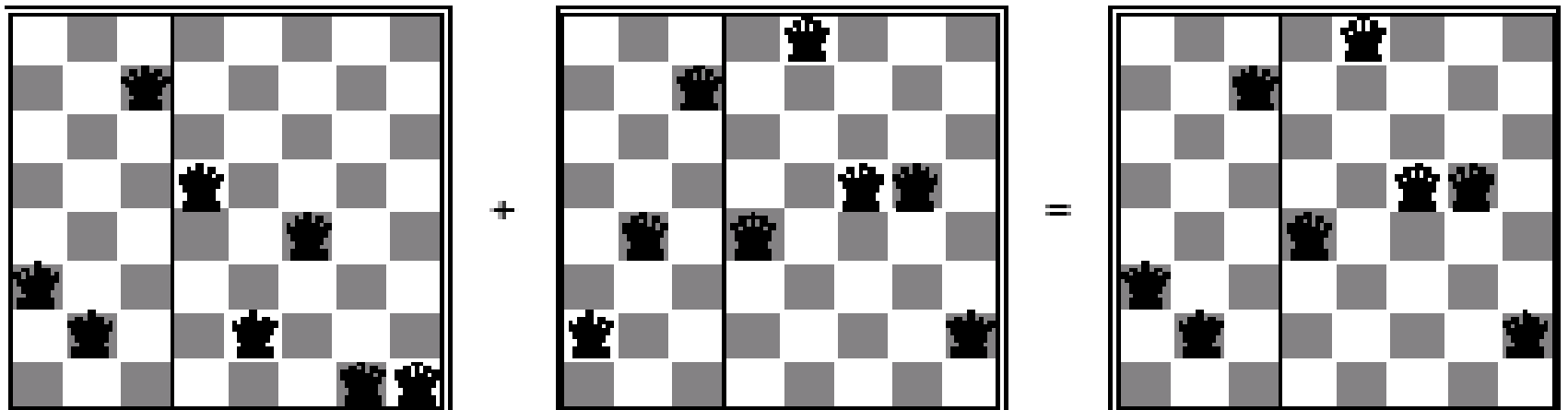
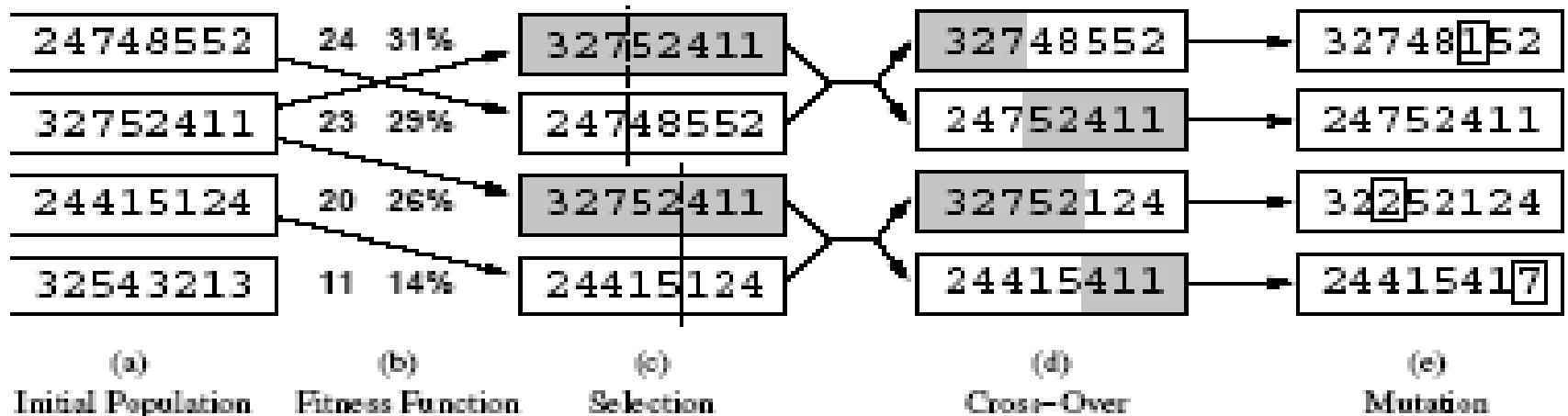


Overview of Performance

TSP30 - Overview of Performance



N-queens: Genetic Algorithms





Benefits of Genetic Algorithms

- Concept is easy to understand
- Modular, separate from application
- Supports multi-objective optimization
- Good for “noisy” environments
- Always an answer; answer gets better with time
- Inherently parallel; easily distributed

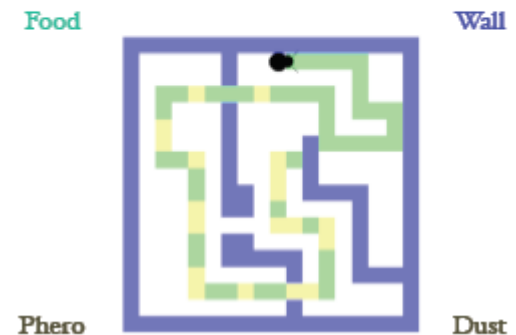
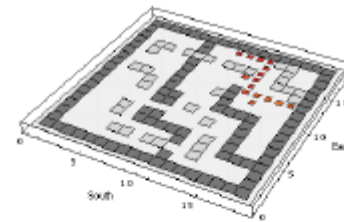
Genetic Programming

Genetic programming (GP)

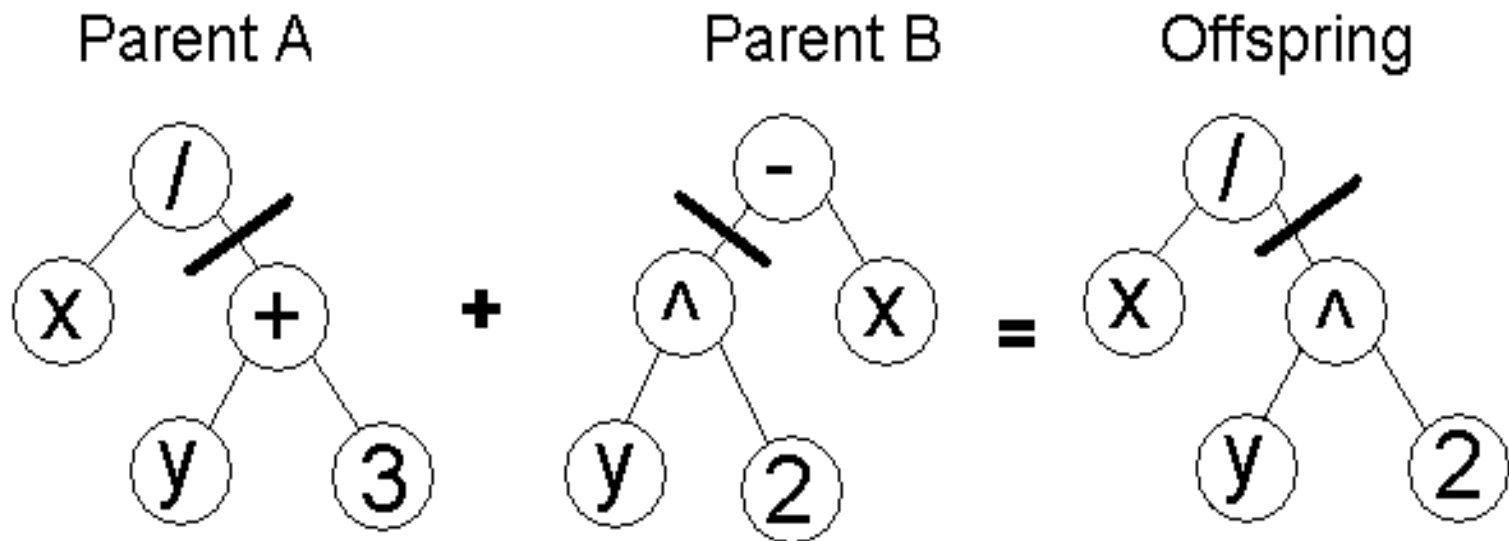
*Programming of Computers
by Means of Simulated
Evolution*

*How to Program a Computer
Without Explicitly Telling It
What to Do?*

*Genetic Programming is
Genetic Algorithms where
solutions are programs ...*



Genetic programming



It is possible to evolve whole programs like this but only small ones. Large programs with complex functions present big problems

Tổng kết

- Local search lưu trữ một số lượng nhỏ các nút tìm kiếm trong bộ nhớ, phù hợp với các bài toán mà lời giải là trạng thái của đích, không phải là đường đi tới đích (bài toán tối ưu tổ hợp)
- Các thuật toán điển hình: Hill climbing, simulated annealing, local beam search
- GA là một thuật giải áp dụng leo đồi cho một quần thể lời giải với các phép biến đổi đặc thù (crossover, mutation)
- Local search hiệu quả với các bài toán mà không gian tìm kiếm cực lớn và không có cách tiếp cận hệ thống (khó phân hoạch các vùng tìm kiếm)

Bài tập

- Cài đặt các thuật toán tìm kiếm cục bộ đã học (leo đồi, mô phỏng tôi thép)
- Giải TSP và/hoặc KP (xếp đồ vào ba lô) bằng GA
- Tìm hiểu Genetic programming
- Tìm hiểu các thuật toán tối ưu cục bộ khác
 - Tối ưu đàn kiến, tối ưu bầy đàn